# CORE-ET CPU Subsystem Micro-Architecture Reference

# Table of Contents


Document Control . . . 1
Document Author . . . 1
Revision History . . . 1
Source Material and Acknowledgements . . . 1
1. ET-SoC-1 Overview . . . 3
1.1. ET-SoC-1 at a Glance . . . 3
1.2. The ET-Minion Hierarchy . . . 3
1.2.1. ET-Minion Core . . . 4
1.2.2. Neighborhood . . . 5
1.2.3. Minion Shire . . . 7
1.2.4. Minion Shire Array . . . 7
1.3. ET-Maxion and Service Processor . . . 8
1.3.1. ET-Maxion Cores . . . 8
1.3.2. Service Processor . . . 8
1.4. Mesh Network-on-Chip . . . 9
1.4.1. Mesh Topology and Mesh Stops . . . 9
1.4.2. Memory Shires . . . 10
1.4.3. PCIe Shire . . . 10
1.4.4. I/O Shire . . . 10
1.5. Memory Map . . . 10
1.6. ET-SoC-1 Accelerator Card . . . 11
2. ET-Minion Core . . . 12
2.1. Core Overview . . . 12
2.1.1. Block Partitioning . . . 12
2.1.2. Hardware Threads . . . 13
2.1.3. Instruction Flow Through the Core . . . 13
2.1.4. Interfaces to the Neighborhood . . . 14
2.1.5. Clocks, Resets and Power . . . 14
2.1.6. Debug Support . . . 14
2.2. Front End . . . 14
2.2.1. Overview . . . 14
2.2.2. Interface . . . 16
2.2.3. Pipeline Stages . . . 21
2.2.4. Interaction with the L0 Micro-Cache . . . 22
2.2.5. Thread Buffer . . . 26
2.2.6. Entry Invalidation, Redirect and Thread Start . . . 31
2.2.7. Decoders . . . 33
2.2.8. Instruction FIFO . . . 34

2.2.9. Thread Scheduler . . . 35
2.2.10. Halt and Debug Program Buffer . . . 36
2.3. Integer Pipeline . . . 37
2.3.1. Overview . . . 37
2.3.2. Pipeline Stages . . . 37
2.3.3. Interfaces . . . 37
2.3.4. Register File and Operand Read . . . 37
2.3.5. Immediate Generation and ALU . . . 37
2.3.6. Multiply/Divide Unit . . . 38
2.3.7. Scoreboards and Hazards . . . 38
2.3.8. Hart Interleaving and Stalls . . . 38
2.3.9. Branches, Redirects and Replay . . . 38
2.3.10. Interface to the Data Cache . . . 38
2.3.11. Interface to the VPU . . . 38
2.3.12. CSRs and Privilege Modes . . . 38
2.3.13. Exceptions and Interrupts . . . 38
2.3.14. Debug and Breakpoints . . . 38
2.3.15. Performance Counters . . . 38
2.3.16. Verification Notes . . . 38
2.4. Data Cache . . . 38
2.4.1. Overview . . . 38
2.4.2. Organization and Operating Modes . . . 38
2.4.3. Interfaces . . . 38
2.4.4. Request Pipeline . . . 38
2.4.5. Address Translation and PMA Checks . . . 39
2.4.6. Tag, Data and LRU Arrays . . . 39
2.4.7. Miss Handling . . . 39
2.4.8. Replay Queue . . . 39
2.4.9. Writeback and Store Merge . . . 39
2.4.10. Atomic Operations . . . 39
2.4.11. Cache-Management Operations . . . 39
2.4.12. Scratchpad Control . . . 39
2.4.13. Tensor Load and Reduce . . . 39
2.4.14. Interface to the Neighborhood . . . 39
2.4.15. Errors . . . 39
2.4.16. Performance . . . 39
2.4.17. Verification Notes . . . 39
2.5. Vector Processing Unit . . . 39
2.5.1. Overview . . . 39
2.5.2. Interfaces . . . 39
2.5.3. Control and Micro-Instruction Decode . . . 39

2.5.4. Register Files . . . 39
2.5.5. Lanes . . . 39
2.5.6. Mask Unit . . . 39
2.5.7. Bypass Network . . . 40
2.5.8. Shift and Swizzle Unit . . . 40
2.5.9. TxFMA Unit . . . 40
2.5.10. Integer Multiply-Accumulate Unit . . . 40
2.5.11. Transcendental Unit . . . 40
2.5.12. Tensor Operations . . . 40
2.5.13. Interfaces to the Integer Pipeline and Data Cache . . . 40
2.5.14. Performance . . . 40
2.5.15. Verification Notes . . . 40
3. Neighborhood . . . 41
3.1. Neighborhood Overview . . . 41
3.1.1. Partitioning . . . 41
3.1.2. Shared Resources and Core Connections . . . 41
3.1.3. Voltage-Domain Crossings . . . 41
3.1.4. External Interfaces . . . 42
3.2. Neighborhood Instruction Cache . . . 42
3.2.1. Structure and Connections . . . 43
3.3. Front End – I-Cache Interface . . . 44
3.3.1. Overview . . . 44
3.3.2. Interface Signals . . . 45
3.3.3. Request Arbitration . . . 46
3.3.4. Request and Response Timing . . . 47
3.3.5. Miss, Sleep and Fill-Done Protocol . . . 49
3.3.6. Fault and Error Reporting . . . 50
3.3.7. Complete Path for Four Cores . . . 51
3.3.8. Livelock and Starvation Avoidance . . . 53
3.3.9. Known Limitations . . . 59
3.4. L0 Micro-Cache . . . 59
3.4.1. Overview . . . 59
3.4.2. Interface . . . 61
3.4.3. Cache Organization and Address Decode . . . 66
3.4.4. Pipeline Stages . . . 68
3.4.5. Instruction TLB . . . 72
3.4.6. Micro-Tag and Micro-Data Arrays . . . 82
3.4.7. PMA Check . . . 85
3.4.8. Miss Handling . . . 86
3.4.9. Flush . . . 91
3.4.10. Bypass Mode . . . 92

3.4.11. Code Prefetcher . . . 97
3.4.12. Debug Access . . . 103
3.5. L1 Instruction Cache . . . 104
3.5.1. Overview . . . 104
3.5.2. Organization . . . 104
3.5.3. Tag, LRU and Data Arrays . . . 104
3.5.4. External Data RAM . . . 104
3.5.5. Request Arbitration Between L0 Micro-Caches . . . 104
3.5.6. Miss Path . . . 104
3.5.7. Errors and ECC . . . 104
3.6. Shared Page-Table Walker . . . 104
3.6.1. Clients and Arbitration . . . 104
3.6.2. Walk Sequence and Page Sizes . . . 104
3.6.3. PTE Cache and Invalidation . . . 105
3.7. Memory Request and Response Path . . . 105
3.7.1. Miss, Evict and Fill FIFOs . . . 105
3.7.2. Request Arbitration . . . 105
3.7.3. Response Routing and Backpressure . . . 105
3.8. Tensor Support . . . 105
3.8.1. Cooperative Tensor Load . . . 105
3.8.2. Tensor Store Buffer . . . 105
3.8.3. TBOX Router . . . 105
3.9. Synchronization, Messaging, Registers and PMU . . . 105
3.9.1. Fast Local Barrier . . . 105
3.9.2. Local Message Network . . . 105
3.9.3. IPI and FCC Paths . . . 105
3.9.4. Neighborhood Registers and APB Access . . . 105
3.9.5. Debug Channel . . . 105
3.9.6. Performance Monitoring Unit . . . 105

# Document Control

## Document Author

Ryan Naveed, 10xEngineers — ryan.naveed@10xengineers.ai

## Revision History

| Version | Date | Author | Notes |
|---|---|---|---|
| v0.1 | 2026-10-06 | Ryan Naveed | • First draft of the document.<br>• Chapter 1, ET-SoC-1 Overview: written, except the Memory Map, which contains a heading only.<br>• Chapter 2, ET-Minion Core: Core Overview (block partitioning, hardware threads, instruction flow) and Front End written. Integer Pipeline, Data Cache and VPU contain headings only.<br>• Chapter 3, Neighborhood: the Overview and the Front End — I-Cache Interface are written. The remaining sections contain headings only. |

## Source Material and Acknowledgements

The original micro-architecture documents are in `docs/uarch/` of this repository, one per module. Where a document and the RTL differ, the RTL is followed.

The engineers who worked on each module are listed below.

| Engineer | Module documents |
|---|---|
| Albert Navarro | FE/Intpipe Description |
| Francisco Torres | Minion DCache Description |
| Guillem Solé | Cooperative TensorLoad Description |
| Ildefonso Gomariz | CORE-ET Neighborhood MAS; Neighborhood I-Cache Description; Front End — I-Cache Interface; CORE-ET Minion Shire Description; Cooperative TensorLoad Description; ET-Link Specification |
| Jordi Muñoz | Minion VPU Specification |
| Juanjo Galindo | CORE-ET Minion Shire Description; Minion Shire DLL Constraints |
| Miquel Izquierdo | ET-Link Specification |
| Sebastià Tortella | Minion Description; FE/Intpipe Description; ET-Link Specification |
| Sri Samudrala | Minion VPU Specification |

| Engineer | Module documents |
| --- | --- |
| Xavier Revés | CORE-ET Neighborhood MAS; Minion Description; Minion DCache Description; CORE-ET Minion Shire Description; Minion Shire DLL Constraints; ET-Link Specification |

# 1. ET-SoC-1 Overview

## 1.1. ET-SoC-1 at a Glance

ET-SoC-1 is a many-core RISC-V system-on-chip developed primarily for artificial-intelligence and machine-learning (AI/ML) workloads. It is designed as a flexible, fast and efficient inference engine for ML applications. Its compute resources are arranged in a multi-level hierarchy that delivers high sustained performance on regular, parallelizable workloads, such as those of ML algorithms.

The device integrates 1,093 general-purpose 64-bit RISC-V cores of three types:

- 1,088 ET-Minion cores for ML computation
- 4 ET-Maxion cores for management and general-purpose tasks
- 1 Service Processor for boot and device management

The cores are connected to on-die SRAM, external memory and a set of standard interfaces through a mesh network-on-chip (NoC). Table 1 summarizes the device.

*Table 1. ET-SoC-1 key characteristics*

| Item | Description |
|---|---|
| ET-Minion cores | 1,088 in-order, dual-threaded RV64IMFC cores, each with an 8-lane vector/tensor unit |
| ET-Maxion cores | 4 single-threaded, superscalar, out-of-order 64-bit RISC-V cores |
| Service Processor | 1 single-threaded core derived from the ET-Minion |
| On-die SRAM | Each Minion Shire's 4 MB Shire Cache can be partitioned by ESRs into L2, a slice of the chip-wide L3, and scratchpad. |
| Interconnect | Mesh NoC with 44 mesh stops |
| External memory | LPDDR4X, 16 channels x 16 bits, up to 4,266 MT/s |
| PCI Express | Gen4, 8 lanes. Endpoint, root complex, or both at once. |
| Other interfaces | eMMC, USB 2.0 OTG, I2C/SMBus, I3C, SPI, UART, GPIO. Debug access through JTAG and a USB 2.0 device port. |

## 1.2. The ET-Minion Hierarchy

The ET-Minion cores are organized in three levels, as shown in Figure 1:

- **ET-Minion core**
- **Neighborhood**: 8 ET-Minion cores
- **Minion Shire**: 4 Neighborhoods (32 ET-Minion cores)

Each level adds resources that are shared by the units it contains. The device contains 34 Minion Shires, which together form the Minion Shire array (Section 1.2.4).

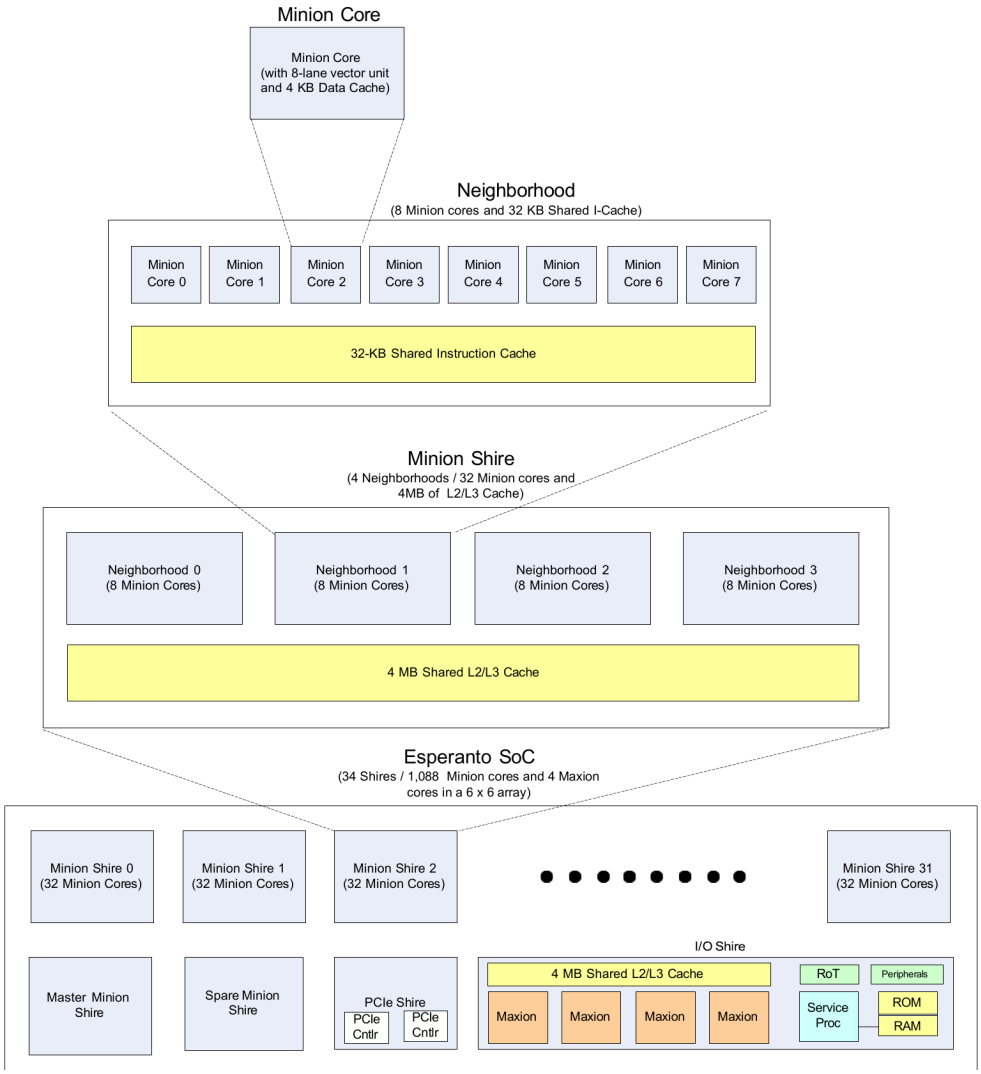



*Figure 1. ET-Minion hierarchy*

### 1.2.1. ET-Minion Core

The ET-Minion is a dual-threaded, in-order, single-issue 64-bit RISC-V core that implements RV64IMFC. Its main features are:

- Two hardware threads (harts)
- 4 KB private L1 data cache, partly configurable as scratchpad memory
- Vector processing unit (VPU) with eight identical lanes operating in lock-step. Each lane contains:
  - Floating-point multiply-add unit (FMA): one 32-bit or two 16-bit operations
  - Two integer multiply-add units (IMA): four 8-bit multiply-accumulates each
  - Integer unit (INT)
  - Transcendental unit (TRANS)
- Esperanto instruction extensions for vector, tensor, atomic, message, cache-management and fast-synchronization operations

The core consists of four top-level blocks, shown in Figure 2:

- **Front end (FE)**
- **Integer pipeline**
- **L1 data cache and scratchpad**
- **VPU**

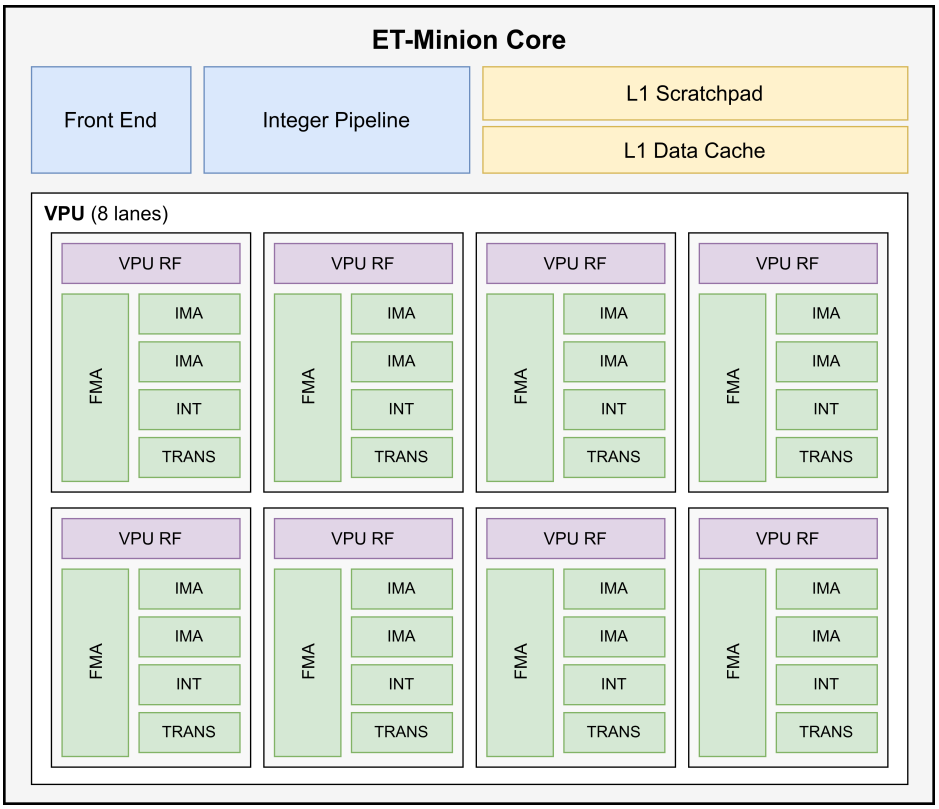



*Figure 2. ET-Minion core*

## 1.2.2. Neighborhood

A Neighborhood groups eight ET-Minion cores that share the following resources. In Figure 3, the L0 micro-caches are labeled "I-Cache".

A Neighborhood also spans two supply-voltage regions. The cores and most of the Neighborhood logic are in the low-voltage region, while the interface towards the Shire Channel and the memory cells that need speed, among them the data RAMs of the L1 instruction cache, are in the high-voltage region (Section 3.1.3).

- **L1 instruction cache**: 32 KB, organized as 128 sets × 4 ways with 64-byte lines. It is shared by all eight cores, and its data RAMs are located outside the Neighborhood.
- **L0 micro-caches**: two, each shared by four cores. Each holds 16 fully associative cache lines, sits between its cores and the L1 instruction cache, and accesses the L1 instruction cache on a miss.
- **Page-table walkers (PTWs)**: two, each shared by four cores. Each PTW serves the data caches of its four cores and the instruction-cache side.
- **Request/response path**: a single path to the memory hierarchy outside the Neighborhood. Requests from the eight data caches, the instruction cache, the two PTWs and, the cooperative TensorLoad/TensorStore units are arbitrated into (256-bit internally which is converted to a 512-bit ET-Link bus).

- **Performance monitoring unit (PMU)**: twelve 64-bit event counters shared by the eight cores. Eight counters count ET-Minion events and four count Neighborhood events.
- **Cooperative TensorLoad**: coalesces the TensorLoad requests that several cores issue for the same address into a single memory request, and delivers the one response back to every cooperating core. Cores in all four Neighborhoods of a Minion Shire can cooperate; one Neighborhood acts as master and sends the combined request.
- **Cooperative TensorStore**: combines the partial writes of cooperating cores into one Shire Cache write, and replicates the acknowledgement to every core that took part. Cores cooperate in two fixed groups, cores 0 to 3 and cores 4 to 7, in one of three modes:
  - **Quad-128**: 128 bits from each of four cores, forming a full 512-bit cache line
  - **Pair-256**: 256 bits from each of two cores, forming a full 512-bit cache line
  - **Pair-128**: 128 bits from each of two cores, forming half a cache line

  Each core holds its request until every core of the group presents one, so operations of different modes cannot mix.
- **Fast Local Messaging Network (FLN)**: carries messages between cores of the same Neighborhood directly, instead of sending them out through the request path and back in through the response path. Connections are not all-to-all; they follow the pattern used by the tensor reduction instructions, and a message arriving over this network takes priority over the other responses to that core.
- **Fast local barrier (FLB) path**
- **Neighborhood ESRs**
- **Interrupts**: routes the external, timer and software interrupt lines to the cores, and delivers the inter-processor interrupts (IPIs). A software write to a Shire ESR raises a machine software interrupt for a hart, or a redirect IPI that sends the selected harts to the PC held in the Neighborhood `ipi_redirect_pc` ESR.

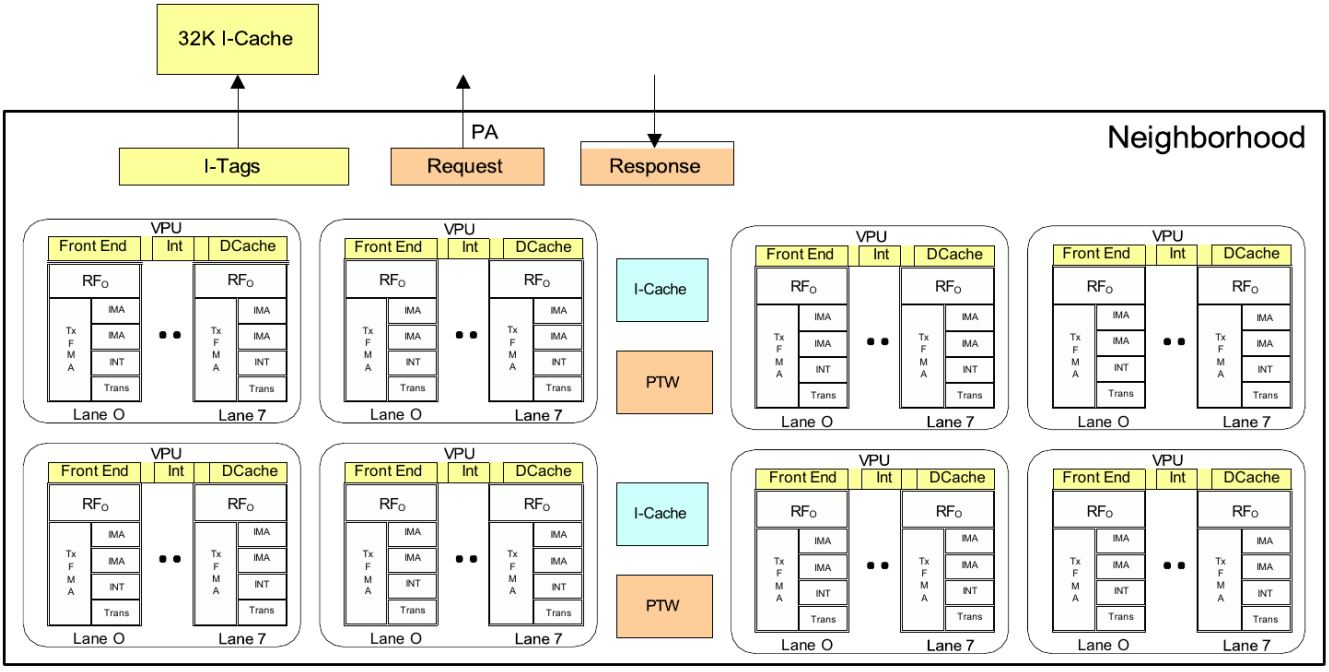



*Figure 3. Neighborhood*

### 1.2.3. Minion Shire

A Minion Shire groups four Neighborhoods (Figure 4). It also contains:

- **Shire Cache**: 4 MB, in four 1 MB banks. Programmable ESRs divide the cache into three kinds of partition:
  - L2 cache, private to the Shire
  - Slice of the chip-wide distributed L3 cache
  - Scratchpad memory
- **Crossbar**: Request and response crossbars connect four Neighborhoods and the RBOX port to the four Shire Cache banks and the UC block. Each Neighborhood has one request and response port into the Shire Cache, and each bank accepts at most one request per cycle. The crossbar also connects the Neighborhoods to the un-cached (UC) block.
- **Uncached (UC) block**: receives the Neighborhood requests that are not sent to the Shire Cache banks. It handles uncacheable accesses, contains the fast local barrier (FLB) and fast local credit counter (FCC) units, and delivers messages sent between ET-Minion cores of different Shires.
- **ESR banks**: the Esperanto System Registers (ESRs) configure the blocks in the Shire.
- **NoC interface**: L2 misses and uncacheable accesses leave the Shire here, bound for other Shires or the memory controllers.

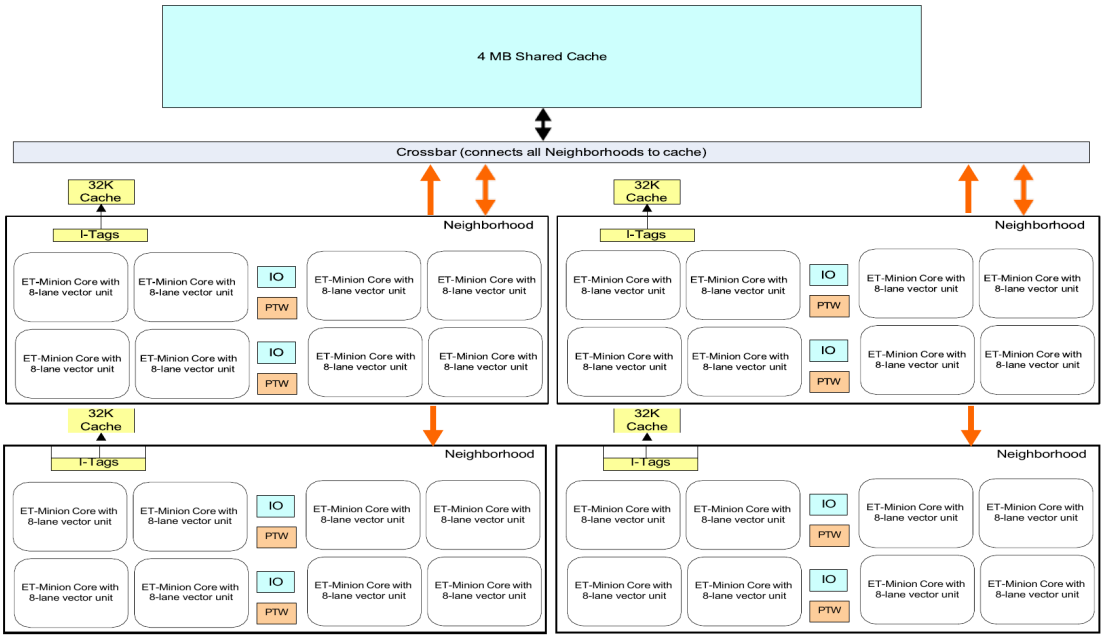



*Figure 4. Minion Shire*

### 1.2.4. Minion Shire Array

The 34 Minion Shires, 1,088 ET-Minion cores in total, form the Minion Shire array. The roles of the Shires are assigned by software:

- **Compute**: 32 Minion Shires operate in parallel as the compute array.
- **Master (management)**: one Minion Shire receives work and distributes it to the compute array.

- **Spare**: one Minion Shire.

All Minion Shires are identical in hardware, so any Minion Shire can take any of these roles.

# 1.3. ET-Maxion and Service Processor

In addition to the ET-Minion cores, the device contains two other core types. Both are located in the I/O Shire (Section 1.4.4). This document does not describe them beyond this section.

## 1.3.1. ET-Maxion Cores

The ET-Maxion is a single-threaded, superscalar, out-of-order 64-bit RISC-V core intended for management and general-purpose tasks. The device contains four identical ET-Maxion cores. Together with a 4 MB shared L2/L3 cache they form the Maxion Neighborhood, and a system bus and coherency hub in the I/O Shire connects them. Figure 5 shows the ET-Maxion pipeline.

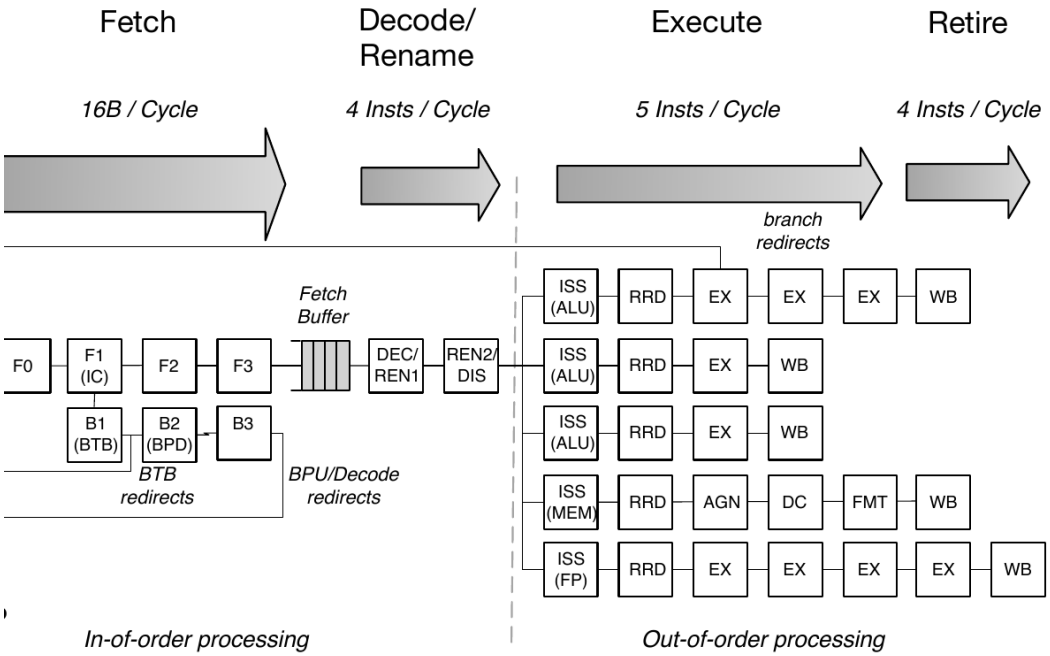



*Figure 5. ET-Maxion pipeline*

## 1.3.2. Service Processor

The Service Processor (SP) is a single-threaded ET-Minion core without the vector/tensor extensions. It has its own ROM and a 1 MB scratchpad SRAM, but none of the Neighborhood or Shire shared resources.

- **Boot.** The SP is the first major block released from reset. It executes the boot code from ROM and initializes the device interfaces, including DDR, PCIe and USB.
- **Run time.** It provides management services for the whole device.
- **Access.** The SP can access every resource on the device, and some resources are accessible only to it.

The I/O Shire also contains a hardware root of trust, and six secure mailboxes for communication

between four agents:

- Service Processor
- Master Minion Shire
- ET-Maxions
- PCIe host

## 1.4. Mesh Network-on-Chip

The mesh NoC is the top-level interconnect of ET-SoC-1 and connects all Shires. Figure 6 shows the device with the Shire array on the mesh, the external memory and PCIe interfaces, and the peripheral, Service Processor and debug networks.

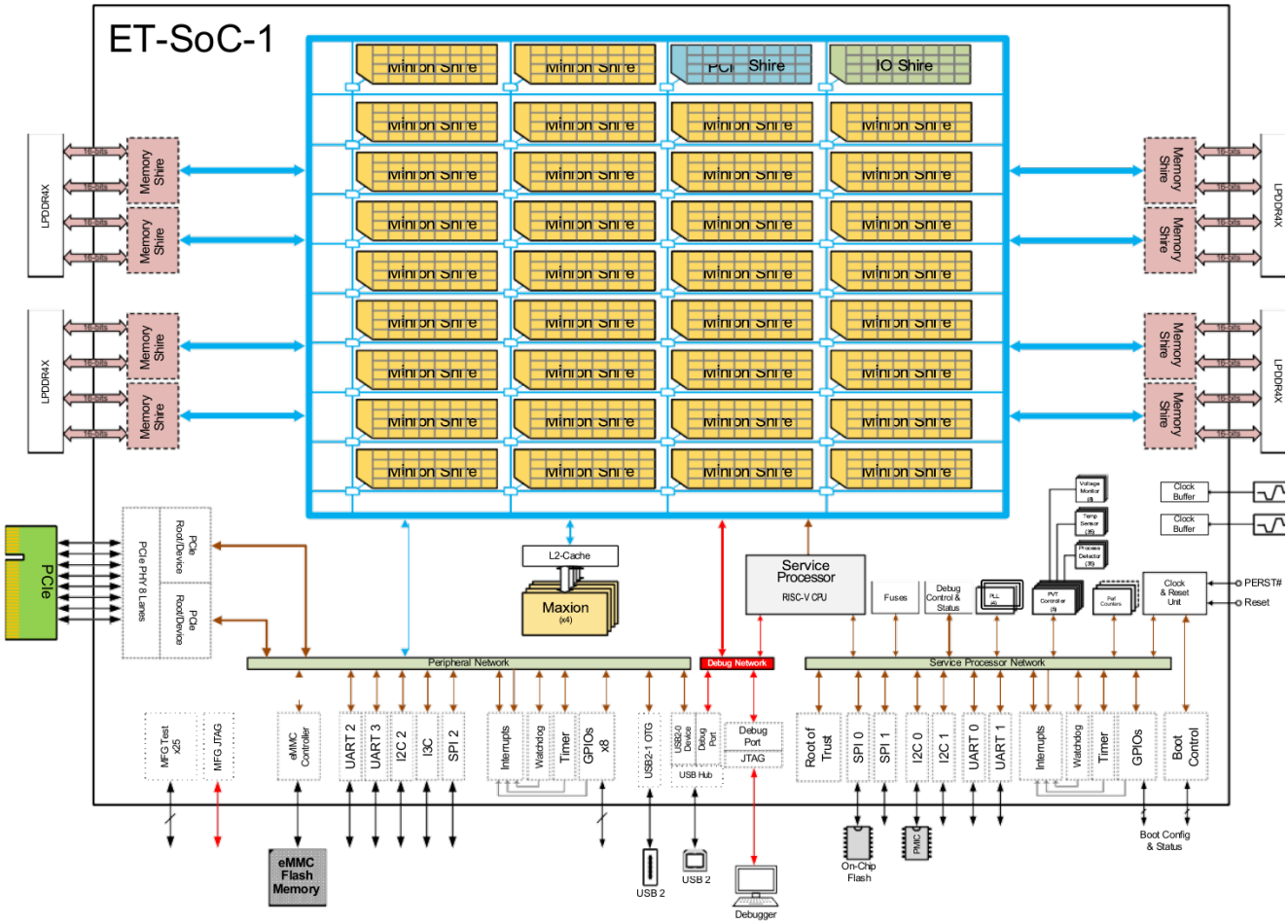



*Figure 6. ET-SoC-1 block diagram*

### 1.4.1. Mesh Topology and Mesh Stops

The mesh is organized as an 8 × 6 grid, and each Shire attaches to it at a mesh stop. The Memory Shires occupy the east and west edges, four on each side, which leaves the corners of the grid unoccupied. The result is 44 mesh stops, listed in Table 2.

*Table 2. Shires on the mesh*

| Shire | Count | Contents |
|---|---|---|
| Minion Shire | 34 | 32 ET-Minion cores and a 4 MB Shire Cache |
| Memory Shire | 8 | LPDDR4X memory controller |
| PCIe Shire | 1 | PCIe controllers and PHY |
| I/O Shire | 1 | ET-Maxions, Service Processor and peripherals |
| **Total** | **44** | |

### 1.4.2. Memory Shires

Each Memory Shire contains a 32-bit LPDDR4X PHY and two memory controllers, each driving one 16-bit channel achieving 4,266 MT/s/channel. The Memory Shires also support global atomic operations.

### 1.4.3. PCIe Shire

The PCIe Shire contains two dual-mode PCIe Gen4 controllers that share an 8-lane PHY. It supports three lane configurations:

- Single 8-lane endpoint or root complex
- Single 4-lane endpoint or root complex
- 4-lane endpoint together with a 4-lane root complex

The interface is normally factory-configured as an endpoint.

### 1.4.4. I/O Shire

The I/O Shire contains the ET-Maxions and the Service Processor (Section 1.3). It also contains the device peripherals, which are grouped on three networks according to which agents may access them.

- **Service Processor network.** Accessible only to the SP. It contains:
  - SPI, including the boot-flash controller
  - I2C, including the link to the external power-management controller
  - UART, including the SP console
  - GPIO, timers, watchdog and boot control
- **Peripheral network.** Shared by the SP, the ET-Maxions and the master Minion Shire. It contains:
  - eMMC, USB 2.0 OTG, I2C/SMBus, I3C, SPI and UART
  - GPIO, timers and watchdog
- **Debug network.** Provides debug access through JTAG and a USB 2.0 device port.

## 1.5. Memory Map

# 1.6. ET-SoC-1 Accelerator Card

ET-SoC-1 operates as an accelerator attached to a host system over PCI Express. It is mounted on a PCIe add-in card (Figure 7) with the following features:

- PCIe x16 card-edge connector, with an 8-lane Gen4 interface to ET-SoC-1
- ET-SoC-1 soldered to the card
- Four LPDDR4X devices forming a 256-bit memory interface
- 64 GB eMMC flash memory
- USB connectors for the ET-SoC-1 device and debug ports and for a UART interface, and a JTAG connector
- DIP switches that select the boot options

Multiple ET-SoC-1 devices can also be integrated on a single card.

*Figure 7. ET-SoC-1 PCIe card*

# 2. ET-Minion Core

## 2.1. Core Overview

This section describes the internal organization of the ET-Minion core: its block partitioning, hardware threads and instruction flow. The features of the core and its position in the ET-Minion hierarchy are summarized in Section 1.2.1.

The ET-Minion is a dual-threaded, in-order, single-issue 64-bit RISC-V core that implements RV64IMFC with the Esperanto instruction extensions. It is implemented in `minion_top` and consists of four units: the Front End, the integer pipeline, the L1 data cache and the VPU. The core fetches instructions from the Neighborhood L0 micro-cache and accesses memory through the Neighborhood memory path; it has no instruction cache of its own.

### 2.1.1. Block Partitioning

The ET-Minion core is implemented in `minion_top` (Figure 2). `minion_top` contains two blocks: `core_top`, which holds the Front End, the integer pipeline, the L1 data cache and the debug APB slave, and `vpu_top`, the vector processing unit. Table 3 lists the blocks.

*Table 3. ET-Minion core blocks*

| Block | Module | Function |
|---|---|---|
| Front End | `frontend_top` | Fetches instructions for both threads from the Neighborhood L0 micro-cache, expands compressed instructions, decodes them for the integer pipeline and the VPU, and issues one instruction per cycle to the integer pipeline (Section 2.2). |
| Integer pipeline | `intpipe_top` | In-order, single-issue pipeline with the stages ID, EX, GSC, TAG, MEM and WB. Contains the integer register file, ALU, multiply/divide unit, integer, FP and mask scoreboards, and the CSRs. Redirects the Front End on a change of control flow. |
| Data cache | `dcache_top` | 4 KB private L1 data cache, partly configurable as scratchpad memory. Contains the data TLB, PMA check, miss handling, replay queue, store merge, atomic (AMO) forwarding to L2 through the miss handler's UC flow, scratchpad controller, write-back unit, cache management, tensor-load and reduce units, and the interface to the Neighborhood memory path. |
| APB slave for debug registers | `minion_debug_apb_slv` | APB slave for debug access to the core, including writes to the debug program buffer. |
| VPU | `vpu_top` | Vector processing unit with a control block and eight identical lanes. Executes floating-point, packed-integer, transcendental and tensor operations. |

### 2.1.2. Hardware Threads

The core runs two hardware threads (`CORE_NR_THREADS` = 2). Each thread has its own architectural state; the pipeline and execution units are shared, and instructions of the two threads are interleaved in the pipeline.

*Table 4. Per-thread and shared resources*

| Per thread | Shared by both threads |
|---|---|
| Fetch PC, thread buffer and instruction FIFO in the Front End | Front End request port, decoders and thread scheduler |
| 32 x 64-bit integer registers, in one register file indexed by thread | Integer pipeline, ALU and multiply/divide unit |
| 32 vector registers, (also the FP f-registers), 256 bits wide, held as a 32-bit slice in each lane's RF, which are indexed by the thread number. Eight 8-bit mask registers. | VPU control and execution units |
| CSR state, interrupt inputs and debug state | L1 data cache and scratchpad |

Each thread is enabled by its corrseponding bit of the `enabled` input of `minion_top`. `minion_top` registers `enabled` once, and the integer pipeline and debug logic uses this registered copy directly. The Front End regsisters it twice more, so that it can detect the edges of the enable and turn each rising and falling edge into a one-cycle enable of disable event (Section 2.2.6.3). When a thread is enabled, it starts fetching at `reset_vector`, which is the same for both threads.

The Front End selects one thread per cycle for issue to the integer pipeline. The integer pipeline can stall each thread independently, for example on `WFI`, a fence, a scoreboard hazard, or while the other thread holds exclusive mode. As a result, the Front End sees a change of `enabled` two cycles later than the integer pipeline does.

### 2.1.3. Instruction Flow Through the Core

The Front End fetches instructions for both threads from the Neighborhood L0 micro-cache, expands compressed instructions, decodes them, and presents one instruction per cycle to ID, the first stage of the integer pipeline. The integer pipeline executes the instructions in order and passes memory operations to the data cache and vector operations to the VPU. In the opposite direction, the integer pipeline redirects the Front End to a new fetch address after a branch misprediction, exception or other change of control flow, and stalls a thread that must not issue.

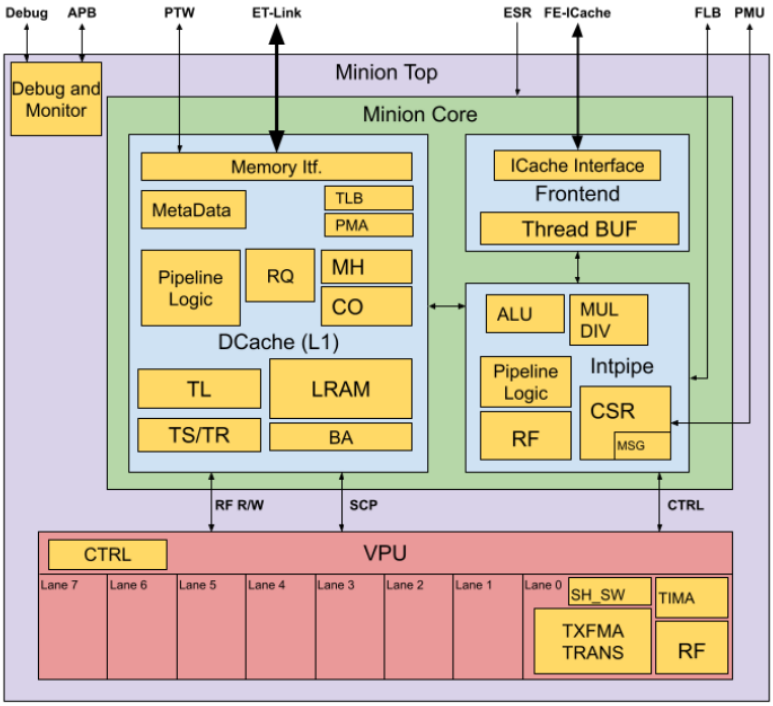



*Figure 8. Front End in the ET-Minion core*

### 2.1.4. Interfaces to the Neighborhood

*Table: ET-Minion core interface groups.*

### 2.1.5. Clocks, Resets and Power

### 2.1.6. Debug Support

## 2.2. Front End

### 2.2.1. Overview

The Front End (`frontend_top`) supplies instructions to the ET-Minion core. It sits between the Neighborhood L0 micro-cache and the ID stage of the integer pipeline. It holds the program counter of each of the two threads and fetches 32-byte instruction blocks from the L0 micro-cache. It then extracts the instruction, expands compressed instructions to 32 bits and decodes them for both the integer pipeline and the VPU. It delivers one decoded instruction per cycle to ID, from either thread. The core has no instruction cache of its own. Each L0 micro-cache is shared by four ET-Minion cores, eight threads in all, so the design of the Front End is driven by the need to hide the latency of shared fetch path.

The Front End is built around the following four concepts:

1. **Fetch is double-buffered:** each thread has a thread buffer with two 256-bit entries, each holding half a cache line, so the next block can be requested while instructions are extracted from the current one.
2. **Each thread has at most one fetch request in flight:** the L0 micro-cache answers every accepted request, hit or miss, after a fixed latency, so a thread recognizes its own by timing

alone, without a thread identifier. On a miss the thread sleeps until the L0 micro-cache reports that a fill is complete, and then retries.

3. **Instructions are decoded before they are queued:** one integer decoder and one VPU decoder, shared by both threads, decode in F7, and each thread has a two-entry instruction FIFO that holds decoded instructions until ID accepts them. This moves decode out of the ID stage and lets a thread keep fetching while ID serves the other thread.
4. **The two threads are independent:** each thread fetches, sleeps, stalls and restarts on its own. They share only the fetch request port, the decoders and the issue slot into ID, and the thread scheduler alternates between them when both are eligible.

The Front End is a pipeline of eight stages, F0 to F7. F1 to F5 overlap the L0 micro-cache pipeline. With an L0 hit and no competing requests, the first instruction of a block reaches ID nine cycles after it is requested. The first target instruction reaches ID ten cycles after a redirect, and the first instruction reaches ID eleven cycles after a thread is enabled.

The integer pipeline controls the Front End through two per-thread signals. A redirect (`f0_core_req_valid`, `f0_core_req`) flushes the thread and restarts fetch at a new PC. It is used for mispredictions, exceptions, replays, interrupts and debug-mode transitions. A stall (`id_core_stall`) stops the thread from being scheduled, for example on WFI, a fence, a CSR access that must wait, or a scoreboard hazard. The Front End attaches the fetch status to each instruction: page and access faults, bus and ECC errors, and a replay flag for uncacheable fetches issued while older instructions are still in flight. The integer pipeline acts on this status in ID.

The Front End also starts and stops threads through their enable inputs and reset vector. It supports debug in two ways: it generates the placeholder instruction on which a halt is taken, and it lets entry 0 of a halted thread's buffer serve as the debug program buffer. Each thread buffer runs on its own gated clock; a chicken bit disables the gating.

Table 5 lists the submodules and Figure 9 shows how they are connected. The sections that follow cover the interface (Section 2.2.2), the pipeline stages (Section 2.2.3), the fetch protocol with the L0 micro-cache (Section 2.2.4), the thread buffer (Section 2.2.5 and Section 2.2.6), decode (Section 2.2.7), the instruction FIFO (Section 2.2.8), the thread scheduler (Section 2.2.9), and halt and debug support (Section 2.2.10).

*Table 5. Front End submodules*

| Submodule | Instances | Function |
|---|---|---|
| Thread buffer (`frontend_thread_buffer`) | 2, one per thread | 64-byte instruction buffer and the thread's program counter. Issues fetch requests to the L0 micro-cache. |
| RVC expander (`frontend_rvc_expander`) | 2, one per thread buffer | Expands compressed (RVC) instructions to 32 bits. |
| Request arbiter (`arb_lru_data`) | 1 | 2:1 LRU arbiter. Selects the thread whose fetch request is sent to the L0 micro-cache. |
| Integer decoder (`intpipe_decode`) | 1 | Decodes the instruction into integer-pipeline control signals. |

| Submodule | Instances | Function |
|---|---|---|
| VPU decoder ( `vpu_decoder`) | 1 | Decodes the instruction into VPU control signals. |
| Instruction FIFO (inline generate block) | 2, one per thread | Two-entry queue of decoded instructions waiting for the integer pipeline. |
| Thread scheduler (`frontend_thread_sched`) | 1 | Selects the thread whose instruction is presented to the integer pipeline. |

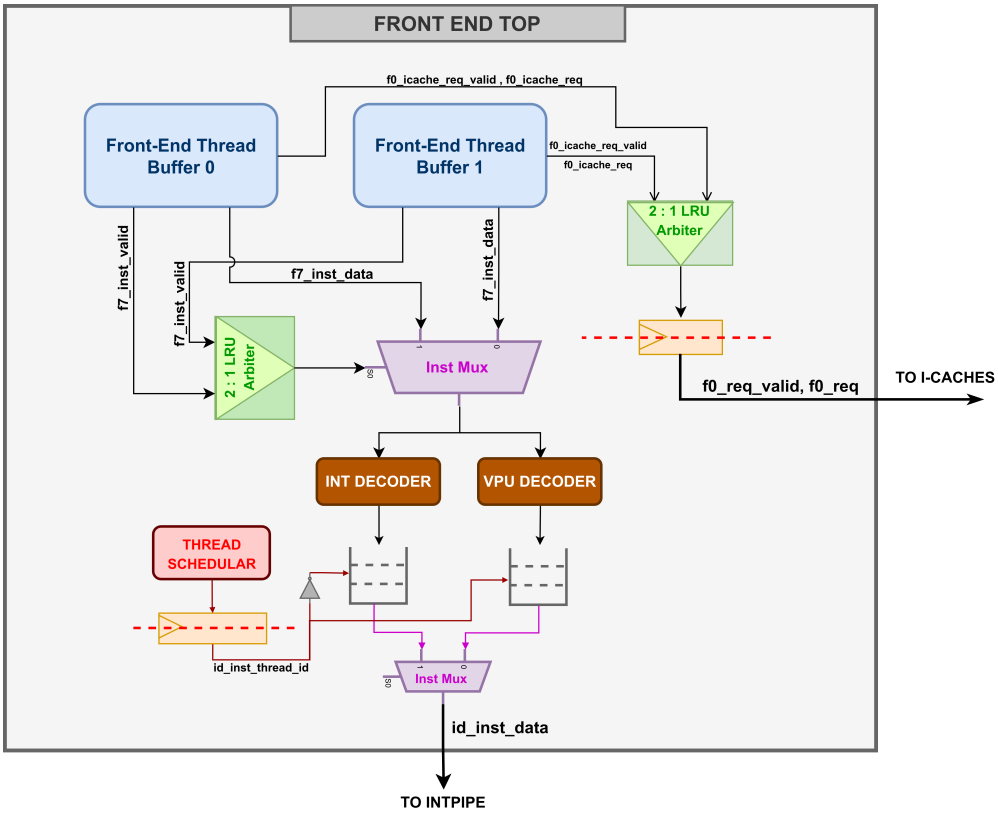



*Figure 9. Front End block diagram*

## 2.2.2. Interface

This section defines the boundary of `frontend_top`. Section 2.2.2.1 lists its ports in six groups. The system group has the clock, the warm and debug resets and the clock-gating chicken bit. Thread control has the thread enables, the reset vector and the virtual-memory status of each thread. The L0 micro-cache group has the fetch request and response and the fill-done signal. The integer-pipeline group has the per-thread redirect and stall inputs. The output group carries the instruction and it's decoded data to ID. The debug group has the halt, halted and program-buffer signals. For each port the table gives the net it connects to in `core_top`.

Section 2.2.2.2 gives the field layout and bit width of each packed structure that crosses these ports. These are the fetch request and response, the virtual-memory status, the redirect, the instruction delivered to ID, the instruction FIFO entry, and the two decoded control words. Section 2.2.2.3 lists the defines that set the number of threads, the buffer geometry and the PC width.

#### 2.2.2.1. I/O Signals

Table 6 lists the ports of `frontend_top`. *Connected to* gives the signal in `core_top`. A width of 2 denotes one bit per thread.

*Table 6. Front End I/O signals*

| Signal | Dir | Width / type | Connected to | Description |
|---|---|---|---|---|
| **System** | | | | |
| `clock` | in | 1 | `clock` | Core clock. Each thread buffer derives a gated clock from it. |
| `reset` | in | 1 | `reset_w` | Warm reset. Synchronous. |
| `reset_debug` | in | 1 | `reset_d` | Debug reset. Clears the program-buffer control and, while the thread is halted, refills the program buffer with `EBREAK`. |
| `chicken_bit` | in | 1 | `chicken_bit_frontend` | Disables Front End clock gating. Driven by the `min_frontend_clock_gate_disable` chicken bit. |
| **Thread control** | | | | |
| `f0_thread_enabled` | in | 2 | `enabled` | Thread enable. A rising edge starts fetch at the reset vector; a falling edge stops fetch. |
| `f0_reset_vector` | in | 48 | `reset_vector` | Fetch address after reset or thread enable. |
| `vm_status` | in | 2 x `minion_vm_status` | `vm_status` | Virtual-memory status of each thread, forwarded with every fetch request. |
| **Fetch request to the L0 micro-cache** | | | | |
| `f1_icache_req_valid` | out | 1 | `icache_req_valid` | The F1 I-cache request register holds a request. |
| `f1_icache_req` | out | `frontend_icache_req` | `icache_req` | Fetch request: thread, VM status and address. |
| `f1_icache_req_ready` | in | 1 | `icache_req_ready` | The Neighborhood accepts the request in this cycle. |
| **Fetch response from the L0 micro-cache** | | | | |
| `f5_icache_resp_valid` | in | 1 | `icache_resp_valid` | A response is present. It carries no thread identifier. |

| Signal | Dir | Width / type | Connected to | Description |
|---|---|---|---|---|
| f5_icache_resp_miss | in | 1 | icache_resp_miss | The response is a miss and carries no instruction data. |
| f5_icache_resp | in | icache_frontend_resp | icache_resp | 32-byte block and status. |
| f6_icache_fill_done | in | 1 | icache_fill_done | A line or translation fill in the L0 micro-cache is complete. |
| **Redirect and stall from the integer pipeline** | | | | |
| f0_core_req_valid | in | 2 | id_core_fe_req_valid | Redirect. Flushes the thread and restarts fetch at f0_core_req.pc. |
| f0_core_req | in | 2 x minion_fe_req | id_core_fe_req | Redirect PC, speculation status and debug-mode transition. |
| id_core_stall | in | 2 | id_core_fe_stall | Thread must not issue. Raised for WFI, CSR accesses that are replayed until a resource is ready (e.g., TensorWait, tensor ops, FCC), a fence, exclusive mode held by the other thread, and integer or FP scoreboard stalls. The WFI/CSR term is suppressed while debug mode or halt request is active. |
| **Instruction to the integer pipeline (ID)** | | | | |
| id_inst_valid | out | 1 | id_fe_core_resp_valid | The instruction presented to the integer pipeline is valid. |
| id_inst_ready | in | 1 | id_core_fe_resp_ready | The integer pipeline is ready to accept the instruction. |
| id_inst_thread_id | out | 1 | id_fe_core_resp_thread_id | Thread of the instruction. |
| id_inst_data | out | frontend_core_resp | id_fe_core_resp | Instruction, PC and fetch status. |
| id_intpipe_ctrl | out | minion_control | id_intpipe_ctrl | Integer decode. |
| id_vpu_decoder_sigs | out | vpu_ctrl_sigs_t | id_vpu_decoder_sigs | VPU decode. Valid only for VPU instructions. |

| Signal | Dir | Width / type | Connected to | Description |
|---|---|---|---|---|
| id_vpu_core_ctrl | out | vpu_minion_id_ctrl | id_frontend_vpu_ctrl | VPU and mask register usage, for hazard checks in the integer pipeline. |
| **Debug** | | | | |
| halt | in | 2 | debug_in_core.halt | A halt request is pending. |
| halted | in | 2 | debug_out.halted | The thread is in debug mode. |
| debug_ffb_wdata | in | 64 | debug_ffb_wdata | Program-buffer write data. |
| debug_ffb_en | in | 4 | debug_ffb_en | Program-buffer word write enables. |
| debug_ffb_thread_sel | in | 1 | debug_ffb_thread_sel | Thread whose program buffer is written. |
| debug_ffb_exec | in | 2 | debug_ex_program_buffer | Start executing the program buffer. |

#### 2.2.2.2. Data Structures

*Table 7. frontend_icache_req: fetch request (58 bits)*

| Field | Width | Description |
|---|---|---|
| thread_id | 1 | Requesting thread. |
| vm_status | minion_vm_status | Virtual-memory status of the thread (Table 8). |
| addr | 49 | Fetch address. The L0 micro-cache returns the aligned 32-byte block that contains it. |

Front End addresses are 49 bits wide: the 48-bit virtual address and one additional bit that allows a non-canonical 64-bit address to be represented.

*Table 8. minion_vm_status: virtual-memory status (8 bits)*

| Field | Width | Description |
|---|---|---|
| prv | 2 | Current privilege level. |
| mprv, mpp, sum, mxr | 1, 2, 1, 1 | Corresponding mstatus fields. |
| debug | 1 | Thread is in debug mode. |

*Table 9. icache_frontend_resp: fetch response (261 bits)*

| Field | Width | Description |
|---|---|---|
| data | 256 | 32-byte instruction block. |
| page_fault | 1 | Instruction page fault. |
| access_fault | 1 | Instruction access fault. |

| Field | Width | Description |
|---|---|---|
| cacheable | 1 | Block is in a cacheable region. |
| bus_err | 1 | Bus error on the line. |
| ecc_err | 1 | ECC error on the line. |

*Table 10. minion_fe_req: redirect (52 bits)*

| Field | Width | Description |
|---|---|---|
| pc | 49 | New fetch PC. |
| speculative | 1 | Thread has older instructions in the integer pipeline. Driven every cycle, independently of f0_core_req_valid. Used for replay (Section 2.2.5.6). |
| debug_info.halt | 1 | Redirect enters debug mode. |
| debug_info.resume | 1 | Redirect leaves debug mode. |

*Table 11. frontend_core_resp: instruction to the integer pipeline (89 bits)*

| Field | Width | Description |
|---|---|---|
| pc | 49 | PC of the instruction. |
| inst_bits | 32 | Instruction. Compressed instructions are delivered expanded. |
| pf0, af0 | 1 each | Page fault / access fault on the entry holding the start of the instruction. |
| pf1, af1 | 1 each | Page fault / access fault on the entry holding the upper half of an instruction that crosses entries. |
| replay | 1 | Instruction must be replayed. |
| rvc | 1 | Instruction was compressed. |
| bus_error, ecc_error | 1 each | Bus error / ECC error on the entry or entries holding the instruction. |

*Table 12. frontend_thread_data: instruction FIFO entry (179 bits)*

| Field | Width | Description |
|---|---|---|
| core_resp | 89 | Instruction and fetch status (Table 11). |
| vpu_ctrl_sigs | 45 | VPU decode (vpu_ctrl_sigs_t). |
| intpipe_ctrl | 45 | Integer decode (minion_control). |

*Table 13. Decoded control words*

| Structure | Width | Content |
|---|---|---|
| minion_control | 45 | Legal instruction; software-emulated (M-code) instruction; branch, JAL, JALR; integer source reads and destination write; ALU operand selects, function and width; immediate format; memory operation, command and size; FP/VPU destination write; mask register read/write; multiply/divide; CSR command; fence; global memory access; gather/scatter; graphics; implicit use of x31. |
| vpu_ctrl_sigs_t | 45 | VPU function; execution unit (TxFMA, shift/swizzle, transcendental ROM); conversion and load/store type; source, third-source and destination reads and write; mask register reads; operand swaps; scalar or packed; data type; integer-register transfer; rounding; FP flag update. |

#### 2.2.2.3. Configuration Parameters

*Table 14. Front End configuration parameters*

| Define | Value | Description |
|---|---|---|
| CORE_NR_THREADS | 2 | Threads per core. |
| FE_FETCH_BUFFERS | 2 | Thread-buffer entries per thread. |
| FE_FETCH_READ_SIZE | 256 | Bits per fetch block and per entry. |
| FE_FETCH_PTR_SIZE | 5 | Read-pointer width: 1 entry bit and 4 halfword bits.<br>log2(FE_FETCH_VALID_INST_SIZE * FE_FETCH_BUFFERS) = log2(16 * 2) = 5 |
| PC_SIZE_EXT | 49 | Front End PC width. |

### 2.2.3. Pipeline Stages

This section describes the eight Front End stages, F0 to F7, and how they line up with the L0 micro-cache pipeline. Two activities run in parallel and meet in the thread buffer. Fetch runs from F0 to F6: a thread's request is arbitrated and sent in F0, waits for acceptance in F1, is looked up in F1 to F5, and the response is written into the thread buffer in F6; a miss writes nothing and the thread goes to sleep. In F7 each thread pulls its next instruction out of its buffer and expands it if compressed. One thread's instruction is decoded and placed in its FIFO. The scheduler presents one thread's FIFO head to the integer pipeline each cycle

Fetching and extracting are independent: a thread keeps extracting instructions from a filled buffer entry while its next request is still in F0-F5.

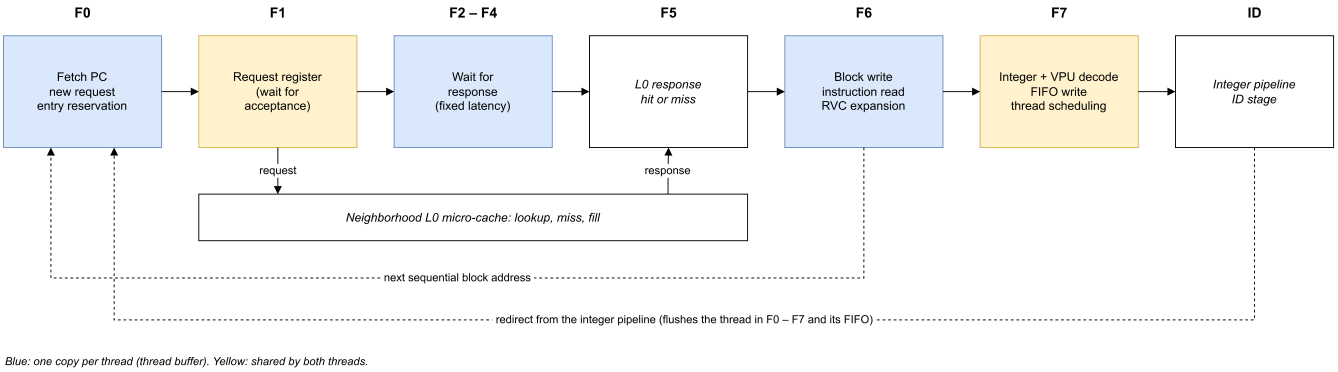



*Figure 10. Front End pipeline*

**F0 Stage**

The F0 stage receives instruction-block requests from both thread buffers and decides which one gets access to the L0 micro-cache using a 2:1 LRU arbiter. The selected request is registered and sent to the L0 micro-cache through the Neighborhood interface. This stage also handles the enabling and disabling of threads and redirect requests from the integer pipeline, for example the new address after a mispredicted branch.

**F1–F5 Stage**

The selected request is held in F1 until the L0 micro-cache accepts it, which happens at the earliest in the second F1 cycle. The thread then waits for the response. The L0 micro-cache always responds to an accepted request, whether it is a hit or a miss, four cycles after the grant, in F5 (Figure 12).

**F6 Stage**

The response from the L0 micro-cache is checked. If it is a hit, the instruction block is stored in the buffer of the thread that issued the request. If it is a miss, nothing is stored, and the thread waits for the cache to be filled. Instruction reading is independent of the fetch request: whenever a buffer entry holds instructions, the thread reads its next instruction, expands it if it is compressed, and passes it on for decoding, in parallel with any request in F0 to F5.

**F7 Stage**

This stage selects which thread's instruction is read from its buffer. The selected instruction is decoded and stored, along with its control signals, in that thread's FIFO. The thread scheduler decides which thread's FIFO issues an instruction to the integer pipeline in the next cycle. If both threads are eligible, they are selected in round-robin order; otherwise, the eligible thread is selected.

With an L0 hit and no competing requests, the first instruction of a block reaches ID nine cycles after its request is issued in F0.

## 2.2.4. Interaction with the L0 Micro-Cache

This section describes the fetch protocol between the Front End and the L0 micro-cache, from the Front End side. It covers the following four things:

1. How a request is arbitrated and accepted: by the 2:1 LRU arbiter and F1 request register in the core, then by the 8:1 arbiter of the Neighborhood.

2. The fixed response timing, and how a thread recognizes its own responses with a token tracker, since responses carry no thread identifier.
3. The miss, sleep and fill-done protocol, and why its timing guarantees that no wake-up is lost.
4. How a redirect cancels requests already in flight, including one still waiting in the F1 register.

Each L0 micro-cache serves the eight threads of four ET-Minion cores. Figure 11 shows the request and response path of one core. The Neighborhood side of the interface, drawn in grey, is described in Section 3.3 and in the sections that follow it.

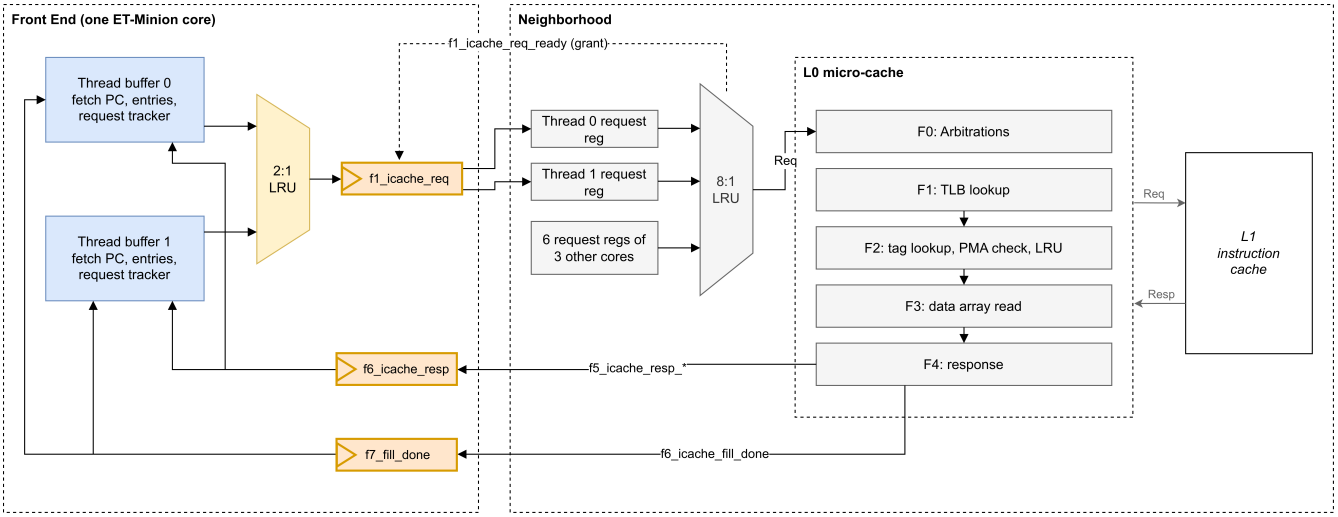



*Figure 11. Front End to L0 micro-cache request and response path*

#### 2.2.4.1. Request Arbitration and Acceptance

A fetch request passes through the following steps:

1. **2:1 LRU arbiter, F0.** Selects between the two threads of the core. With two clients the grant alternates when both threads request. The priority is updated on every grant.
2. **F1 I-cache request register (`f1_icache_req`).** Holds the granted request until the Neighborhood accepts it with `f1_icache_req_ready`. The register accepts a new request when it is empty or its current request is accepted in the same cycle; otherwise the 2:1 arbiter issues no grant.
3. **8:1 LRU arbiter, Neighborhood.** The request is first captured in a per-thread request register, then arbitrated against the other seven threads served by the same L0 micro-cache. The grant of this arbiter is returned as `f1_icache_req_ready`. Because of the capture register, a request is accepted at the earliest in its second F1 cycle.

#### 2.2.4.2. Response Timing and Ownership

The L0 micro-cache has a five-stage pipeline and returns exactly one response per accepted request, from its last stage, for a hit and for a miss alike. The response reaches the Front End in F5.

The response carries no thread identifier. Each thread buffer identifies its responses by position in time: when its request is granted, the thread inserts a token (a valid bit and the reserved entry number) into its own F1–F6 tracker. The token remains in F1 until the request is accepted and then advances one stage per cycle. A response registered in F6 together with a valid token belongs to that thread, and the token selects the entry to be written. The response and fill-done signals are

registered in `frontend_top` and broadcast to both thread buffers.

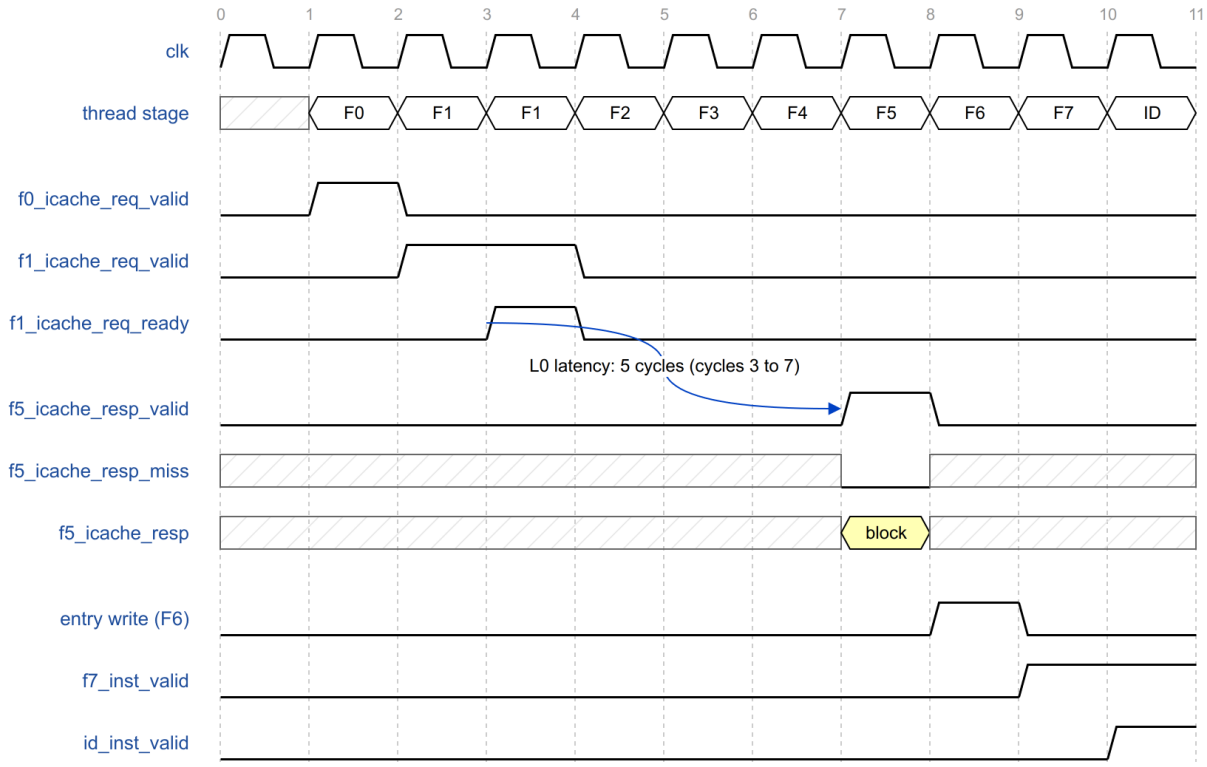



*Figure 12. Request and hit response*

In Figure 12 the request is issued in cycle 1, accepted in cycle 3, and the block returns in cycle 7. It is written in cycle 8; the first instruction of the fetched block is in the F7 stage in cycle 9 and in the ID stage in cycle 10.

#### 2.2.4.3. Miss Handling and Fill Done

A miss response has the same timing as a hit and carries no data. On a miss the thread writes nothing, keeps its fetch PC and enters a sleep state (`f0_miss_pending`) in which it issues no requests.

On completion of a line fill from the L1 instruction cache, or of a translation fill, the L0 micro-cache pulses `f6_icache_fill_done`. The signal is broadcast to all threads of the L0 micro-cache and carries no address. Every sleeping thread wakes and retries; a thread whose line is still absent misses again and sleeps until the next fill. A redirect, a thread disable or a pending halt also clears the sleep state.

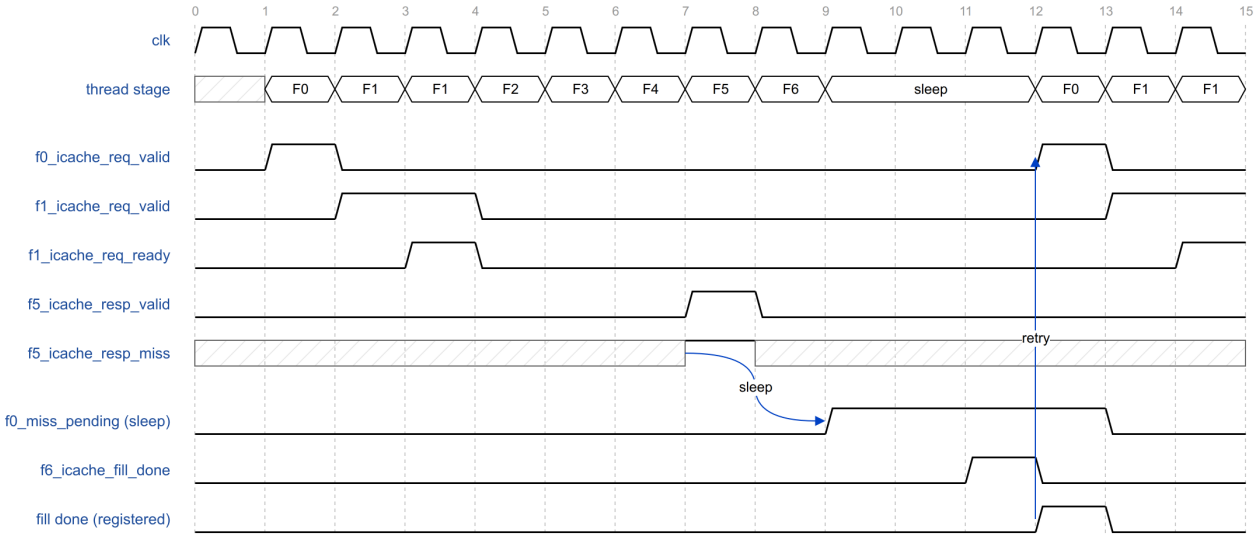



*Figure 13. Miss, sleep and retry*

In Figure 13 the miss response arrives in cycle 7 and the thread sleeps from cycle 9. Fill done arrives in cycle 11, is registered, and the thread retries in cycle 12.

Fill done is issued one cycle after the response slot. A request that looks up the L0 micro-cache in the cycle in which a fill writes the array still misses, and its response is the last miss that the fill can cause. Issuing fill done one cycle after that response, and registering it again in the Front End, ensures that every thread is in the sleep state before the wake-up arrives. If the wake-up arrives first, it is lost, and the thread waits for an unrelated fill.

#### 2.2.4.4. Request Cancellation

A redirect invalidates the thread's tokens in F2 to F5 and suppresses the F6 write, so the responses of in-flight requests are discarded.

A request still waiting in the F1 I-cache request register cannot be withdrawn: the register is shared and the Neighborhood already holds a copy. The thread records the cancellation and does not advance the token when the request is accepted. The I-cache lookup is performed and its response is discarded. The thread's new request is issued only after the Neighborhood accepts the cancelled request.

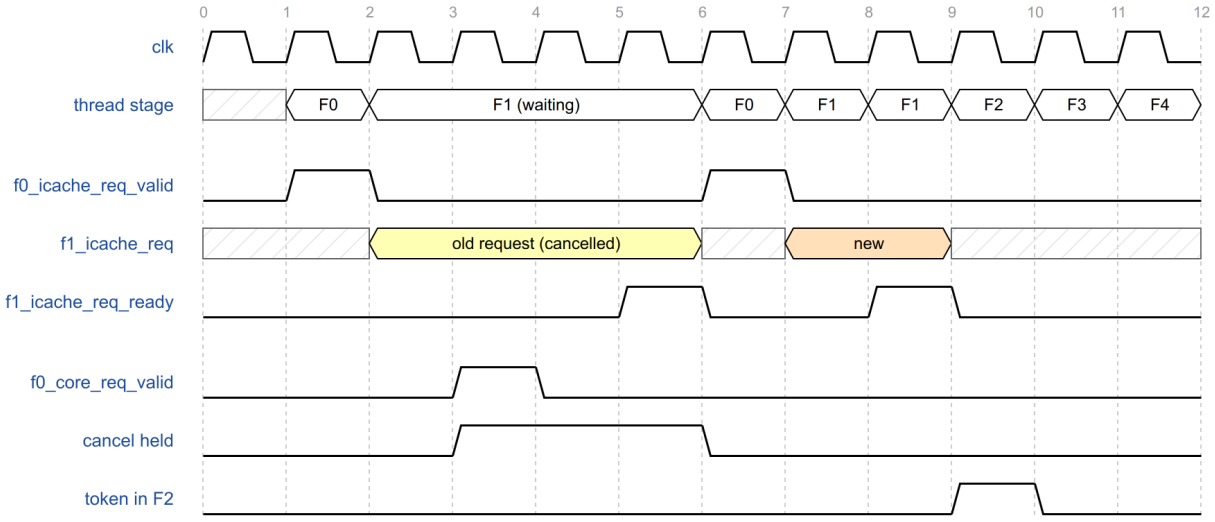



*Figure 14. Redirect while a request waits in F1*

In Figure 14 the redirect arrives in cycle 3 while the request waits in F1. The cancellation is held until the request is accepted in cycle 5; no token enters F2. The new request is issued in cycle 6 and accepted in cycle 8.

### 2.2.5. Thread Buffer

The thread buffer (`frontend_thread_buffer`) holds almost all of a thread's Front End state; there is one buffer per thread. This section describes how the buffer stores instructions: two 256-bit entries in a latch register file with a 32-bit, wrap-around read port. It then describes the state kept for each entry, the fetch PC and how it is updated, and the conditions under which the thread requests a new block. It also shows how an entry is reserved and written.

The section then shows the instruction side: how the read pointer extracts one instruction per cycle, including instructions that cross from one entry to the other and the bypass from incoming data. It then covers how the RVC expander turns compressed instructions into 32-bit ones, and which fault, error and replay status is attached to each instruction on its way to the integer pipeline.

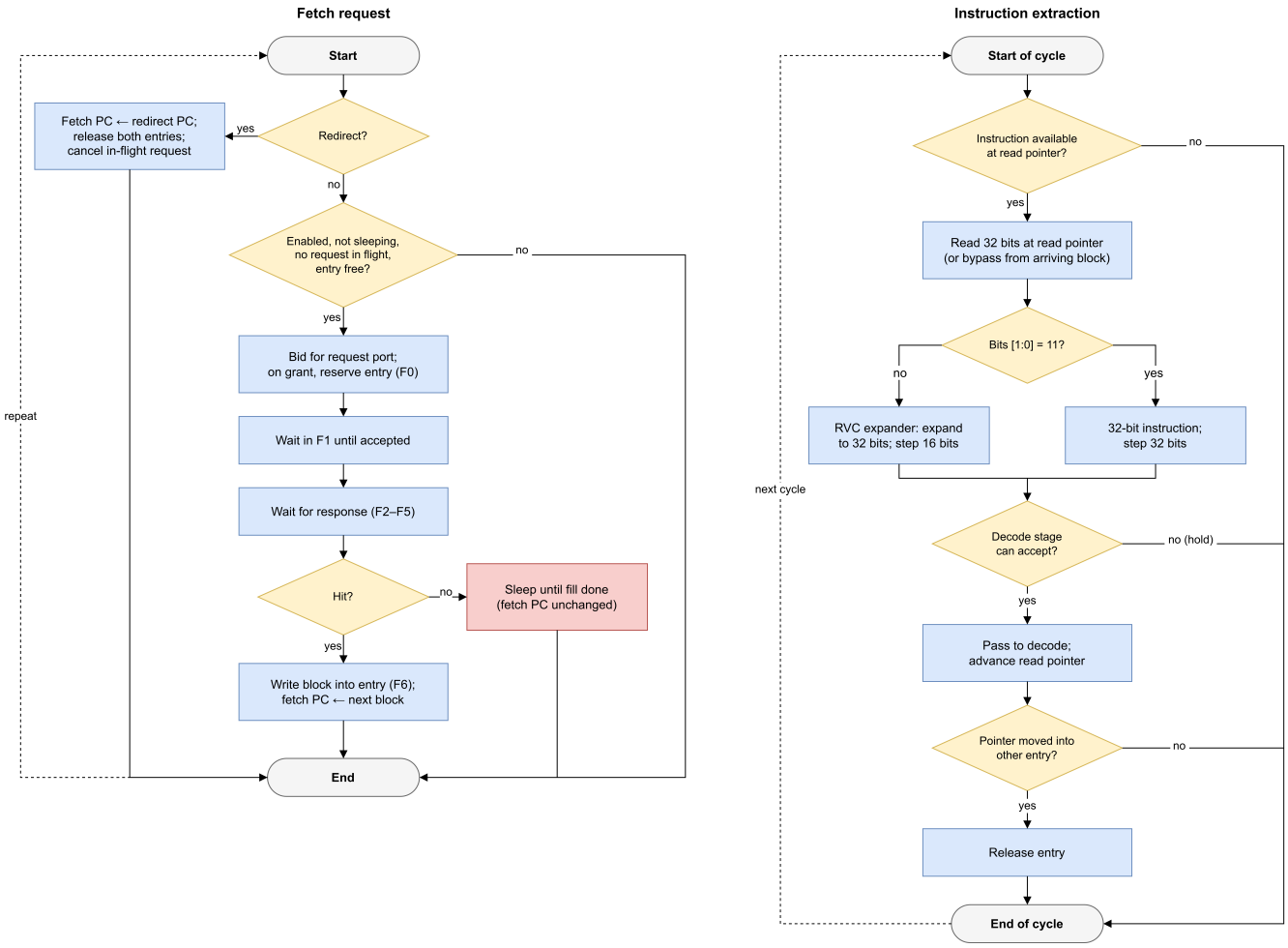



*Figure 15. Thread buffer operation*

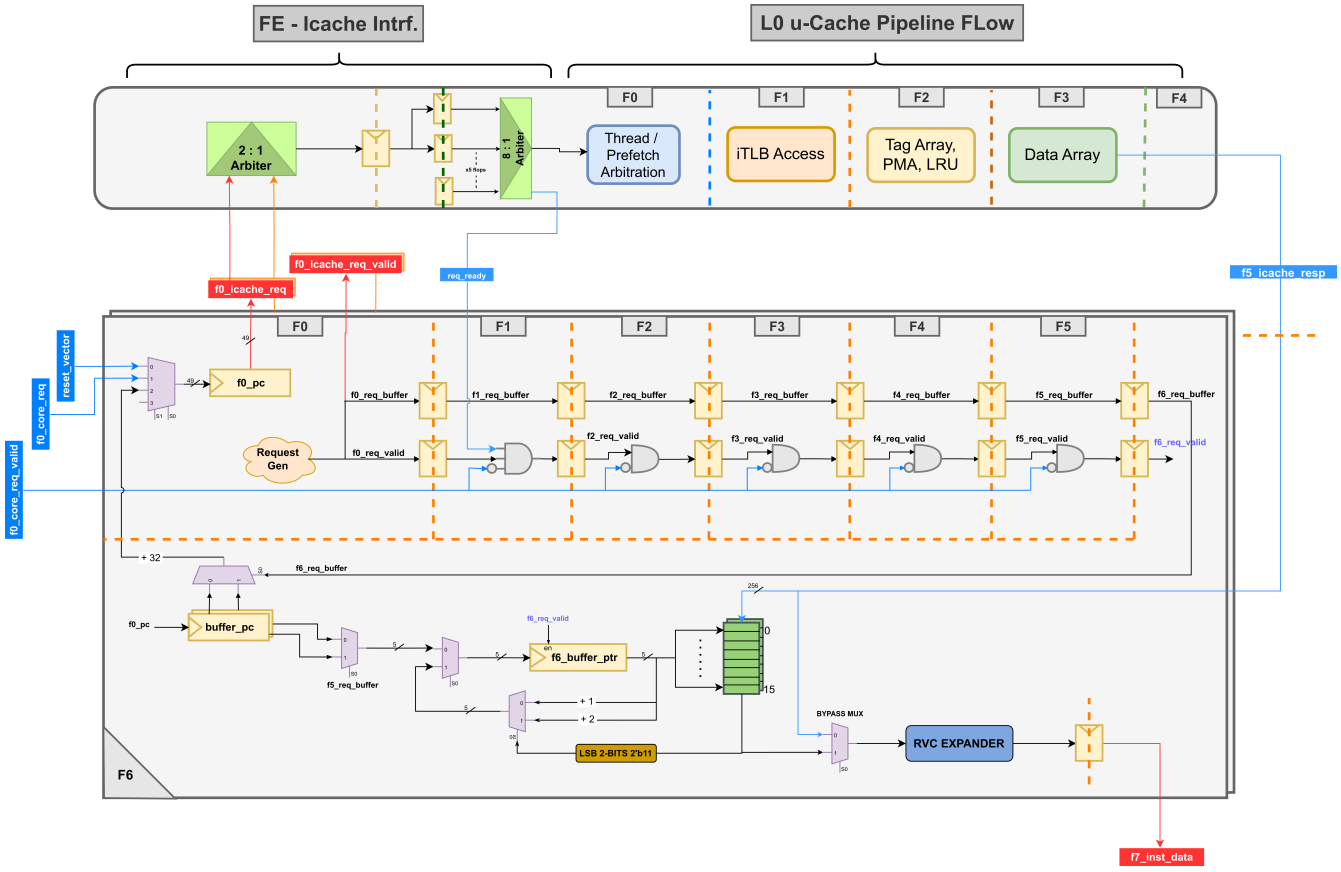



*Figure 16. Thread buffer pipeline*

#### 2.2.5.1. Buffer Entries

The instruction buffer has two entries of 256 bits (32 bytes). An entry holds one fetch block: eight 32-bit instructions, sixteen compressed instructions, or a combination. Two entries allow the next block to be fetched while the current block is consumed.

The entries are implemented as one latch-based register file with two ports:

- **Write port, 256 bits.** Writes a complete block into one entry, with one write enable per 32-bit word.
- **Read port, 32 bits.** Addressed by the 5-bit read pointer `buffer_ptr`: bit 4 selects the entry, bits 3:0 select a halfword within it. The read returns the 32 bits starting at that halfword.

The read port wraps around: halfword 15 of one entry is followed by halfword 0 of the other. The entries therefore form a ring of 32 halfwords, and a 32-bit instruction that starts in the last halfword of an entry is read in one access.

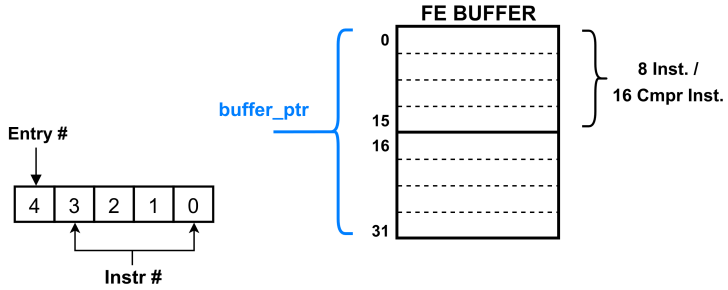



*Figure 17. Thread-buffer entries and read pointer*

*Table 15. Thread-buffer entry state*

| State | Description |
|---|---|
| Empty flag | Entry holds no instruction still to be sent. Set for both entries at reset. |
| Block address | Address of the instruction block held in the entry (`buffer_pc`). Only the words from this address onward are written, extraction in the entry starts at this address, and it provides the upper PC bits of every instruction read from the entry. |
| Fault flags | Page fault and access fault of the block. |
| Error flags | Bus error and ECC error of the block. |
| Cacheable flag | Block is in a cacheable region. Used for replay. |

#### 2.2.5.2. Program Counter

Each thread buffer holds a 49-bit fetch PC, the address of the next block to request. It is updated as listed in Table 16.

*Table 16. Fetch PC update*

| Event | New fetch PC |
|---|---|
| Reset, or thread enabled | `f0_reset_vector`, sign-extended. |

| Event | New fetch PC |
|---|---|
| Block written into an entry | Start of the next aligned 32-byte block. |
| Redirect (highest priority) | `f0_core_req.pc`. |

Only the first block after a reset or redirect can start within a 32-byte block; all later blocks are aligned. A miss does not change the fetch PC, so the same address is requested again after the fill.

The PC of each extracted instruction is formed from the upper bits of its entry's block address and the halfword position of the read pointer.

#### 2.2.5.3. Request Generation and Entry Fill

A thread issues a request in F0 when all conditions in Table 17 are met.

*Table 17. Request conditions*

| Condition | Reason |
|---|---|
| Thread enabled, and not in the enable cycle | Reset vector is loaded at the end of the enable cycle. |
| No redirect in the current cycle | Redirect PC is loaded at the end of the cycle. |
| Thread not sleeping, or registered fill done in this cycle | After a miss the thread waits for a fill; the request is issued in the cycle the registered fill done arrives. |
| No request of the thread in F1 to F6 | At most one request per thread is in flight, which keeps responses in order and allows ownership by timing. |
| An entry is empty or is released in the current cycle | The response cannot be stalled and must have a destination. |
| No halt pending | While a halt is pending the thread sends no request to the L0 micro-cache; it produces a placeholder instruction instead (Section 2.2.10.1). |
| Not in reset; not executing the debug program buffer | — |

On a grant, the thread reserves an entry and records the fetch PC of the request as the entry's block address. If both entries are empty, entry 1 is reserved.

Table 18 shows the entry sequence after a thread is enabled at address A. `[m, n]` gives the block number in entry 0 and entry 1; `x` denotes an empty entry.

*Table 18. Entry fill sequence*

| Step | Event | Entries |
|---|---|---|
| 1 | Thread enabled at address A. | `[x, x]` |
| 2 | Block containing A written into entry 1; extraction starts at the halfword of A. | `[x, A>>5]` |

| Step | Event | Entries |
|---|---|---|
| 3 | Entry 0 empty: the next block is requested when the first request completes. | `[(A>>5)+1, A>>5]` |
| 4 | Last instruction of entry 1 extracted; entry 1 released. | `[(A>>5)+1, x]` |
| 5 | Next block requested into entry 1; extraction continues in entry 0. | `[(A>>5)+1, (A>>5)+2]` |

A block write enables only the words from the word containing the request address to word 7. The register file requires its write-data enables one clock phase before the write; the thread computes them in F5 from its token and `f5_icache_resp_miss`, holds them in a phase latch, and applies the word write enables in F6.

#### 2.2.5.4. Instruction Extraction

When both entries are empty, the read pointer is loaded in F5 with the halfword position of the request address within the reserved entry, so that extraction can start within a block. Afterwards the pointer advances with each extracted instruction.

An instruction is available when the entry it is read from is not empty. A 32-bit instruction that starts in halfword 15 requires both entries to be non-empty; the wrap-around read returns both halves, the pointer moves to halfword 1 of the other entry, and the first entry is released.

If the entry being read is empty but its block is written in the same cycle, the instruction is taken directly from the response data (bypass). The upper half of an instruction that crosses into an entry being written is bypassed in the same way. The first instruction of a block is therefore extracted in the cycle the block is written. The bypass does not check the destination entry of the response; with one request in flight per thread, the only block that can arrive is the one being waited for.

#### 2.2.5.5. RVC Expander

The 32 bits read at the pointer are passed to the RVC expander (`frontend_rvc_expander`), a combinational RV64C decompressor:

- If bits [1:0] are not 11, the instruction is compressed. It is expanded to its 32-bit equivalent and the pointer advances by 16 bits.
- If bits [1:0] are 11, the instruction is 32 bits wide. It passes through unchanged and the pointer advances by 32 bits.

All instructions are therefore 32 bits wide after F6. The `rvc` bit of `frontend_core_resp` indicates a compressed instruction; the integer pipeline uses it to advance the PC by 2 instead of 4.

#### 2.2.5.6. Fault and Error Status

Each instruction carries the status of the entries it was read from:

*Table 19. Instruction fetch status*

| Field | Source |
|---|---|
| `pf0`, `af0` | Fault flags of the entry holding the start of the instruction. |
| `pf1`, `af1` | Fault flags of the entry holding the upper half of an instruction that crosses entries; zero otherwise. |
| `bus_error`, `ecc_error` | Error flags of the instruction's entry, or of both entries for an instruction that crosses entries. |
| `replay` | Set when the entry's block is not cacheable and `f0_core_req.speculative` indicates older instructions of the thread in flight. Captured when the instruction is written into the FIFO. |

All status fields are forced to zero while the debug program buffer is executed.

## 2.2.6. Entry Invalidation, Redirect and Thread Start

This section describes the events that discard buffered instructions or restart fetch. Section 2.2.6.1 lists when a buffer entry is released: normally after its last instruction is extracted, but also on a redirect, a thread disable or entry to debug mode. Section 2.2.6.2 describes a redirect from the integer pipeline: what triggers it, what it clears in each stage, and how long the thread takes to deliver its first target instruction. Section 2.2.6.3 describes how thread enable and disable events are derived from the enable input, how fetch starts at the reset vector, and what state a thread has after reset.

### 2.2.6.1. Entry Invalidation

An entry is released when its last instruction leaves F6, or when the whole buffer is flushed (Table 20).

*Table 20. Entry release conditions*

| Condition | Entries released |
|---|---|
| 32-bit instruction at byte offset 28 of the entry is extracted | This entry |
| Compressed instruction at byte offset 30 is extracted | This entry |
| 32-bit instruction at byte offset 30 is extracted (upper half in the other entry) | This entry |
| Redirect (`f0_core_req_valid`) or thread disable | Both entries |
| Entry to debug mode | Both entries |

A response that reports a fault or error marks the reserved entry non-empty even if no data is written, so that the fault is delivered to the integer pipeline with the next instruction read from that entry.

### 2.2.6.2. Redirect

The integer pipeline redirects a thread by asserting `f0_core_req_valid` with the new PC in `f0_core_req.pc`. Redirects are issued for branch and jump mispredictions, exceptions and returns from trap, replays, inter-processor interrupts, and entry to or exit from debug mode. Table 21 lists

the effect in the redirect cycle.

*Table 21. Effect of a redirect*

| Stage | Effect |
|---|---|
| F0 | Fetch PC loaded with the redirect PC. No request in this cycle. Sleep state cleared. |
| F1 | Request waiting in the F1 I-cache request register marked cancelled. |
| F2–F5 | Tokens invalidated. |
| F6 | No block written. Both entries released. |
| F7 | Instruction discarded. |
| FIFO | Emptied. The thread is not eligible for scheduling in this cycle. |

The target block is requested in the following cycle. Its first instruction reaches ID ten cycles after the redirect (Figure 18).

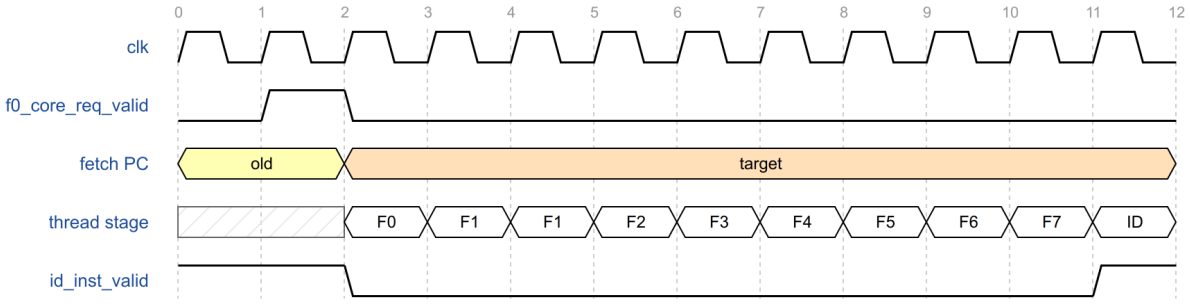



*Figure 18. Redirect to first target instruction*

#### 2.2.6.3. Thread Enable and Reset Vector

`f0_thread_enabled` is registered on entry to the Front End, and one-cycle enable and disable events are derived from its edges.

- **Enable.** The fetch PC is loaded with `f0_reset_vector` at the end of the enable cycle, and the first request is issued in the next cycle. The first instruction reaches ID eleven cycles after `f0_thread_enabled` rises (Figure 19).
- **Disable.** No further requests are issued, both entries are released and the sleep state is cleared. A fetch already in flight is not cancelled: its response is still written into its reserved entry. The instruction in F7 and the FIFO contents are not cleared. The thread is removed from the scheduler's eligible set one cycle later. Because the scheduler defaults to thread 0, FIFO entries left by a disabled thread 0 can still be presented to ID. The eleven-cycle enable latency applies to a thread whose F7, FIFO and buffer are empty, for example the first enable after reset.

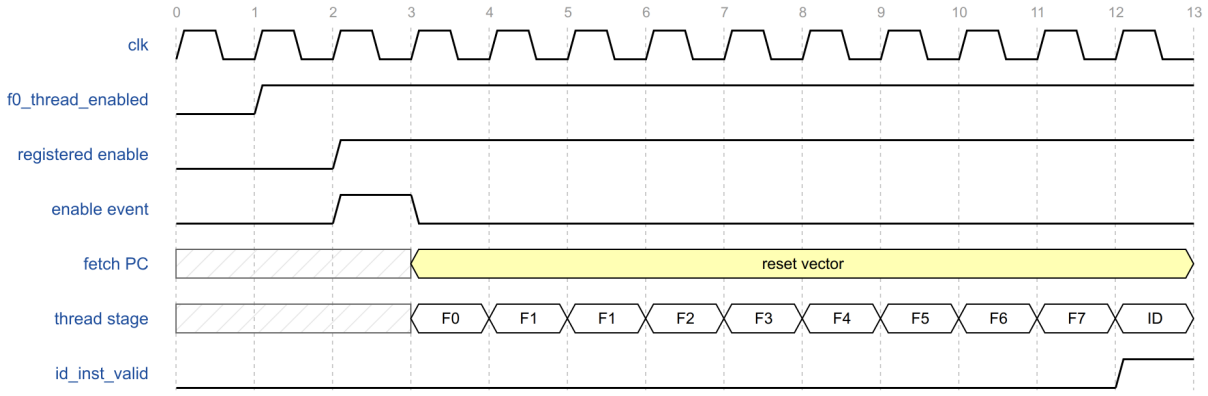



*Figure 19. Thread enable to first instruction*

After reset, both entries are empty and the fetch PC holds the reset vector.

## 2.2.7. Decoders

This section describes the integer decoder (`intpipe_decode`) and the VPU decoder (`vpu_decoder`). Both are shared by the two threads and decode in F7, before the instruction is queued. It describes the control words the decoders produce, `minion_control` and `vpu_ctrl_sigs_t`, and how each decoder handles an encoding it does not recognize. It also describes the rule that selects which thread's instruction is decoded in a cycle.

The section also explains how the VPU decode is used once the instruction reaches ID. The Front End does not issue to the VPU itself: the VPU control word goes directly to the VPU. The integer pipeline issues the instruction to the VPU and uses a summary of the control word for its FP and mask scoreboard checks

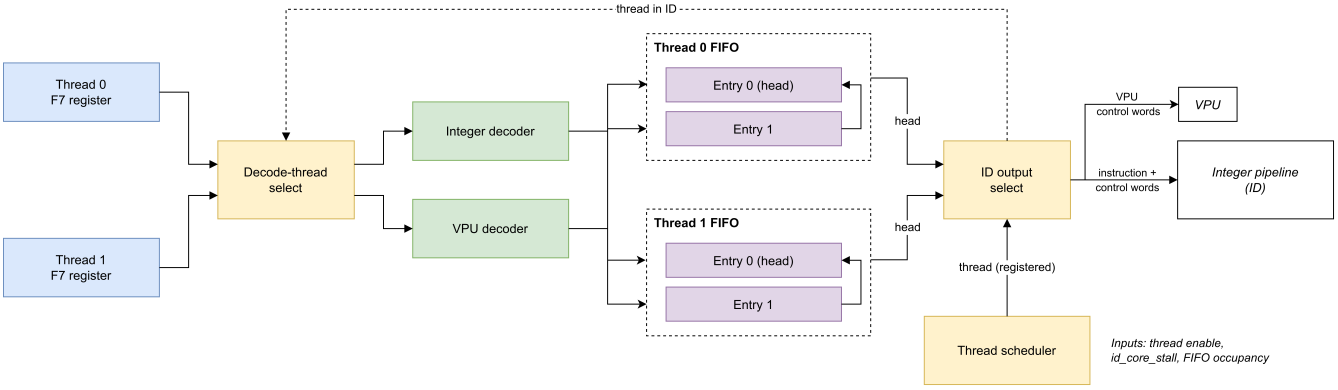



*Figure 20. Decoders, instruction FIFOs and thread scheduler*

Both decoders are combinational look-up tables over the 32-bit instruction:

- **Integer decoder.** Produces `minion_control` (Table 13). An unlisted encoding is marked not legal, and the integer pipeline raises an illegal-instruction exception.
- **VPU decoder.** Produces `vpu_ctrl_sigs_t` and a flag that identifies VPU instructions. An unlisted encoding is a non-VPU instruction.

A thread is a decode candidate when it has an extracted instruction, its FIFO is not full and it is awake (Section 2.2.9). The decoded thread is selected as follows:

*Table 22. Decode thread selection*

| Candidates | Thread decoded |
|---|---|
| Thread 0 only | Thread 0 |
| Thread 1 only | Thread 1 |
| Both | Thread not presented in ID in the current cycle |
| None | Thread not presented in ID in the current cycle |

The instruction is written into the FIFO when the selected thread has an extracted instruction and its FIFO is not full; the awake condition affects only the selection.

The Front End does not issue instructions to the VPU. In ID, the VPU control word of the presented instruction (`id_vpu_decoder_sigs`) is sent directly to the VPU, bypassing the integer pipeline. The integer pipeline issues the instruction to the VPU in the same cycle, when the instruction is accepted in ID and decoded as an FP or VPU instruction, together with its thread and that thread's fcsr rounding mode. The integer pipeline also receives a summary of the VPU control word (`id_vpu_core_ctrl`), with the VPU and mask register reads and writes of the instruction, for its FP and mask scoreboard checks.

### 2.2.8. Instruction FIFO

This section describes the two-entry instruction FIFO that each thread has (uses) between F7 and ID. The FIFO decouples fetch from issue. While ID is stalled or serving the other thread, the thread keeps extracting and decoding, and the FIFO holds the decoded instructions with their fetch status. The section covers what an entry holds and the pointer scheme that tracks occupancy. It explains why the storage is arranged for timing, as a fixed head register and a holding register, and gives the update rules. It also covers when the VPU control word is valid and how a redirect empties the FIFO.

The FIFO is written when its thread is selected for decode and read when the integer pipeline accepts the thread's instruction (`id_inst_valid`, `id_inst_ready` and `id_inst_thread_id`). It uses 2-bit read and write pointers; it is full when the pointers differ only in the wrap bit and empty when they are equal.

The storage is organized for timing rather than as an indexed array. Entry 0 is always the head, whose content drives the ID outputs; entry 1 is a holding register. The ID multiplexer therefore selects between two fixed registers, one per thread.

*Table 23. Instruction FIFO update*

| Occupancy | Event | Update |
|---|---|---|
| Empty | Write | Entry 0 ← new instruction |
| One | Write and read | Entry 0 ← new instruction |
| One | Write | Entry 1 ← new instruction |
| One | Read | None; FIFO becomes empty |
| Full | Read | Entry 0 ← entry 1 |

The VPU control word of an entry is written only for VPU instructions. For other instructions it

retains the value of an earlier VPU instruction, so `id_vpu_decoder_sigs` and `id_vpu_core_ctrl` are valid only for VPU instructions.

A redirect empties the thread's FIFO.

### 2.2.9. Thread Scheduler

The integer pipeline accepts one instruction per cycle from either thread. This section describes how the thread scheduler (`frontend_thread_sched`) chooses which thread's FIFO head is presented to ID in the next cycle. It defines when a thread is awake (enabled and not stalled by the integer pipeline) and when it is ready (it has an instruction and is not being redirected). It gives the round-robin rule between eligible threads, including the default choice when neither thread is eligible. It then describes the handshake with ID: what drives the ID outputs, why the valid signal is not qualified with the stall, and what happens to an instruction that ID does not accept.

**Awake**

The thread is enabled and `id_core_stall` is deasserted. Awake is registered, so a stall affects scheduling one cycle after it is asserted.

**Ready**

The thread's FIFO is not empty or receives a write in the current cycle, and no redirect is in progress for the thread. Including the write allows an instruction to pass from F7 to ID in consecutive cycles.

A thread that is awake and ready is eligible. The scheduler selects between eligible threads in round-robin order:

*Table 24. Thread selection*

| Thread 0 eligible | Thread 1 eligible | Selected |
|---|---|---|
| Yes | No | Thread 0 |
| No | Yes | Thread 1 |
| Yes | Yes | Thread not selected previously |
| No | No | Thread 0 |

The previous selection is updated only when at least one thread is eligible. The selection is registered and drives `id_inst_thread_id` in the next cycle.

The head of the selected thread's FIFO drives `id_inst_data`, `id_intpipe_ctrl`, `id_vpu_decoder_sigs` and `id_vpu_core_ctrl`. `id_inst_valid` is asserted when that FIFO is not empty, and the integer pipeline accepts the instruction with `id_inst_ready`. `id_inst_valid` is not qualified with `id_core_stall`; the integer pipeline applies the stall. The thread in ID is reselected every cycle, so an instruction that is not accepted can be replaced by the other thread's instruction in the next cycle; it remains at the head of its FIFO.

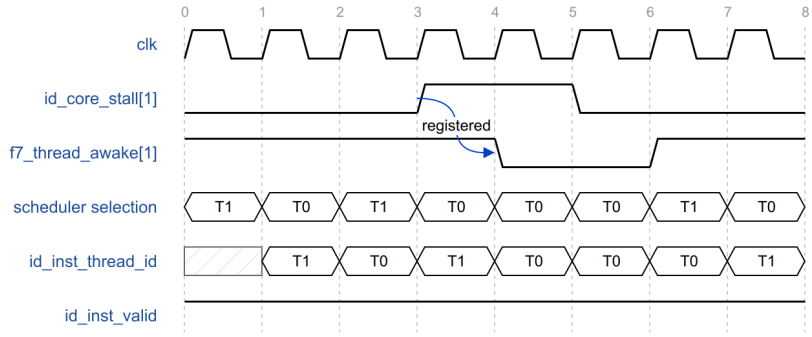



*Figure 21. Round-robin issue with a stall*

In Figure 21 both threads are eligible and alternate. `id_core_stall` of thread 1 is asserted in cycles 3 and 4, and thread 1 is not awake in cycles 4 and 5. Thread 0 is selected in cycles 3 to 5 and presented in ID in cycles 4 to 6; the selection alternates again from cycle 6.

## 2.2.10. Halt and Debug Program Buffer

This section describes the Front End's two debug functions. Section 2.2.10.1 covers halt. The integer pipeline takes a halt only on a valid instruction in ID, so while a halt is requested the thread buffer generates a placeholder instruction instead of fetching, and passes it through the normal pipeline to ID.

Section 2.2.10.2 covers the debug program buffer. In debug mode, entry 0 of the halted thread's buffer holds the instructions written by the debugger. The section describes how the buffer is prepared with **EBREAK** on entry to debug mode, how the debugger writes words into it and starts execution, and how a resume redirect ends program-buffer mode. It also shows which write data goes to which word, and why the last word always holds **EBREAK**.

### 2.2.10.1. Halt

The integer pipeline takes a halt on a valid instruction in ID. While `halt` is asserted for a thread, the thread buffer generates a placeholder instruction instead of fetching:

1. The thread follows the normal request conditions but does not bid for the request port. The request is treated as accepted internally, and its token enters the tracker.
2. The token passes through F1 to F6.
3. In F6 an entry write is forced without a response. The data is the last registered response data; fault and error flags are cleared, and the instruction is treated as 32 bits wide.
4. The placeholder passes through decode and the FIFO to ID, where the integer pipeline takes the halt.

The content of the placeholder is undefined. A pending halt also clears the sleep state.

### 2.2.10.2. Debug Program Buffer

In debug mode, entry 0 of the halted thread's buffer is used as the program buffer: the debugger writes instructions into it and the thread executes them.

*Table 25. Debug program-buffer sequence*

| Step | Operation |
|---|---|
| Entry | A redirect with `debug_info.halt` arrives; the thread stops fetching in the same cycle. In the next cycle both entries are released, the extracted instruction is discarded, and words 0 to 4 of entry 0 are written with `EBREAK`. |
| Write | The debugger writes the program buffer through `debug_ffb_wdata`, `debug_ffb_en` and `debug_ffb_thread_sel` (Table 26). The data and enables come together but the data is written into the entry in the cycle after the enables. |
| Execute | `debug_ffb_exec` resets the read pointer to halfword 0 of entry 0. In the next cycle entry 0 is marked non-empty and entry 1 empty, and instructions are extracted from word 0. |
| Exit | A redirect with `debug_info.resume` ends program-buffer mode; fetch restarts at the redirect PC. |

*Table 26. Program-buffer word mapping*

| Word | Write enable | Data |
|---|---|---|
| 0 | `debug_ffb_en[0]` | `debug_ffb_wdata[31:0]` |
| 1 | `debug_ffb_en[1]` | `debug_ffb_wdata[63:32]` |
| 2 | `debug_ffb_en[2]` | `debug_ffb_wdata[31:0]` |
| 3 | `debug_ffb_en[3]` | `debug_ffb_wdata[31:0]` |
| 4 | — | `EBREAK` |

Word 4 cannot be written by the debugger and always holds `EBREAK`, so execution returns to debug mode at the end of the program buffer. While the program buffer is executed, all fault, error and replay status is forced to zero. A debug reset while the thread is halted rewrites words 0 to 4 with `EBREAK`.

# 2.3. Integer Pipeline

## 2.3.1. Overview

*Figure: Integer pipeline block diagram.*

## 2.3.2. Pipeline Stages

*Figure: Integer pipeline stages (ID, EX, TAG, MEM, WB).*

## 2.3.3. Interfaces

*Table: Integer pipeline interface signals.*

## 2.3.4. Register File and Operand Read

## 2.3.5. Immediate Generation and ALU

### 2.3.6. Multiply/Divide Unit

### 2.3.7. Scoreboards and Hazards

#### 2.3.7.1. Integer Scoreboard

#### 2.3.7.2. FP Scoreboard

#### 2.3.7.3. Mask Scoreboard

### 2.3.8. Hart Interleaving and Stalls

### 2.3.9. Branches, Redirects and Replay

### 2.3.10. Interface to the Data Cache

### 2.3.11. Interface to the VPU

### 2.3.12. CSRs and Privilege Modes

*Table: CSR groups.*

### 2.3.13. Exceptions and Interrupts

### 2.3.14. Debug and Breakpoints

### 2.3.15. Performance Counters

### 2.3.16. Verification Notes

## 2.4. Data Cache

### 2.4.1. Overview

*Figure: Data cache block diagram.*

### 2.4.2. Organization and Operating Modes

*Table: Data cache operating modes.*

### 2.4.3. Interfaces

*Table: Data cache interface signals.*

### 2.4.4. Request Pipeline

*Figure: Data cache request pipeline.*

### 2.4.5. Address Translation and PMA Checks

### 2.4.6. Tag, Data and LRU Arrays

### 2.4.7. Miss Handling

### 2.4.8. Replay Queue

### 2.4.9. Writeback and Store Merge

### 2.4.10. Atomic Operations

### 2.4.11. Cache-Management Operations

### 2.4.12. Scratchpad Control

### 2.4.13. Tensor Load and Reduce

### 2.4.14. Interface to the Neighborhood

### 2.4.15. Errors

### 2.4.16. Performance

### 2.4.17. Verification Notes

## 2.5. Vector Processing Unit

### 2.5.1. Overview

*Figure: VPU block diagram.*

### 2.5.2. Interfaces

*Table: VPU interface signals.*

### 2.5.3. Control and Micro-Instruction Decode

### 2.5.4. Register Files

### 2.5.5. Lanes

*Figure: VPU lane.*

### 2.5.6. Mask Unit

### 2.5.7. Bypass Network

### 2.5.8. Shift and Swizzle Unit

### 2.5.9. TxFMA Unit

### 2.5.10. Integer Multiply-Accumulate Unit

### 2.5.11. Transcendental Unit

### 2.5.12. Tensor Operations

#### 2.5.12.1. Tensor FMA

#### 2.5.12.2. Tensor Quantization

#### 2.5.12.3. Tensor Reduce

### 2.5.13. Interfaces to the Integer Pipeline and Data Cache

### 2.5.14. Performance

### 2.5.15. Verification Notes

# 3. Neighborhood

## 3.1. Neighborhood Overview

### 3.1.1. Partitioning

*Figure: Neighborhood block diagram.*

### 3.1.2. Shared Resources and Core Connections

### 3.1.3. Voltage-Domain Crossings

A Neighborhood is divided into two clock and voltage domains:

- **Low-voltage (LV) region**: the eight ET-Minion cores and most of the Neighborhood channel, running on the Neighborhood clock.
- **High-voltage (HV) region**: the logic that interfaces to the Shire Channel, running on the Shire clock.

Blocks were assigned to the two regions for performance and power: the cores and the channel logic run at the lower voltage to save power, while the logic and the memory cells that need speed run at the higher one. Two properties of an SRAM cell decide where its memory has to go, and both improve with voltage:

- **Access time.** A cell driven from a higher supply switches its bit lines faster, so the array is read and written in less time.
- **Noise margin.** Noise margin is the amount of disturbance, from coupling between neighbouring lines, supply droop or process variation, that a cell can absorb before the value it holds or the value being read out flips. It shrinks as the supply falls, so reads and writes become less reliable. A memory macro is therefore characterized with a minimum operating voltage, and carries trim settings, read assist, write assist and write-pulse width, that buy margin back at low voltage at the cost of speed.

Memory cells were kept at the higher voltage for both reasons, which is also why the cells of the crossing FIFOs themselves are always placed in the high-voltage region, whichever direction the interface goes. Margin is not the same as immunity, so the data path from the memory is protected as well: the line returned from the L1 instruction cache data RAM is covered by ECC, the error is reported to the requesting core with the response and it is logged in the Neighborhood ESRs (Section 3.3.6).

The two regions are connected by a semi-synchronous interface, and every path between them passes through voltage-crossing FIFOs, whose crossing elements contain the level shifters. The paths that cross are the memory request and response paths, the fast local barrier path, the debug and ESR accesses, and the SRAM interface of the L1 instruction cache, whose data RAM is in the high-voltage region of the Shire Channel. Each crossing costs latency, which is why these paths are longer than their logic alone suggests.

### 3.1.4. External Interfaces

*Table: Neighborhood interface groups.*

## 3.2. Neighborhood Instruction Cache

A Neighborhood groups eight ET-Minion cores. Each core has its own private L1 data cache, but none of them has a private instruction cache. Instead, the eight cores share one instruction cache, a 32 KB L1 I-Cache that the Neighborhood adds as a shared resource. The cores of a Neighborhood normally execute the same code, so a shared cache holds one copy of it instead of eight, and that saves both area and power.

The shared instruction cache has two levels of access because of the voltage region its memory has to be built in. SRAM cells are faster and hold their contents more reliably at a higher supply voltage, so the cache's data RAM is placed in the high-voltage region, while the Neighborhood itself operates at low voltage (Section 3.1.3).

The cache's data RAM is therefore placed outside the Neighborhood, in the Shire Channel, and every access to it passes through the voltage-domain crossing, which lengthens the access time. Serving every core from that cache alone would also leave all 16 threads of the Neighborhood, eight cores with two threads each, competing for its single request port. Access to that shared port is handled by LRU arbitration, so a thread would wait behind up to 15 others for its turn, even when the line it needs is already present.

To overcome both, the long read latency of its data RAM in another voltage region and the wait for a turn among 16 bidding threads, a micro level was added in front of that shared cache: two small first-level caches, called L0 micro-caches, inside the Neighborhood, next to the cores. Their data arrays are built in the Neighborhood channel, so a hit is served without crossing into the Shire Channel, and each micro-cache serves half the cores, so eight threads share a port instead of 16.

The shared instruction cache of the Neighborhood is thus split into the following hierarchy:

- **L0 micro-caches**: 16 fully associative lines of 64 bytes each, with their own instruction TLB, PMA check, LRU victim selection and page-table walker port (Section 3.6), and a five-stage lookup. There are two of them, called the north and the south micro-cache, serving Minions 0 to 3 and Minions 4 to 7 respectively, that is eight threads each. Both are identical except for one function, the code prefetcher, enabled in the north micro-cache only, for fetching cache lines into the L1 instruction cache.
- **L1 instruction cache**: 32 KB, 128 sets x 4 ways, 64-byte lines, shared by both micro-caches and accessed only when one of them misses. Its tags, LRU and control are inside the Neighborhood; its data RAM is the one in the Shire Channel.

Figure 22 shows the structure. Section 3.2.1 describes how the two cache levels are packaged in the RTL and what blocks they connect to. The micro-caches and the L1 instruction cache are described in detail in Section 3.4 and Section 3.5, and the path from the core's Front End to the cache, with its arbitration and priority updates, in Section 3.3.

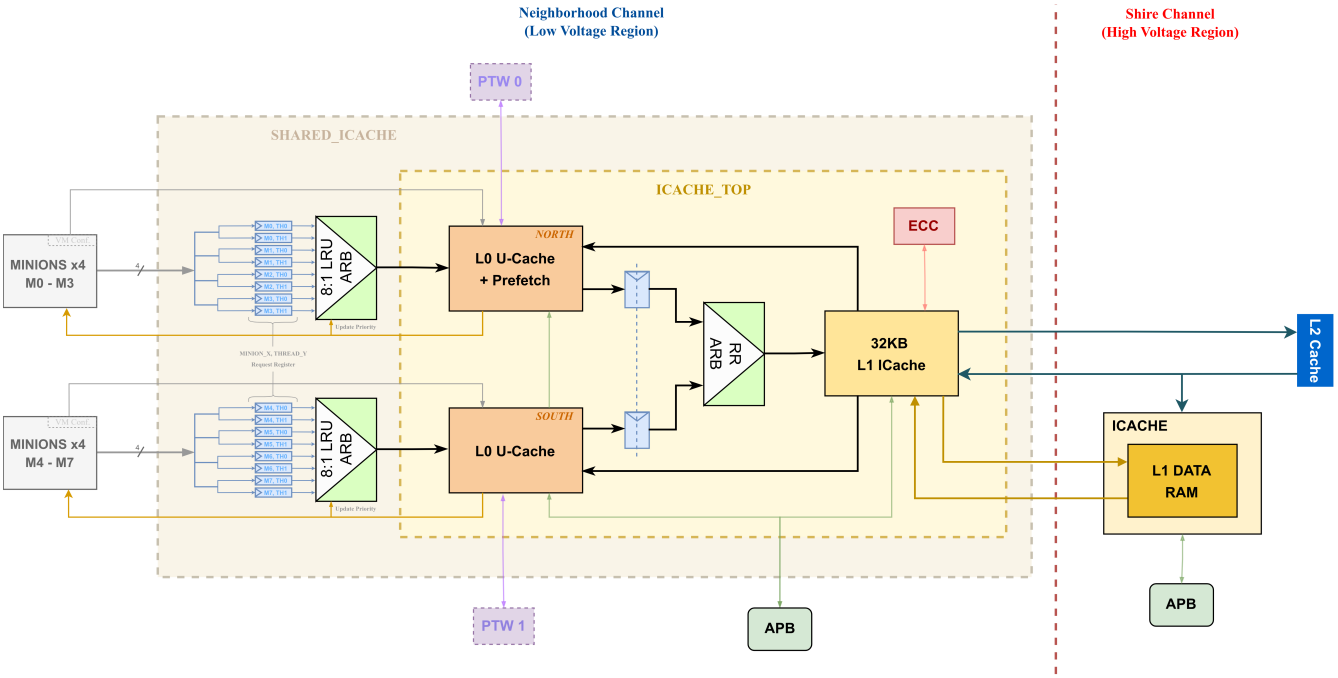



*Figure 22. Neighborhood instruction cache*

### 3.2.1. Structure and Connections

The cache is implemented in two modules:

- `icache_top` contains both L0 micro-caches and the L1 instruction cache, everything except the L1 data RAM, that is the micro-cache arrays and pipelines, the instruction TLBs, the PMA checks, the L1 tag, LRU and data-array control, and the round-robin arbiter through which the two micro-caches reach the L1.
- `neigh_shared_icache` wraps `icache_top` and adds the Neighborhood-side logic of the core interface, the logic that receives the fetch request of each of the eight cores, holds it per thread and presents one of them to a micro-cache: one request register per thread, the 8:1 LRU arbiter that selects among them, and the per-thread miss counters and bid mask that prevent livelock. These are described in Section 3.3.

The top-level `neigh_shared_icache` block connects to five things outside itself:

| Interface | Purpose |
|---|---|
| Cores | Fetch requests and address-translation configuration, one port per core. Responses and the fill-done indication, one port per L0 micro-cache, each driven to the four cores that micro-cache serves. |
| Page-table walkers | Translation requests from the instruction TLB of each micro-cache, one walker per micro-cache. |
| Shire Cache | Line requests on an L1 miss, and their responses, over the Neighborhood request and response path. |
| L1 data RAM | SRAM read and write requests to the data RAM in the Shire Channel, and the read data returned from it. |
| ESR and APB | Configuration (prefetch, bypass, protection and page size), error logging, and debug access to the arrays. |

The signals of each interface are listed with the block that uses them: the core-side signals in Section 3.3.2, and the rest in the detailed sections of the two cache levels.

# 3.3. Front End – I-Cache Interface

This section describes the fetch protocol between the ET-Minion cores and the instruction cache, from the Neighborhood side. The core side is described in Section 2.2.4.

## 3.3.1. Overview

The eight cores of a Neighborhood are served by two L0 micro-caches, each shared by four cores, that is by eight threads, two threads per core. Both L0 micro-caches are backed by the L1 instruction cache of the Neighborhood. Figure 23 shows the whole fetch path and names the signals of the interface: the two thread buffers and the 2:1 LRU arbiter inside each core, the per-thread request registers and the 8:1 LRU arbiter of each micro-cache, and the round-robin arbiter through which both micro-caches reach the L1 instruction cache.

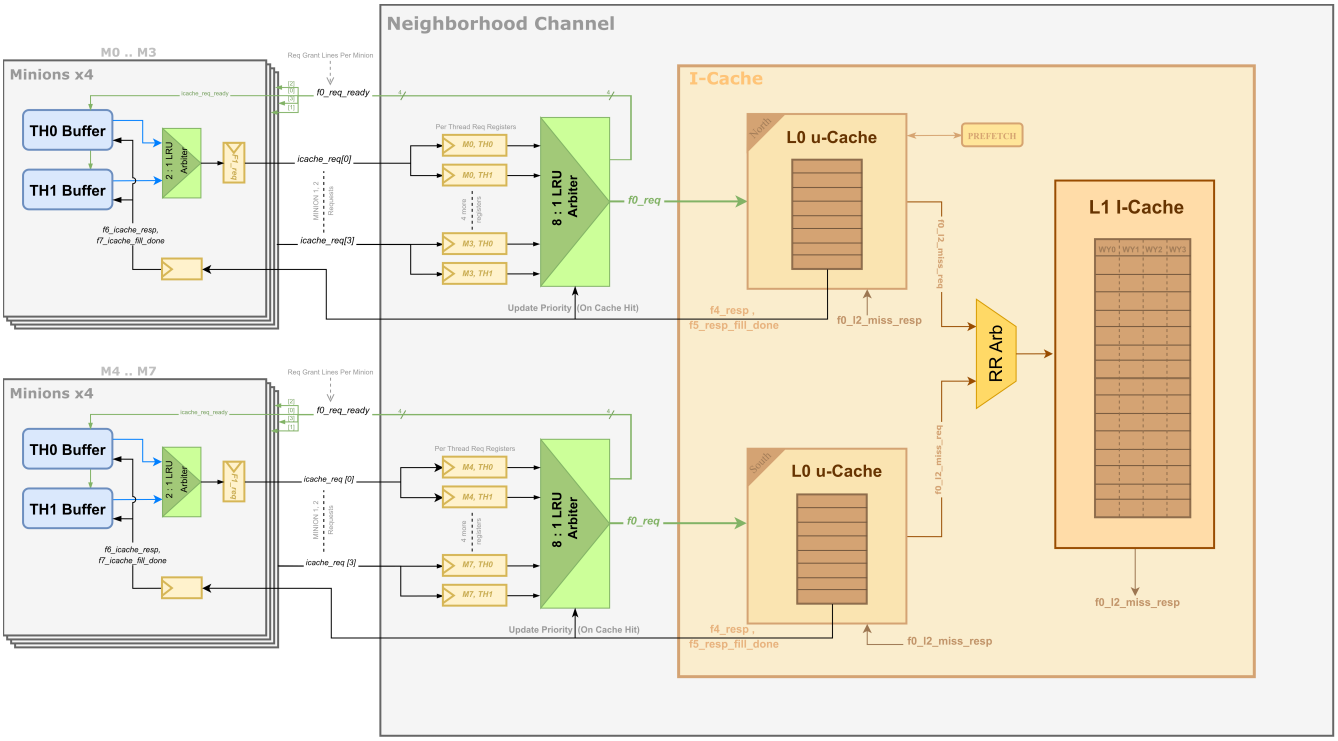



*Figure 23. Front End – I-cache interface*

A fetch request, that is a request from the Front End of a core for the instruction block at a thread's fetch address, travels as follows:

1. In the core, the 2:1 LRU arbiter selects one of the two threads, and the selected request is flopped into the Front End's F1 request register, which holds it until the Neighborhood accepts it. Each core therefore presents one request at a time.
2. In the Neighborhood, the request is steered into the request register of the thread that made it, using the thread identifier it carries. One register exists per thread, so eight per L0 micro-cache, which lets the threads of all four cores arbitrate on equal terms.
3. The 8:1 LRU arbiter selects one of the eight captured thread requests and presents it to the L0 micro-cache. The grant is returned to the core in the same cycle, as `icache_req_ready`, and it

frees the core's F1 request register: the core's 2:1 arbiter runs in that same cycle and loads the other thread's request, so the core presents it on the next cycle with no idle cycle in between.

4. The L0 micro-cache looks the request up in five stages: arbitration, instruction-TLB translation, tag lookup and PMA check, data-array read, and response.
5. One response is returned a fixed four cycles after the grant, for a hit and for a miss alike, and it reaches the core in its F5 stage. It is shared by the eight threads the micro-cache serves and carries no thread identifier; on a hit it carries the instruction block. How a core recognizes its own response is described in Section 2.2.4.2.
6. On a miss, which can be a TLB miss or a line miss, the L0 micro-cache requests a translation from its page-table walker, or requests the line from the L1 instruction cache through a round-robin arbiter that the two micro-caches share. When the fill completes, the micro-cache pulses fill done to all the threads it serves, and the sleeping ones wake and retry.

### 3.3.2. Interface Signals

Table 27 lists the signals between a single core and the Neighborhood. Directions are given from the Neighborhood side.

*Table 27. Front End – I-cache interface signals*

| Signal | Dir | Width / type | Description |
|---|---|---|---|
| **Fetch request** | | | |
| `icache_req_valid` | in | 1 | The core presents a fetch request. |
| `icache_req` | in | `frontend_icache_req` | Requesting thread, its virtual-memory status and the fetch address. |
| `icache_req_ready` | out | 1 | The 8:1 arbiter grants the request. Returned in the grant cycle, it frees the core's F1 request register, so the core can load its other thread's request in the same cycle and present it on the next. |
| **Fetch response** | | | |
| `icache_resp_valid` | out | 1 | A response is returned in this cycle. |
| `icache_resp_miss` | out | 1 | The response is a miss and carries no instruction data. |
| `icache_resp` | out | `icache_frontend_resp` | 32-byte block with its fault, error and cacheable status. |
| `icache_fill_done` | out | 1 | A line or translation fill in the L0 micro-cache is complete. |
| **Address translation and maintenance** | | | |
| `satp_info`, `matp_info` | in | 2 × `minion_satp_info` | Supervisor and machine address-translation registers of the core, used by the instruction TLB. |

| Signal | Dir | Width / type | Description |
|---|---|---|---|
| tlb_invalidate | in | 1 | Invalidate the instruction-TLB entries of this core. |
| icache_flush_data | in | 1 | Invalidate the data stored in the L0 micro-cache. |

The fetch response ports are the exception to that one-core view. The Neighborhood drives one set of them per L0 micro-cache, common to the four cores it serves, so icache_resp_valid, icache_resp_miss, icache_resp and icache_fill_done reach all four at the same time. Being common, the response carries no thread identifier and nothing that says which core it belongs to. Each core works that out on its own: it tracks its outstanding request down the pipeline, so it knows in which cycle its response is due and which of its two threads made it, and it ignores the responses of the other three cores (Section 2.2.4.2).

## 3.3.3. Request Arbitration

This section describes how the Neighborhood decides which thread's request reaches the L0 micro-cache in a given cycle, and when the arbiter updates the priority that decision is based on. Lets start with the registers that hold a thread's request on its way in.

A fetch request is flopped twice before it enters the L0 micro-cache.

**First flop, in the core.** Two threads share one fetch port of the core, so the 2:1 LRU arbiter selects one of them and its request is held in the Front End's F1 request register. That register keeps the request until the Neighborhood (the L0 micro-cache in this case) accepts it, so a core presents one request at a time, carrying the identifier of the thread that made it.

**Second flop, in the Neighborhood.** Each micro-cache serves four cores, so it sees at most four requests in a cycle, and it holds one request register per thread, eight in all. Each request is written into the register of the thread its identifier names. At most four of the eight registers are valid at a time, because a core frees its F1 register only when its request is accepted. Eight registers exist nevertheless so that every thread bids for itself, which is what makes the arbitration fair between the two threads of a core as well as between cores.

The eight registers bid through the 8:1 LRU arbiter (arb_lru_delayed_data). A bid is the register's valid qualified with the thread's bid mask. The mask exists to avoid livelock and is described in Section 3.3.8; until then, take it that masking suppresses a thread's valid at the arbiter, so the thread takes no part in the arbitration while its request stays in its register.

The arbiter is stalled while the micro-cache cannot accept a request, which the micro-cache reports with f0_req_ready. Its pipeline never stalls, so this signal is tied high: a request is accepted in every cycle in which at least one thread bids, and the rest of this section takes that for granted.

### 3.3.3.1. Priority Matrix

The 8:1 LRU arbiter holds its priority in an aging matrix: one bit for every pair of the eight threads it serves, recording which of the two was served more recently. A thread wins when it has priority over every other thread bidding in that cycle, which makes the grant one-hot. Two threads can

never hold the same priority, because every pair is ordered by exactly one bit. Granting a thread clears all of its bits, that is it is placed behind every other thread, and the threads with the longest wait move up.

The grant is combinational. In the same cycle it is returned to the core as `icache_req_ready`, the thread's request register is freed, so it reads invalid in the next cycle, and the core may load the request of its other thread into the F1 register. A request is therefore accepted at the earliest in its second cycle in the Front End's F1 stage, because that register has to hold it for one more cycle while it is flopped into the Neighborhood's per-thread register.

#### 3.3.3.2. Deferred Priority Update

Every accepted request gets exactly one response, and it comes back a fixed four cycles after the grant, whether it hits or misses (Section 3.3.4). The arbiter waits for it: priority is not updated when the grant is issued, but when that response arrives.

Waiting means the arbiter has to remember whose response is due, because the response is shared and carries no thread identifier. The grant vector therefore travels down with the request in its own registers, `f1_grant` to `f4_grant`, so that in F4 it can be told which thread the response belongs to (Table 28).

*Table 28. Grant pipeline in the Neighborhood*

| Stage | Register | Content |
|---|---|---|
| F1 | `f1_grant` | One-hot grant of the arbiter, loaded in the cycle the L0 micro-cache accepts the request |
| F2 | `f2_grant` | Same vector, one cycle later |
| F3 | `f3_grant` | Same vector, one cycle later |
| F4 | `f4_grant` | Thread whose response is returned in this cycle; selects the priority bits and the miss counter to update |

The update is enabled by `f4_update_prio`, asserted when a response is valid in F4 and that response is a hit, or reports a page fault or an access fault:

- **Hit.** Priority is updated and the thread moves behind the others, which gives the waiting threads their turn.
- **Miss.** Priority is left unchanged, so the thread keeps its position while it sleeps and is served early when it retries after the fill (Figure 25).
- **Page fault or access fault.** Priority is updated, because the thread does not retry the same fetch: the core records the fault with the instruction and takes the trap (Section 3.3.6).

Bus and ECC errors do not appear in this condition. Both are reported only with a hit, and a hit already updates the priority.

### 3.3.4. Request and Response Timing

The L0 micro-cache looks a request up in five stages and drives fill done one stage later (Table 29).

*Table 29. L0 micro-cache pipeline stages*

| Stage | Operation |
|---|---|
| F0 | Arbitration; the granted request enters the pipeline. |
| F1 | Instruction-TLB lookup, which translates the virtual address. |
| F2 | Tag lookup, PMA check and LRU update. A miss is detected here, and accepted if the miss handler is idle. |
| F3 | Data array read. On an accepted miss, the request to the L1 instruction cache is driven in this cycle. |
| F4 | Response to the cores (`f4_resp_valid`, `f4_resp_miss`, `f4_resp`). |
| F5 | Fill-done pulse to the cores (`f5_resp_fill_done`), one stage after the response. Not part of the lookup: it reports that a line or translation fill is complete. |

These stages are the micro-cache's own, and they are numbered independently of the Front End's F0 to F7: the F1 request register of the core and the F1 stage of the micro-cache are different things.

Every accepted request receives exactly one response from the last stage, for a hit and for a miss alike, so the response arrives a fixed four cycles after the grant. The response is broadcast to the four cores of the L0 micro-cache and carries no thread identifier; each core identifies its own response from the position of its request in time (Section 2.2.4.2).

The fill-done signal is driven one stage later than the response, from the stage after F4.

In the waveforms that follow, a thread is written M0, TH0, meaning thread 0 of Minion 0. Figure 24 shows a request that hits: the core presents it, the 8:1 LRU arbiter grants it in cycle 1, and the response is returned in cycle 5, four cycles later. The grant frees both registers that hold the request, the core's F1 request register and the Neighborhood's per-thread register, which is why both valids fall in the next cycle. `f4_grant` still names M0, TH0, the thread that was granted four cycles earlier, which is how F4 knows whose priority to update: the response is a hit, so `f4_update_prio` is asserted and M0, TH0 moves behind the other threads in the LRU order.

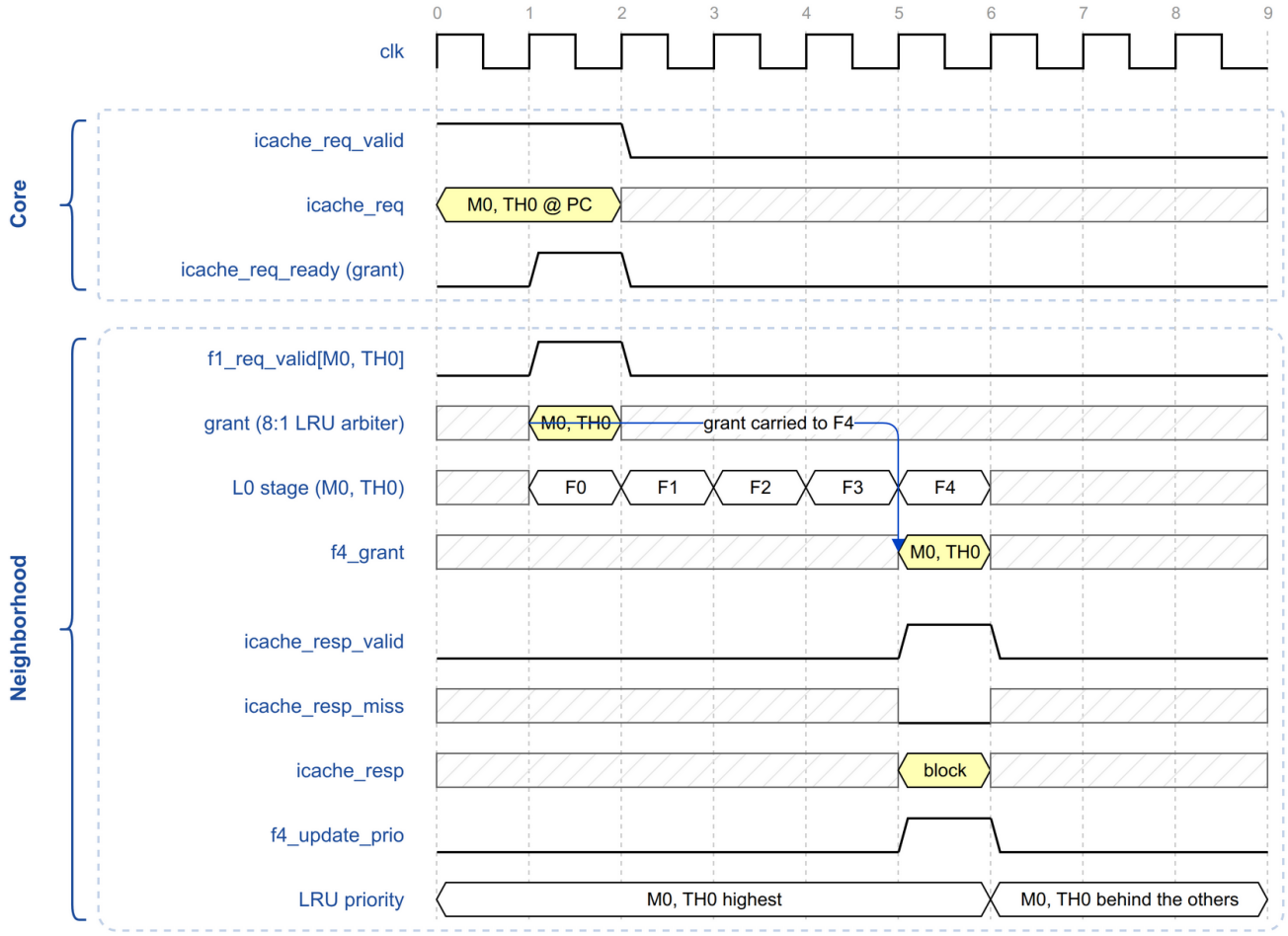



*Figure 24. Request and hit response at the L0 micro-cache*

### 3.3.5. Miss, Sleep and Fill-Done Protocol

A request misses in the L0 micro-cache either because the instruction TLB has no translation for it, or because the line is not in the micro-cache:

- **TLB miss.** A request is sent to the shared page-table walker (Section 3.6), and the lookup continues down the pipeline, so the requesting thread receives its miss response with the normal timing.
- **Tag miss.** A request for the line is sent to the L1 instruction cache. The two L0 micro-caches share one request port to the L1 and are selected by a round-robin arbiter, each with a holding register for its request.

In both cases the core that made the request receives a miss response and puts the requesting thread to sleep.

When the fill completes, that is, when the translation is in the TLB or the line is in the micro-cache data array, the L0 micro-cache pulses `icache_fill_done`. The pulse is common to all eight threads it serves and carries no address, so every sleeping thread wakes and retries. A thread whose line is still absent misses again and sleeps until the next fill.

Fill done is deliberately driven one stage later than the response. A lookup that reaches F2 in the cycle the fill is written hits, because the micro-tag array checks the tag being written as well as the tags it already holds. The last lookup that misses on the line is the one in F3 while the line is

written, so its miss response is returned two cycles before `f5_resp_fill_done`. Every thread has therefore entered the sleep state before its wake-up arrives, and no wake-up is lost.

Each L0 micro-cache has one miss handler, which holds one miss at a time; it is described in Section 3.3.8.1. Figure 25 follows one thread through a miss and its retry. The lookup misses in F2, the handler leaves `Ready`, and the request to the L1 instruction cache is driven in F3. The response in F4 is a miss, so the thread sleeps and its priority is left unchanged; the grant has already freed both the core's F1 request register and the Neighborhood's per-thread register.

The break in the waveform stands for the access time of the L1 instruction cache, and of the Shire Cache behind it if the L1 misses as well. That time is not fixed and is far longer than the rest of the sequence. The handler stays in `Fill_Wait` for all of it, so every other lookup that misses meanwhile is turned away, and the core leaves the thread asleep and issues nothing for it. When the line arrives the handler returns to `Ready` and fill done is pulsed; the thread wakes, requests again and hits, and only that second response updates the priority.

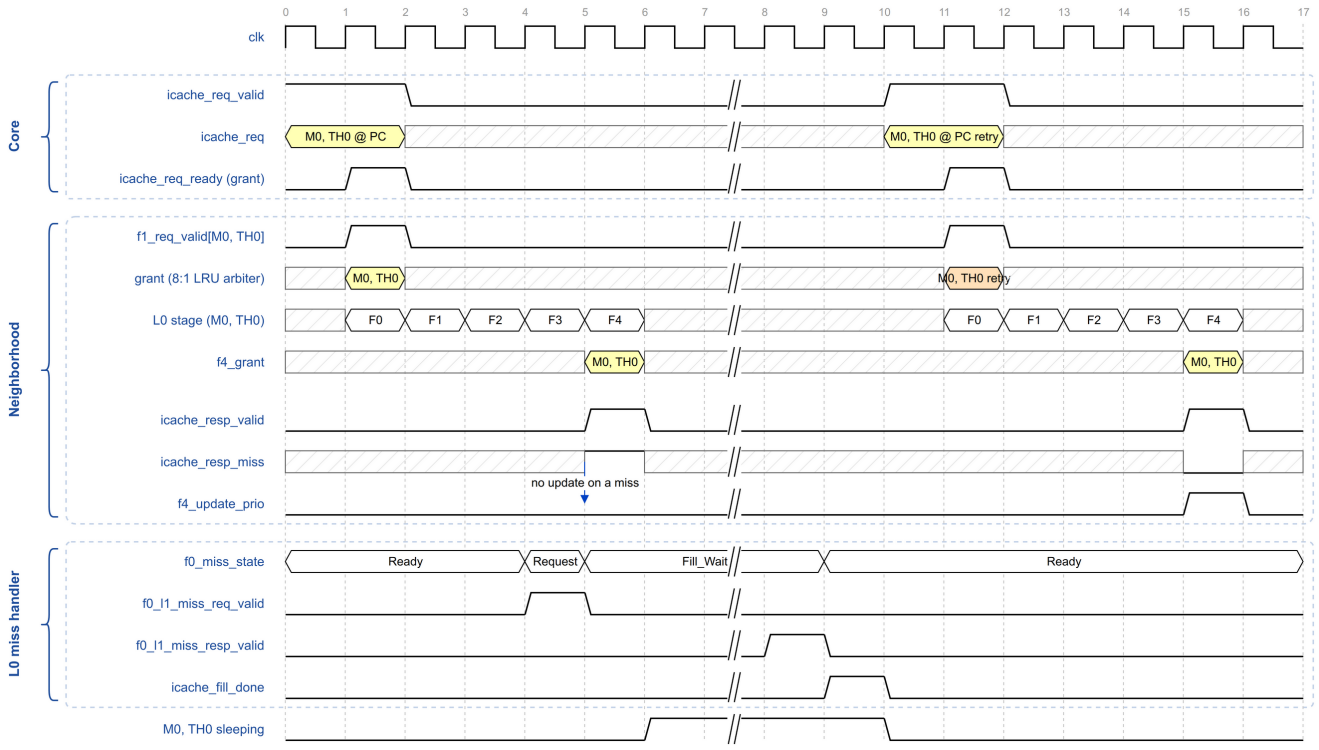



*Figure 25. Miss, sleep, fill done and retry*

## 3.3.6. Fault and Error Reporting

The response carries the fault and error status of the fetch together with the data:

| Field | Meaning |
|---|---|
| `page_fault` | The instruction TLB could not translate the address, or the translation forbids the access. Reported whether the lookup hit or missed. |
| `access_fault` | The PMA check rejected the physical address. Reported whether the lookup hit or missed. |
| `cacheable` | The block comes from a cacheable region, as reported by the PMA check. Valid on a hit. |

| Field | Meaning |
|---|---|
| `ecc_err` | The line was filled with an ECC error. The micro-cache has no ECC of its own: it stores the error flag that arrived with the fill next to the line and returns it on every hit on that line, until the line is replaced. Only the error classes enabled in the error-log control ESR set the flag. Valid on a hit. |
| `bus_err` | The line was returned with an error by the memory hierarchy. Valid on a hit. |

A response that reports a fault is delivered like any other response, and the core marks the faulting instruction so that the integer pipeline raises the exception (Section 2.2.5.6). Errors detected in the L1 instruction cache data RAM are also logged in the Neighborhood ESRs, which hold a sticky single-bit and double-bit error flag and, for the access that was logged, its way, set and physical address and the ECC blocks of the line in which the error was found.

### 3.3.7. Complete Path for Four Cores

Figure 26 puts the pieces together: four cores requesting in the same cycle, the two flops each request passes through, the arbitration between the threads, and the five responses, one for each of the five threads that were granted. In the `LRU order` lane alone the threads are written as digits, numbered 2 × Minion + thread, because eight names do not fit in one cycle; the lane lists them highest priority first.

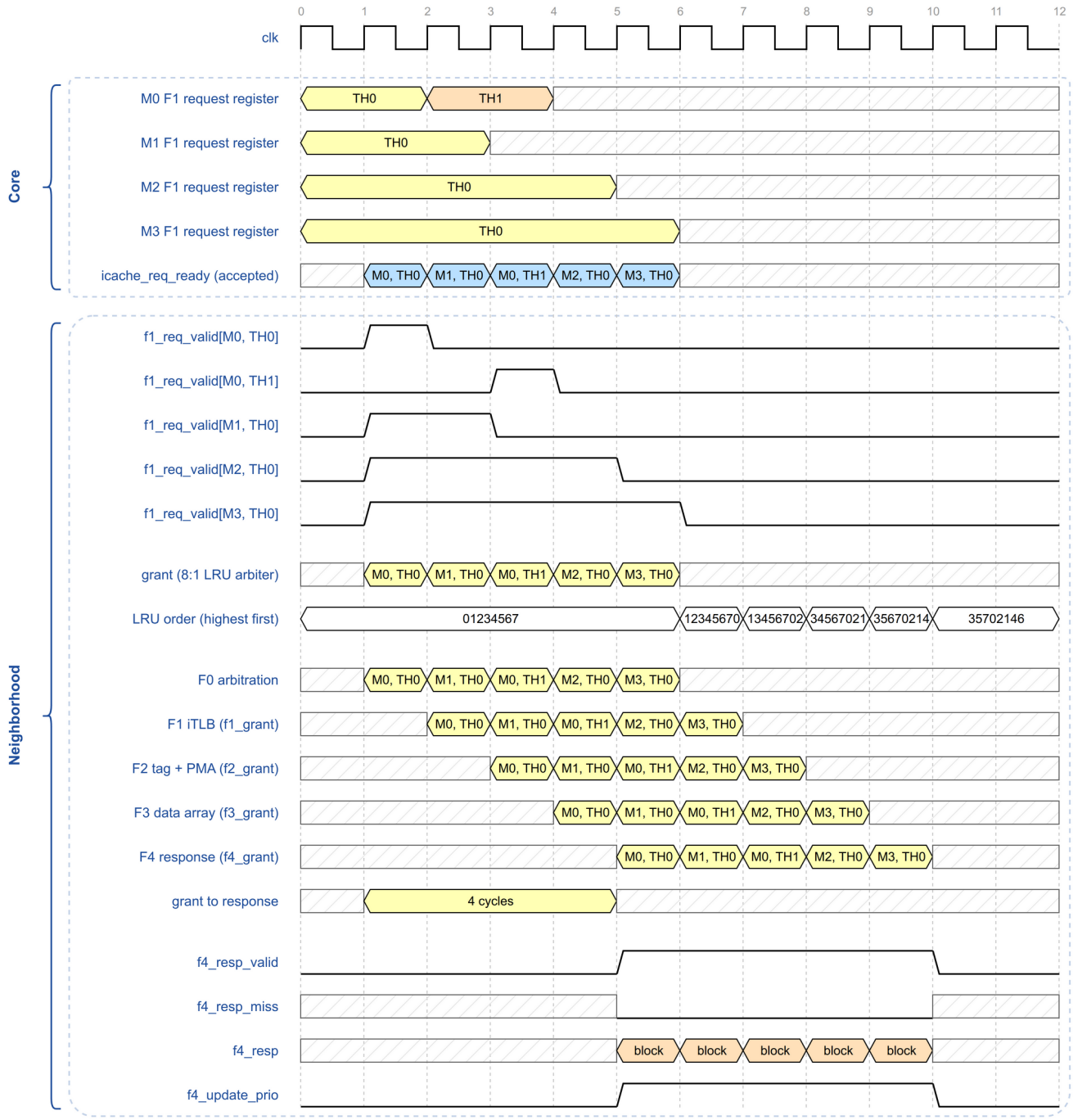



*Figure 26. Request capture, arbitration and response for four cores*

In cycle 0 the four cores present a TH0 request each, and the F1 registers hold them. The thread registers go valid in cycle 1, M0, TH0 wins and `icache_req_ready` frees M0's F1 register, which reloads with TH1 in cycle 2. M1, TH0 is granted in cycle 2, and in cycle 3 M0, TH1 is granted, while M2 and M3 are still holding their first request.

M0 is therefore served twice before M2 and M3 are served once, and the reason is that the aging matrix has one entry per thread, not one per Minion. Granting M0, TH0 demotes M0, TH0 and nothing else, so M0, TH1 keeps a position that is still ahead of M2 and M3 and wins the arbitration in cycle 3. The deferred update does not cause this: if the priority update occurs at the grant instead of at the response, M0, TH1 still wins in cycle 3.

Per-thread entries are what make the arbitration fair, because the two threads of a core succeed and fail independently. One may hit while the other misses, and a single entry shared by both

would demote the thread that missed every time its sibling was served, costing it the position it needs to win its retry after the fill. What the deferred update adds is the next step: a thread whose own response is a miss keeps its place, so it wins its retry immediately, and the miss-count guard bounds how long any thread can be passed over (Section 3.3.8).

The response for M0, TH0 appears in F4 in cycle 5, four cycles after its grant, with `f4_grant` naming the thread. It hits, `f4_update_prio` is asserted and the order changes from `01234567` to `12345670`.

### 3.3.8. Livelock and Starvation Avoidance

Up to eight threads share each L0 micro-cache, and the two threads of a core share its fetch port, decoders and issue slot. At every shared point one thread can be delayed by the others. Each shared point therefore has a policy that limits how long a thread can be held back: LRU or round-robin arbitration, a priority rule that favours threads whose requests missed, a wake-up protocol that never loses a wake-up, and a miss-count guard that gives a starving thread exclusive access to the L0 micro-cache. Table 30 lists these mechanisms, and the subsections that follow describe them.

Figure 27 adds to the interface of Figure 23 the two blocks that carry the guard: the per-thread miss counters, one set per micro-cache, and the mask they drive, which gates the bids into the 8:1 LRU arbiter.

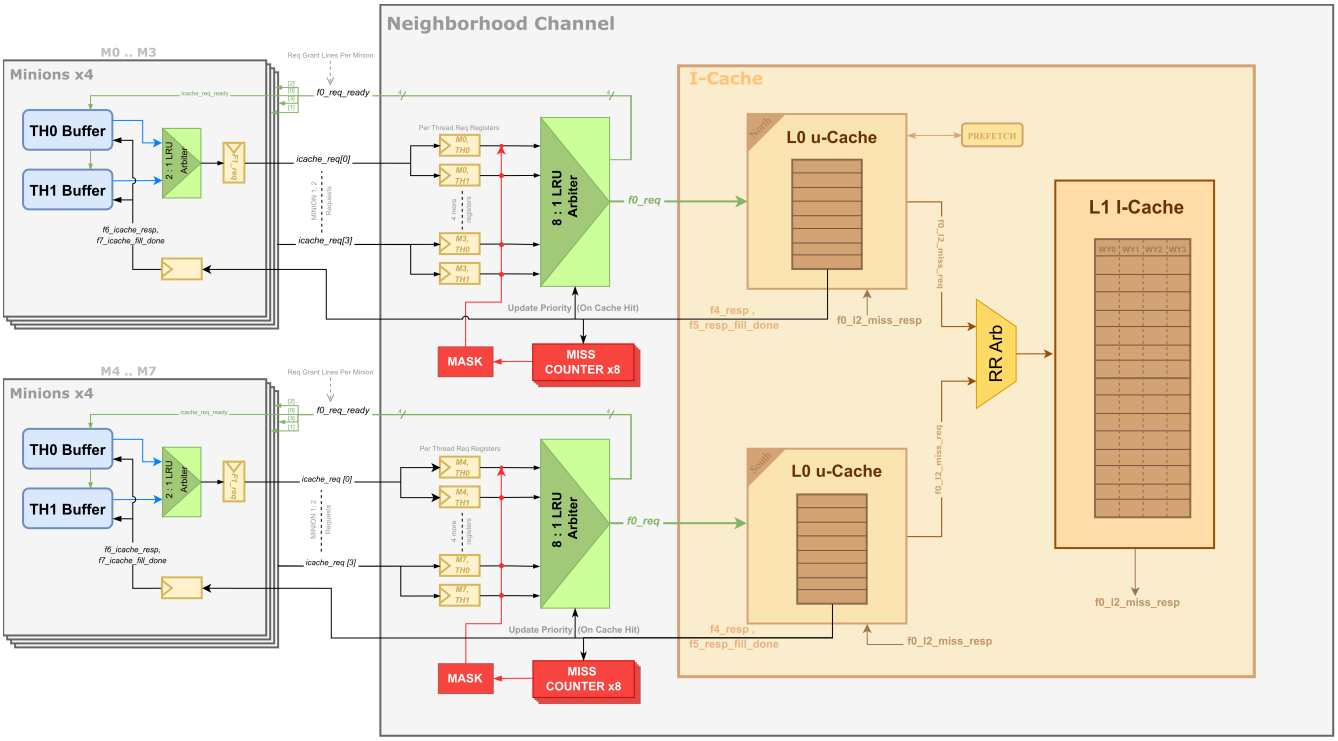



*Figure 27. Miss counters and bid mask in the fetch path*

*Table 30. Starvation avoidance mechanisms in the instruction fetch path*

| Shared resource | Contenders | Mechanism | Guarantee |
|---|---|---|---|
| Core fetch port (F1 request register) | 2 threads of a core | 2:1 LRU arbiter, priority updated on every grant | A waiting thread is granted after at most one request of the other thread |

| Shared resource | Contenders | Mechanism | Guarantee |
|---|---|---|---|
| L0 request arbiter | 8 threads of 4 cores | 8:1 LRU arbiter whose priority is updated only on a hit or a fault | A thread whose request missed keeps its priority for its retry |
| L0 miss handler | 8 threads and the prefetcher | Fill-done broadcast with ordered timing; thread wakes on fill done, redirect, disable or halt | A thread sleeps only while a fill is in progress, and always wakes |
| L0 miss handler (repeated misses) | 8 threads | Per-thread consecutive-miss counters and bid mask | A thread with 31 consecutive misses gets exclusive access for its core until it hits |
| L0 miss handler (prefetch) | Prefetcher and threads | Prefetch requests enter the L0 micro-cache only in cycles with no core request | Demand fetches always win the L0 pipeline over prefetch |
| L0 miss handler (retry windows) | Prefetcher and waking threads | 8-cycle prefetch pause after each fill done; prefetch blocked after an unattended miss until the next fill done, then until a core request or 16 cycles | Waking threads get the first chance at the freed miss handler, and prefetching is never blocked for good |
| L1 I-cache miss port | 2 L0 micro-caches | Round-robin arbiter | Each L0 micro-cache waits for at most one miss of the other |
| Decoders and ID issue slot | 2 threads of a core | Alternating decode selection and round-robin thread scheduler, per-thread FIFOs and stalls | Each eligible thread issues at least every other cycle |

The mechanisms in the first two rows are described with the hardware that implements them: the 2:1 LRU arbiter of the core fetch port in Section 2.2.3, and the 8:1 LRU arbiter with its deferred priority update in Section 3.3.3. The code prefetcher and the two timed windows that hold it behind every thread are described in Section 3.4.11, the round-robin arbiter between the two micro-caches in Section 3.5.5, and the decoders, instruction FIFOs and thread scheduler in Section 2.2.7 and Section 2.2.9. What remains, and what the rest of this section describes, is the one shared resource with no arbitration of its own: the miss handler of the L0 micro-cache.

#### 3.3.8.1. The L0 Miss Handler

The miss handler is described in full in Section 3.4.8. Enough of it is repeated here for this section to stand on its own.

Each L0 micro-cache has one miss handler: the logic that handles the L0 micro-cache's single outstanding request to the L1 instruction cache. It consists of the following:

1. a three-state machine (`f0_miss_state`: Ready, Request, Fill_Wait),

2. a register holding the physical address of the missing line (`f0_miss_addr`), and
3. a flag indicating the miss as a prefetch miss (`f0_miss_is_prefetch`).

A miss is accepted only in the Ready state. The handler then sends the request to the L1 instruction cache (Request state), waits for the line (Fill_Wait state), writes it into the L0 micro-cache unless it is a prefetch, and returns to the Ready state, at which point fill done is generated.

Behind it, the L1 instruction cache is also single-transaction. It accepts one request, closes its request port until that request's response, and always answers with the block: on an L1 hit from its data RAM, on an L1 miss once the line has come back from the Shire Cache. A micro-cache miss that reaches the L1 therefore occupies the L1 for its whole duration, which on an L1 miss is a full round trip out of the Neighborhood.

The code prefetcher shares this same handler. It never delays a fetch in the same cycle, because a prefetch request enters the L0 pipeline only in cycles with no core request, but after a prefetch takes the handler, the prefetcher holds the handler exactly as a demand miss does (Section 3.4.11).

A lookup that does not hit returns a miss response to the thread in one of the following three cases:

1. The line is not present in the L0 micro-cache and the handler is idle, so the handler accepts the miss and sends it to the L1 instruction cache.
2. The line is not present in the L0 micro-cache but the handler is busy with another miss, so the miss is not recorded and no request is sent to the L1 instruction cache for it.
3. The address misses in the L0 TLB, so a translation is requested from the page-table walker (Section 3.6) and no request is sent to the L1 instruction cache for it.

In all three cases the thread enters the sleep state (`f0_miss_pending`) and no request is issued. In the second and third cases nothing is requested for that thread, and it is the second that this section is about.

#### 3.3.8.2. Sleep and Wake-Up

The L0 micro-cache pulses `f6_icache_fill_done` to all its threads whenever an L1 fill completes, whether it is a demand miss or a prefetch, and whenever a translation fill completes. On seeing the fill done every sleeping thread wakes and retries according to the arbitration priority. A thread whose line is now present hits, including any thread that missed on the line already being fetched, and a thread whose line is still absent misses again; if the handler is now idle, its miss is sent to the L1 instruction cache.

A thread sleeps because its own response was a miss; that is the only thing that puts it to sleep. What decides when it is woken is the task of the miss handler at that moment: the thread's own miss, the fill of another thread that keeps the handler busy, or a translation. In all three a fill is in progress, so every sleeping thread is woken when it receives the fill-done signal. The fill-done timing (Section 2.2.4.3) makes sure that no wake-up is lost: fill done is issued one cycle after the last response the fill can affect, and it is registered again in the Front End. A redirect, a thread disable or a pending halt clears the sleep state of the affected thread only; fill done is the only event that wakes all sleeping threads.

Which thread is served in a round is decided in F2. That is the stage where the tag is compared,

where a miss is therefore first indicated, and where the handler accepts it and flops its state so that the request to the L1 instruction cache goes out in F3 (Table 29). A lookup that reaches F2 any later in the round finds the handler no longer Ready.

Two properties of this protocol follow, and they set up the rest of the section. Because the handler accepts a miss only while it is Ready, and because it is Ready only between one fill completing and the next miss being accepted, each fill-done round serves exactly one new miss. And because the whole set of sleeping threads is woken at once, the thread that is served in a round is whichever one reaches F2 first with a miss.

#### 3.3.8.3. How a Thread Can Be Left Behind

The thread that reaches F2 first is not always the thread that waited longest. A thread that was woken by something other than fill done is already moving while the others are still asleep, and it keeps that head start all the way down the pipeline.

The following sequence is built from ordinary events and needs no fault, no error and no unusual configuration.

**Starting point.** Each fetch returns a 32-byte block, half an L0 line, and a thread buffer holds two of them (Section 2.2.5). Thread A has the first half of line X in its buffer and executes from it. It requests the second half of X. While A executes from the first half, a fill for another thread selects X as its LRU victim and evicts it, so A's request misses. The miss handler is busy with that other fill, so A's miss is **not recorded**. A receives a miss response and sleeps.

**The redirect.** While A sleeps, the integer pipeline resolves a branch in the instructions A already consumed and redirects it. The redirect clears A's sleep bit at once, and A presents a request for the branch target without waiting for any fill.

**The capture.** A's new request is in F1 when `icache_fill_done` arrives for the other thread's line. The seven other threads wake on that pulse, but they are a cycle behind: they must be granted before they enter F0, while A is already one stage inside the pipeline. A reaches F2 first, misses on the branch target, finds the handler just returned to `Ready`, and takes it.

**The others.** Each of the seven woken threads reaches F2 one or more cycles later and finds the handler in `Request` or `Fill_Wait`. Every one of their misses is discarded. They receive miss responses, sleep again, and because a miss response does not update the arbitration priority, the next round begins in exactly the same order as this one.

No part of the design is at fault: the arbiter grants A on merit, and the miss handler accepts its miss on merit. The point is that a redirect let A bypass the queue of sleeping threads entirely, and that one fill-done round can serve only one miss. A thread that is redirected often enough, or a prefetch engine that is already in the pipeline when the threads wake, can take that single opportunity repeatedly.

The result is a thread that is granted in every round, answered in every round, and never once has a line fetched on its behalf. Its arbitration priority does not change, because priority is only updated on a hit or a fault, so the arbiter cannot lift it out of the position it is in. Raising its priority would not be enough in any case: priority decides who enters the pipeline first, and the thread that enters first is not necessarily the thread that finds the handler idle.

This is the case the miss-count guard exists for.

#### 3.3.8.4. Miss-Count Livelock Guard

The Neighborhood counts, for each thread, the number of consecutive miss responses to that thread's own requests (`th_miss_cnt`, 5 bits, saturating at 31). The counter advances only when that thread's own response returns, so a thread that is not being granted does not accumulate.

While no counter is saturated, every thread may bid. When one or more saturate, a mask register (`th_miss_mask`) allows only the two threads of one Minion: the lowest-numbered Minion that has a saturated thread. Both of its threads are allowed, because the Minion holds one request at a time and the sibling's request may have to be accepted before the starving thread's can be presented at all. The mask is recomputed one cycle after every response.

The mask does not discard anything. A bid is the thread's request-register valid qualified with the mask, so masking only removes the thread's valid at the arbiter: the request itself stays in its register, untouched, and bids again in the cycle the mask is released. Nothing is re-issued and no state is lost.

With every other Minion masked, only the selected Minion's misses can reach the handler, so the starving thread's line is fetched and cannot be displaced before the thread retries. The thread then hits, which clears its counter, and the mask passes to the next Minion with a saturated thread or is removed if there is none. The mask is released by a hit, and only by a hit.

Figure 28 shows the guard at work, with the waits between rounds compressed into breaks. Every miss costs M0, TH0 a full round: the miss response puts it to sleep, fill done wakes it, it requests again and is granted, and its next response arrives four cycles after that grant.

The first response shown takes the counter to 31. The counter is loaded at the end of that cycle, so `th_miss_cnt_sat` only reads one in the next, and `th_miss_mask_en` is a registered copy of the counter enable, so the mask itself changes one cycle after that: the restrictive mask is in force two cycles after the response that saturated the counter. From that point only the two threads of Minion 0 bid.

Being alone does not give M0, TH0 its line, it gives it the miss handler. On the next round its lookup is the only one to reach F2, so its miss is accepted: `f0_l1_miss_req_valid` is driven in F3 and `f0_l1_miss_req` carries that thread's line address out to the L1 instruction cache. The response in F4 is still a miss and the thread sleeps again; the counter holds at 31 because it saturates. Only when that fill completes and the thread retries does it hit, and that hit clears the counter and releases the mask.

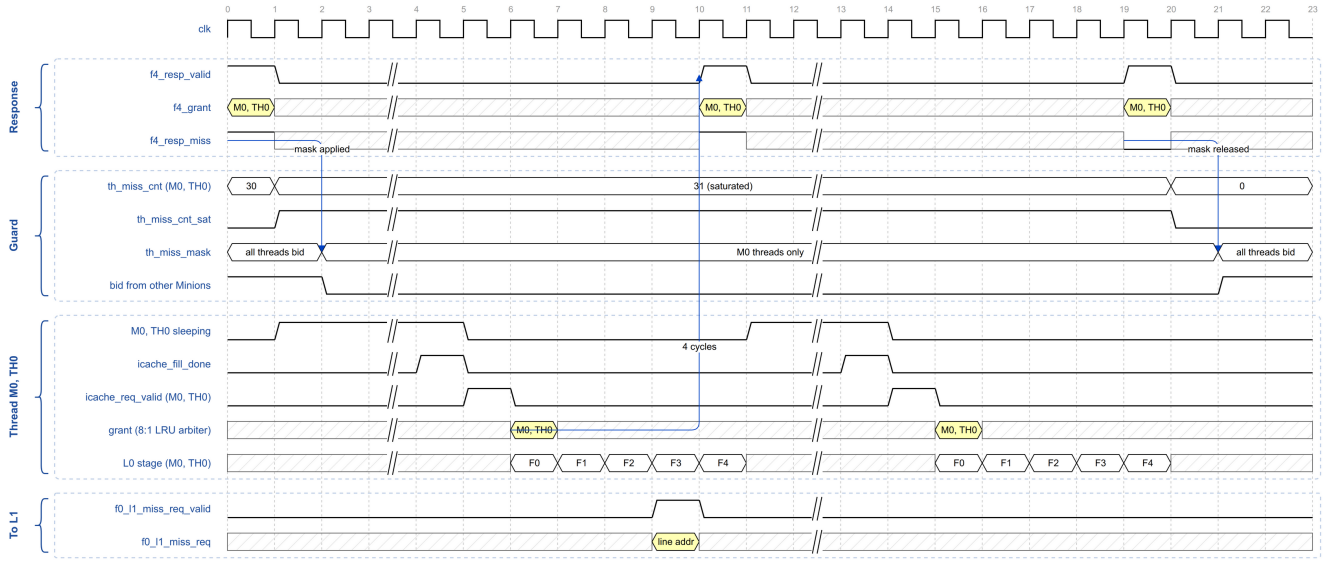



*Figure 28. Miss-count guard: saturation, bid mask and release*

#### Counters and Error Responses

A response updates the counters of more threads than the one it belongs to. Table 31 lists the cases.

*Table 31. Effect of a response on the per-thread miss counters*

| Response returned in F4 | Counter of the thread it belongs to | Counters of the other seven threads |
|---|---|---|
| Hit | Cleared | Unchanged |
| Miss, no fault or error | Incremented, saturating at 31 | Unchanged |
| Page fault or access fault | Cleared | Cleared |
| Bus error or ECC error | Cleared | Cleared |

The first two rows are the counter doing its job. The last two are deliberate: an error clears every thread's counter, not only the counter of the thread that received it.

The reason is a corner case in which a core would otherwise hang on a faulting address. A faulting thread never hits on that address, so its counter would saturate and it would hold exclusive access, which is only ever released by a hit. Clearing the counters and the mask lets the fault be delivered and handled normally rather than leaving a mask in place based on stale counts. If the fetch pattern that caused the pressure persists, the counters saturate again and the guard is triggered again.

The cost of clearing all eight is stated in Section 3.3.8.5: a sustained stream of faulting fetches from any thread defers the guard for all of them.

### 3.3.8.5. Assumptions

These mechanisms guarantee forward progress in the fetch path under the following conditions:

- Every L1 miss request eventually returns a fill, and every translation request eventually completes.

- Software does not keep a thread's exclusive mode, `WFI` or other architectural stalls asserted indefinitely. These deliberately stop the affected thread and are not starvation.
- Error responses are not sustained. Because an error clears every thread's miss counter, a continuous stream of faulting fetches from other threads would defer the livelock guard. In practice each fault redirects the faulting thread to its trap handler, so such a stream does not persist.

### 3.3.9. Known Limitations

The properties below follow from the design of the interface. They are not defects, but they constrain how the interface may be changed and how it behaves under load.

- **Response carries no thread identifier.** A core matches a response to its request by counting cycles, so the fixed response latency is part of the interface contract. A change to the depth of the L0 pipeline has to be matched in the Front End.
- **Fill done carries no address.** Every fill wakes every sleeping thread of the micro-cache. Threads waiting for a different line arbitrate, miss again and sleep again, which costs L0 bandwidth on every fill and grows with the number of threads waiting on different lines.
- **One outstanding miss per L0 micro-cache.** A miss that arrives while the miss handler is busy is answered with a miss response but is not recorded, so its line is not requested. That thread makes progress only after an unrelated fill wakes it and it retries.
- **Starving Minions are served in a fixed order.** When several Minions have a saturated thread, the mask always selects the lowest-numbered one first. Its waiting time is bounded, because each exclusive period ends when the selected thread hits, but the order is not fair.
- **The L1 instruction cache is blocking.** It accepts one request at a time and holds its request port closed until that request's own response, which on an L1 miss waits for the line to come back from the Shire Cache. The round-robin bound of one transaction of the other micro-cache is therefore correct as a count but stands for a full round trip out of the Neighborhood, during which the other micro-cache cannot even begin a lookup that might have hit.

## 3.4. L0 Micro-Cache

### 3.4.1. Overview

The L0 micro-cache (`icache_micro_cache`) is the first level of the Neighborhood's shared instruction cache: a fully associative, physically tagged cache of 16 lines of 64 bytes, 1 KB in all. Each Neighborhood has two instances, each shared by four Minions, eight threads: the north micro-cache serves Minions 0 to 3 and the south micro-cache serves Minions 4 to 7. The cache is split in two so that each instance is next to the Minions it serves, which reduces congestion in the Neighborhood channel. The two instances are identical except for the code prefetcher, which loads code into the L1 instruction cache at the request of software and is active only in the north micro-cache.

The micro-cache has a single request port, so the eight threads of an instance are arbitrated in the Neighborhood by an 8:1 LRU arbiter (Section 3.3.3), which presents the virtual fetch address of one request per cycle. The micro-cache translates the address with its own instruction TLB and, on a hit,

returns a 32-byte instruction block, half a line. On a miss it requests the complete 64-byte line from the L1 instruction cache, the next level, which both micro-caches share.

The design rests on four decisions:

1. **Never stalls, fixed latency:** a miss does not block the requests behind it, so the cache accepts one request in every cycle, serves hits while a miss is outstanding, and returns a response for every request, hit or miss, four cycles after the grant. The Front End depends on the fixed latency, because it recognizes its responses by timing alone (Section 2.2.4.2).
2. **Translate, then look up:** the tags are physical, so the instruction TLB translates the fetch address in the cycle after the request is accepted, and the physical tag is compared with all 16 lines in the next cycle.
3. **One fill at a time:** one line request to the L1 instruction cache and one page-table walk can be outstanding. A miss that finds the resource it needs busy is answered but not recorded, and every completed fill wakes all sleeping threads (Section 3.3.5).
4. **One TLB for four Minions:** each Minion owns four of the 16 TLB entries, except in cooperative mode, a Shire-wide setting held in the `shire_coop_mode` ESR, in which the four Minions share all 16 entries.

Table 32 lists the submodules. The sections that follow describe the interface (Section 3.4.2), the organization and address decode (Section 3.4.3), the pipeline (Section 3.4.4), the instruction TLB (Section 3.4.5), the arrays and the replacement policy (Section 3.4.6), the PMA check (Section 3.4.7), miss handling (Section 3.4.8), flush (Section 3.4.9), bypass mode (Section 3.4.10), the code prefetcher (Section 3.4.11) and debug access (Section 3.4.12).

*Table 32. L0 micro-cache submodules*

| Submodule | Function |
|---|---|
| Instruction TLB (`icache_tlb_array`, wrapping `tlb_top`) | Translates the fetch address. Requests a page-table walk on a miss. |
| Micro-tag array (`icache_micro_tag_array`) | 16 tags with their valid bits. Compares the physical tag with all lines in parallel and generates the one-hot hit vector. |
| Micro-data array (`icache_micro_data_array`) | 16 lines with their error flags. Read with the one-hot hit vector that the micro-tag array generates. |
| PMA unit (`icache_pma_unit`) | Checks the physical address and reports an access fault and whether it is cacheable. |
| LRU selector (`lru_sel`) | Chooses the victim among the 16 lines. |
| Miss handler (inline) | Holds the one outstanding line request to the L1 instruction cache. |
| Code prefetcher (inline) | Generates prefetch requests. Active only in the north micro-cache. |
| Bypass register (inline) | Holds the last fetched line while the cache is bypassed. |
| APB debug access (inline) | Debug reads and writes of the micro-tag and micro-data arrays. |

Figure 29 shows how the submodules connect. A request from a core or from the code prefetcher passes the arbiter and the instruction TLB, and its physical address goes to the micro-tag array, the bypass register and the miss handler. The hit and miss logic forms the response to the Minions. The miss handler sends the line request to the L1 instruction cache, and supplies the victim and the tag when the line returns. The translation returns from the shared page-table walker at the top, and the line returns from the L1 instruction cache at the right. The walk state machine of the instruction TLB is drawn as a separate block, PTW FSM.

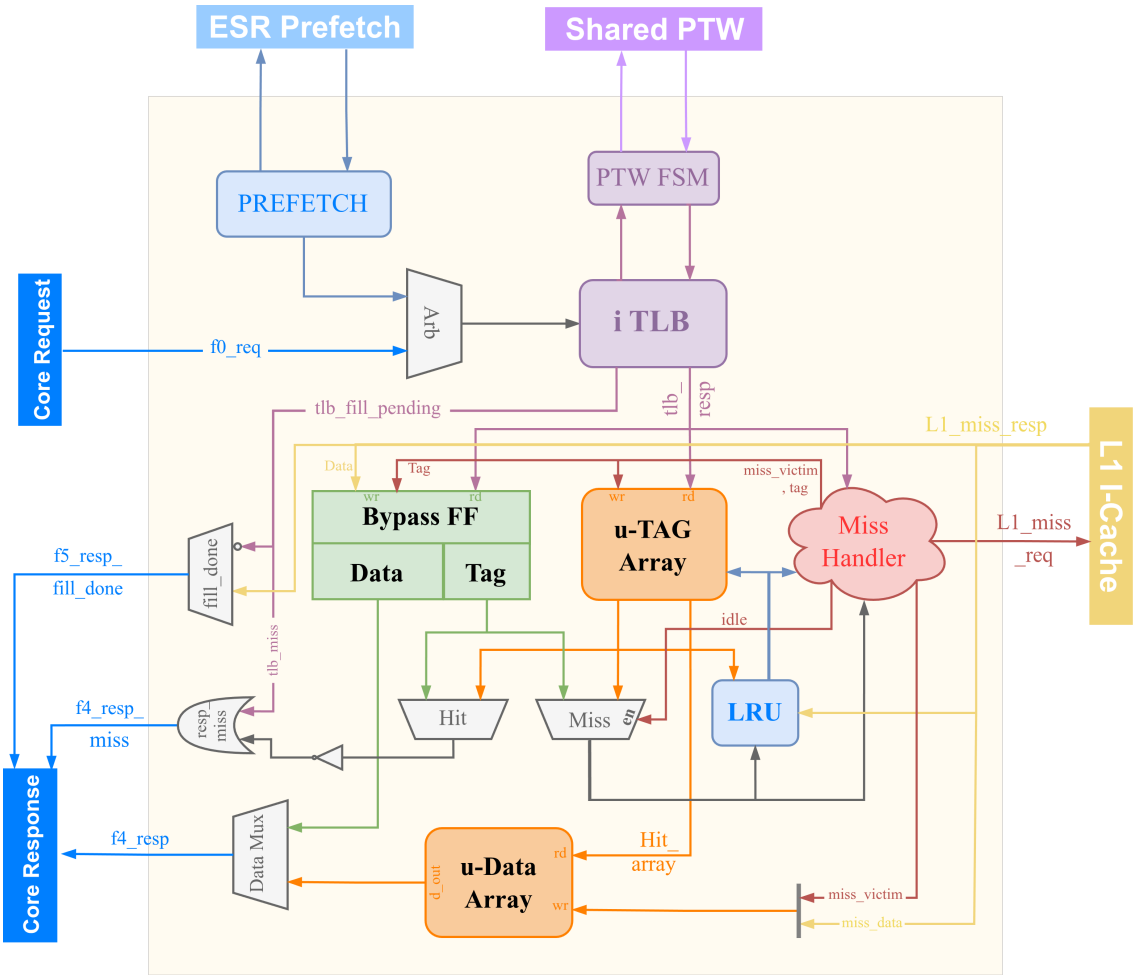



*Figure 29. L0 micro-cache functional block diagram*

The functional block diagram does not show the PMA unit (Section 3.4.7) or the APB debug access (Section 3.4.12). Both appear in Figure 31, the detailed pipeline diagram, which places every block in the stage where it works. The sections that follow draw each transaction on the functional block diagram, starting with a core request hit in Section 3.4.4.

## 3.4.2. Interface

This section defines the boundary of `icache_micro_cache`. Section 3.4.2.1 lists its ports in eleven groups: system, ESR configuration, fetch request, fetch response, flush, line request to the L1 instruction cache, line response from it, address-translation control, page-table walker, APB debug access and the Status Monitor outputs. Section 3.4.2.2 gives the field layout of the packed structures that cross these ports.

#### 3.4.2.1. I/O Signals

Table 33 lists the ports of `icache_micro_cache`. The module is instantiated with `NR_MINIONS` set to 4, so the per-Minion ports are four wide and `f0_req_min_id` is 2 bits.

*Table 33. L0 micro-cache I/O signals*

| Signal | Dir | Width / type | Description |
|---|---|---|---|
| **System** | | | |
| `clock` | in | 1 | Clock. |
| `reset` | in | 1 | Reset. Synchronous, active high. |
| **ESR configuration** | | | |
| `esr_prefetch_conf` | in | `icache_prefetch_conf_t` | Privilege, start line and line count of a prefetch operation. Zero in the south instance. |
| `esr_prefetch_start` | in | 1 | Starts a prefetch operation. A pulse during an operation cancels that operation and starts the new one. Tied low in the south instance. |
| `esr_prefetch_done` | out | 1 | Cleared when the prefetcher starts an operation, set when the operation completes. Resets high. The north instance drives `esr_prefetch_done` of `icache_top`. |
| `esr_mprot` | in | `esr_mprot_t` | Memory-protection setting for the PMA check. |
| `esr_vmspagesize` | in | `tlb_entry_type` | Page size of every TLB entry: 4 KB, 2 MB or 1 GB. |
| `esr_bypass_icache` | in | 1 | Bypasses both cache levels. Driven by the `bypass_icache` chicken bit. Sampled with each request in F0. |
| `esr_shire_coop_mode` | in | 1 | Cooperative mode: the four Minions share all TLB entries. |
| **Fetch request** | | | |
| `f0_req_valid` | in | 1 | Request granted by the 8:1 LRU arbiter of the Neighborhood. |
| `f0_req` | in | `frontend_icache_req` | Thread, virtual-memory status and fetch address. |
| `f0_req_min_id` | in | 2 | Minion that made the request, 0 to 3 within the instance. Selects its translation registers and, outside cooperative mode, its TLB entries. |

| Signal | Dir | Width / type | Description |
|---|---|---|---|
| f0_req_ready | out | 1 | Tied high. The pipeline never stalls. |
| **Fetch response** | | | |
| f4_resp_valid | out | 1 | Response to a core request, four cycles after the grant. Not driven for prefetch requests. |
| f4_resp_miss | out | 1 | Line or TLB miss. The response carries no instruction data. |
| f4_resp | out | icache_frontend_resp | 32-byte block and status. |
| f5_resp_fill_done | out | 1 | A line or translation fill is complete. For a line, pulsed three cycles after f0_l1_miss_resp_valid. |
| **Flush** | | | |
| f0_flush_data | in | 1 | Invalidates all 16 lines. It also flushes the L1 instruction cache, and it is the OR of the flush requests of the eight Minions, registered once in neigh_shared_icache. |
| **Line request to the L1 instruction cache** | | | |
| f0_l1_miss_req_valid | out | 1 | The miss handler requests a line. Driven from F3 of the missing request until accepted. |
| f0_l1_miss_req_addr | out | 40 | Physical address of the missing line. |
| f0_l1_miss_req_ready | in | 1 | The holding register of this instance in icache_top is free. |
| **Line response from the L1 instruction cache** | | | |
| f0_l1_miss_resp_early_valid | in | 1 | Asserted one cycle before f0_l1_miss_resp_valid. Early write enable of the latch-based tag and data arrays. |
| f0_l1_miss_resp_valid | in | 1 | The line is returned and written into the victim entry in this cycle. Steered to the instance that made the request. |
| f0_l1_miss_resp_data | in | 512 | Line data. Driven to both instances. |
| f0_l1_miss_resp_ecc_err | in | 1 | ECC error in the L1 data RAM, of a class enabled in the error-log control ESR. Stored with the line. |
| f0_l1_miss_resp_l2_err | in | 1 | Error response from the Shire Cache for this line. Stored with the line. |
| **Address-translation control** | | | |

| Signal | Dir | Width / type | Description |
|---|---|---|---|
| satp_info | in | 4 x minion_satp_info | Supervisor address-translation register of each of the four Minions of this instance. |
| matp_info | in | 4 x minion_satp_info | Machine address-translation register of each of the four Minions of this instance, used for fetches in M-mode. |
| tlb_invalidate | in | 4 | Invalidates the TLB entries of one of the four Minions of this instance, or the whole TLB in cooperative mode. |
| **Page-table walker** | | | |
| ptw_req_valid | out | 1 | Translation request. Held until accepted. |
| ptw_req_data | out | minion_ptw_req | Translation mode, root page number, privilege and virtual page number. |
| ptw_req_ready | in | 1 | The walker accepts the request. |
| ptw_invalidate | out | 1 | Any tlb_invalidate, registered once and forwarded to the walker. |
| ptw_resp_valid | in | 1 | The walk is complete. |
| ptw_resp_data | in | minion_ptw_pte | Leaf page-table entry, access fault or cancellation. |
| **APB debug access** | | | |
| apb_paddr | in | 8 | Address in the debug space of the micro-cache: micro-tag array, or micro-data array and error flags. |
| apb_psel, apb_penable, apb_pwrite | in | 1 each | APB control. apb_psel and apb_penable are decoded for this instance by icache_top. |
| apb_pwdata | in | 64 | Write data. |
| apb_pready | out | 1 | Transfer complete. |
| apb_prdata | out | 64 | Read data. |
| apb_pslverr | out | 1 | Tied low. |
| **Status Monitor** | | | |
| dbg_sm_signals | out | icache_dbg_sm_t | Prefetch and miss state-machine states, F2 valid, F2 hit vector and F2 line address, for the Status Monitor debug logic. icache_top merges both instances. |

#### 3.4.2.2. Data Structures

The fetch request and response, `frontend_icache_req` and `icache_frontend_resp`, are the structures of the Front End interface and are given in Table 7 and Table 9. The structures below are used only by the instruction cache and its page-table walker.

*Table 34. `icache_prefetch_conf_t`: prefetch operation (50 bits)*

| Field | Width | Description |
|---|---|---|
| `prv` | 2 | Privilege level used to translate the prefetch addresses. |
| `start_addr` | 42 | Virtual address of the first line, bits 47:6. |
| `num_lines` | 6 | Number of lines minus one: 0 prefetches one line, 63 prefetches 64. |

*Table 35. `minion_satp_info`: address-translation register (32 bits)*

| Field | Width | Description |
|---|---|---|
| `mode` | 4 | Translation mode: Bare (0), Sv39 (8) or Sv48 (9). |
| `ppn` | 28 | Physical page number of the root page table. |

*Table 36. `minion_ptw_req`: page-table walk request (70 bits)*

| Field | Width | Description |
|---|---|---|
| `satp_mode` | 4 | Translation mode of the register used for this fetch, `satp_info` or, in M-mode, `matp_info`. |
| `satp_ppn` | 28 | Root page-table page number from the same register. |
| `prv` | 2 | Privilege level of the fetch. |
| `addr` | 36 | Virtual page number, virtual address bits 47:12. |

*Table 37. `minion_ptw_pte`: page-table walk response (64 bits)*

| Field | Width | Description |
|---|---|---|
| `canceled_req` | 1 | The walk was cancelled. The TLB is not written. |
| `access_fault` | 1 | The walk ended in an access fault. Recorded in the TLB entry and reported as an access fault when the entry hits. |
| `rsvd_for_hw` | 8 | Bit 0 is the execution mode the translation was walked for, 1 for M-mode. A TLB entry hits only for a fetch in the same mode. |
| `ppn` | 44 | Physical page number. The TLB keeps the low 28 bits. |
| `rsvd_for_sw` | 2 | Not used by the TLB. |
| `d`, `a`, `g`, `u`, `x`, `w`, `r`, `v` | 1 each | Page-table entry flags. The TLB keeps `d`, `a`, `u` and the `r`, `w`, `x` permissions; a fetch needs `x` and `a`. |

*Table 38. `esr_mprot_t`: memory-protection setting (9 bits)*

| Field | Width | Description |
|---|---|---|
| mprot_en | 1 | Enables the M-mode and S-mode protection boxes checked by the PMA unit (Section 3.4.7). |
| mmode_size | 4 | Size of the M-mode box, encoded as a power of two. |
| smode_size | 4 | Upper bound of the S-mode box, encoded as a power of two. |

tlb_entry_type is a 2-bit page size: 0 for 4 KB, 2 for 2 MB and 3 for 1 GB; 1 is reserved.

### 3.4.3. Cache Organization and Address Decode

This section gives the organization of the micro-cache, the parameters that size it, the storage they result in, and how a fetch address is divided on its way through the TLB and the tag lookup.

Table 39 summarizes the organization of one micro-cache.

*Table 39. L0 micro-cache organization*

| Property | Value |
|---|---|
| Capacity | 1 KB: 16 lines of 64 bytes |
| Associativity | Fully associative. The lookup has no index; the tag is compared with all 16 lines. |
| Tag | Physical address bits 39:6, 34 bits, and one valid bit for each line (Section 3.4.6) |
| Access size | 32-byte block, half a line, selected by physical address bit 5 |
| Replacement | LRU over all 16 lines (Section 3.4.6.3) |
| Allocation | A line is allocated only for a core miss accepted by the miss handler. Prefetch and bypassed fills are not allocated. |
| Write policy | Read-only. Lines are written only by fills and by APB debug writes, and nothing is written back. |
| Coherence | None. The micro-cache does not snoop. A change to code in memory is seen only after a flush (Section 3.4.9). |

#### 3.4.3.1. Configuration Parameters

*Table 40. L0 micro-cache configuration parameters*

| Define | Value | Description |
|---|---|---|
| ICACHE_NR_REQS | 2 | Micro-cache instances per Neighborhood. |
| NR_MINIONS (module parameter) | 4 | Minions served by one instance, eight threads. |
| ICACHE_MICRO_CACHE_SIZE | 16 | Lines. |
| ICACHE_MICRO_CACHE_ADWIDTH | 4 | Bits that number a line. log2(ICACHE_MICRO_CACHE_SIZE) = log2(16) = 4 |
| ICACHE_BLOCK_BITS | 512 | Line size, 64 bytes. |

| Define | Value | Description |
|---|---|---|
| ICACHE_BLOCK_ADDR_SIZE | 6 | Byte-offset bits in a line.<br>log2(ICACHE_BLOCK_BITS / 8) = log2(64) = 6 |
| FE_FETCH_READ_SIZE | 256 | Response block, 32 bytes. |
| ICACHE_CHUNK_ADDR_SIZE | 1 | Bits that select the block within a line.<br>log2(ICACHE_BLOCK_BITS / FE_FETCH_READ_SIZE) = log2(512 / 256) = 1 |
| ICACHE_MICRO_TAG_SIZE | 34 | Tag bits.<br>PA_SIZE - ICACHE_BLOCK_ADDR_SIZE = 40 - 6 = 34 |
| ICACHE_TLB_ENTRIES | 16 | Instruction TLB entries. |
| ICACHE_PREFETCH_MAX_LINES | 64 | Lines in one prefetch operation. |
| VA_SIZE | 48 | Virtual-address bits. |
| VA_SIZE_EXT | 49 | Fetch-address bits: the 48-bit virtual address and the extension bit, bit 48.<br>VA_SIZE + 1 = 49. The TLB detects a non-canonical address from VA[48] XOR VA[47]. |
| PA_SIZE | 40 | Physical-address bits. |
| VA_UNTRANS_SIZE, PA_UNTRANS_SIZE | 12 | Page-offset bits, not translated. |
| PTW_PG_IDX_SZ | 9 | Virtual-page-number bits per page-table level. Places the TLB index and tag for 2 MB and 1 GB pages. |

#### 3.4.3.2. Storage

Table 41 lists the storage of one micro-cache. The tag array, the data array and the TLB are latch-based register files (rf_latch_1r_1w_par, rf_latch_1r_1w_dec and rf_latch_1r_1w); the tag array reads all its entries in parallel for the fully associative compare, and the data array is read with a one-hot entry select. The valid bits are flip-flops beside the arrays.

*Table 41. L0 micro-cache storage*

| Structure | Organization | Bits |
|---|---|---|
| Micro-tag array | 16 entries of a 34-bit tag, and 16 valid bits | 560 |
| Micro-data array | 16 entries of 512 data bits, an ECC-error flag and a bus-error flag | 8,224 |
| Instruction TLB | 16 entries of a 34-bit tag (per-Minion mode, not cooperative mode, Section 3.4.5.1) and 36 bits of entry data, and 16 valid bits | 1,136 |
| Bypass register | 34-bit tag, 512 data bits, two error flags and a valid bit | 549 |

#### 3.4.3.3. Address Decode

The micro-cache receives a 49-bit fetch address, the 48-bit virtual address and its extension bit, and

generates a 40-bit physical address (Figure 30). With 4 KB pages the low 12 bits, the page offset, pass through unchanged. The virtual page number, bits 47:12, goes to the instruction TLB with the extension bit, and the TLB returns a 28-bit physical page number (Section 3.4.5). The micro-cache divides the physical address into three fields: bits 39:6 are the 34-bit tag, compared with the tags of all 16 lines; bit 5 selects the 32-byte half of the line that is returned; and bits 4:0 are not used by the micro-cache, because the Front End extracts the instruction from the block (Section 2.2.5.4).

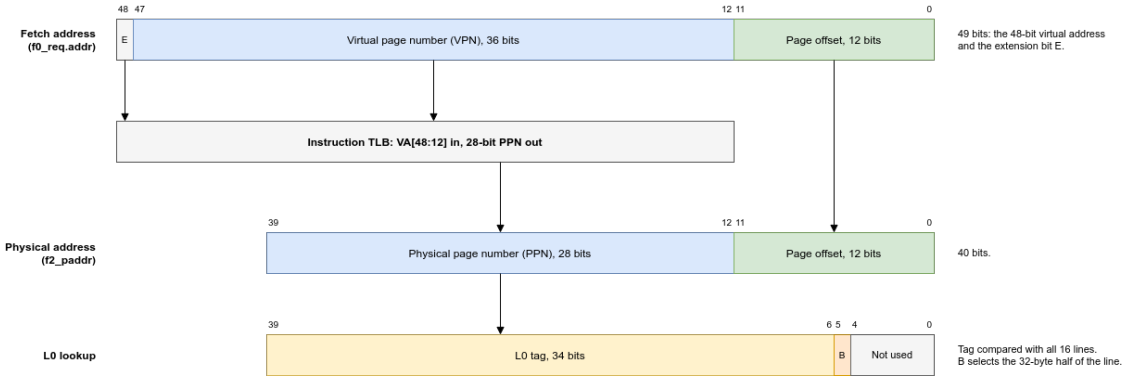



*Figure 30. Address decode from fetch address to L0 lookup*

## 3.4.4. Pipeline Stages

This section describes the five L0 micro-cache stages, F0 to F4, and the fill-done stage F5. A lookup runs from F0 to F4: a request enters in F0, is translated by the instruction TLB in F1, is compared with the tags and checked by the PMA unit in F2, reads the data array in F3, and returns its response in F4. A miss accepted in F2 sends its line request to the L1 instruction cache in F3. The fill runs independently of the lookup, and fill done is driven in F5, three cycles after the fill finishes.

Figure 31 shows the north micro-cache, the instance that contains the prefetcher, with each block drawn in the stage in which it operates. Blocks outside the frame are its neighbors: the cores, the per-thread request registers and the 8:1 LRU arbiter of the Neighborhood, the page-table walker, the L1 instruction cache, and the ESR and APB interfaces.

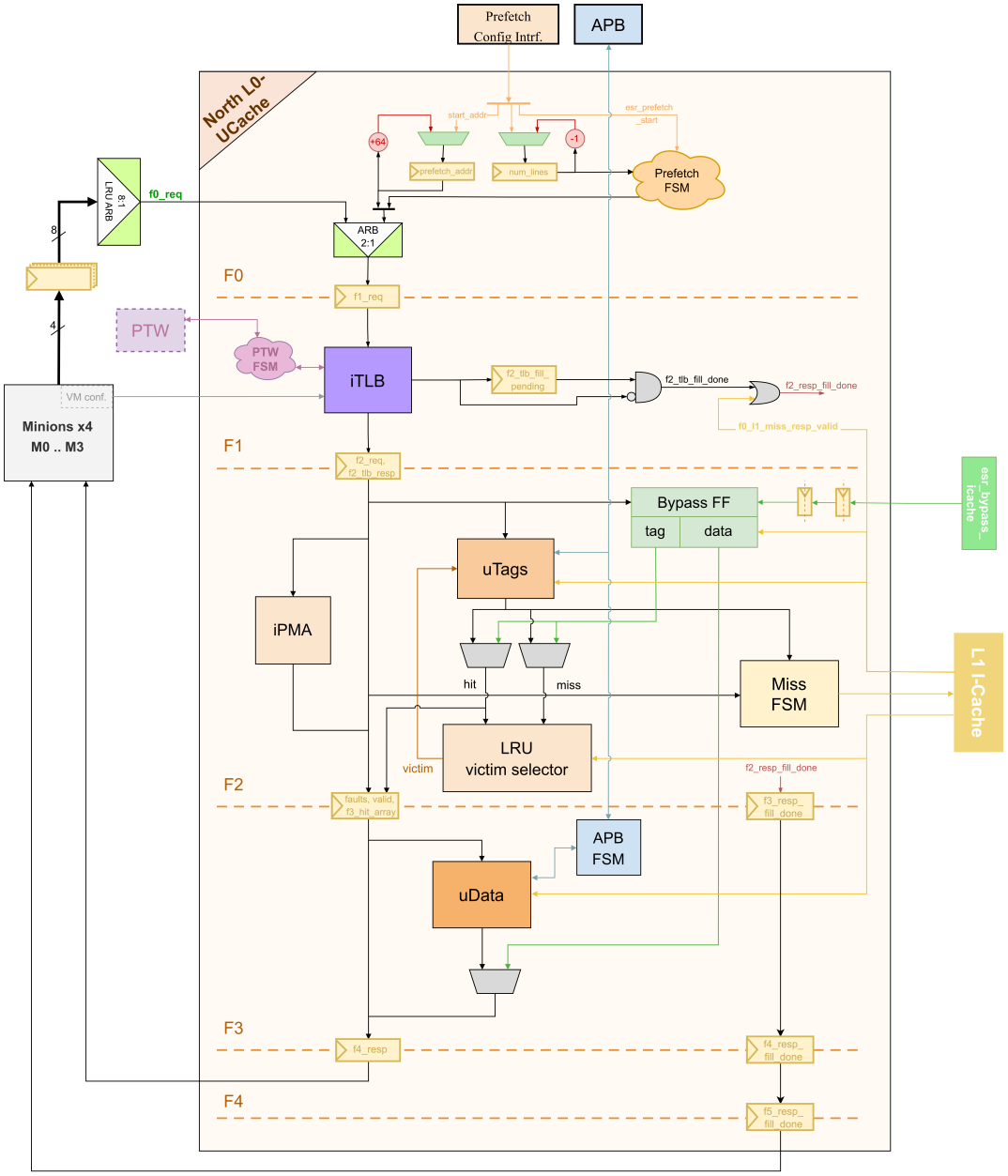



*Figure 31. L0 micro-cache detailed pipeline diagram*

### F0 Stage

The F0 stage arbitrates between the core request granted by the 8:1 LRU arbiter and a request from the code prefetcher, and always gives priority to the core request. In a cycle with no core request, the code prefetcher can enter its request, if it is in its Request state and no miss unattended by the miss handler is pending (unattended-miss state machine). A prefetch request uses the same path through the pipeline as a core request, so a prefetch flag goes with each request to the next stages to tell the two apart. The request, the identifier of its Minion, the bypass bit and the prefetch flag are registered into F1.

### F1 Stage

The F1 stage translates the virtual address of the request to a physical address in the instruction TLB. The translation uses the supervisor and machine address-translation registers that each Minion sends to the micro-cache (`satp_info` and `matp_info`), and the privilege level of the fetch selects one of the two (Section 3.4.5.2). The TLB returns the physical page number, a miss indication, and page and access faults. A TLB miss starts a page-table walk if no walk is in progress and the address is valid, and the walk request goes to the page-table walker one cycle

later (Section 3.4.5.3). The request continues to F4 and receives a miss response, and the thread retries after fill done.

**F2 Stage**

The F2 stage forms the physical address from the page number and the page offset. The tag is compared with all 16 lines of the micro-tag array, and with the line that a fill writes in the same cycle, while the PMA unit checks the address. The LRU state is updated on a hit, on an accepted miss, and when the L1 response writes the line into the victim entry. A line miss with no TLB miss and no fault is accepted if the miss handler is Ready: the handler records the physical address, the LRU victim and the bypass and prefetch flags, and the victim's valid bit is cleared. A miss that finds the handler busy is not recorded and receives only the miss response in F4. In bypass mode the tag is compared with the bypass register instead of the micro-tag array, and a prefetch request is forced to miss.

**F3 Stage**

The F3 stage reads the micro-data array with the one-hot hit vector that the micro-tag array generates in F2, and selects the 32-byte half-line named by address bit 5; in bypass mode the half comes from the bypass register. The response is a miss when the tag misses or the TLB misses. The ECC and bus-error flags stored with the line are valid only on a hit. If F2 accepted a miss, the miss handler is in its Request state in this cycle and drives the line request to the L1 instruction cache.

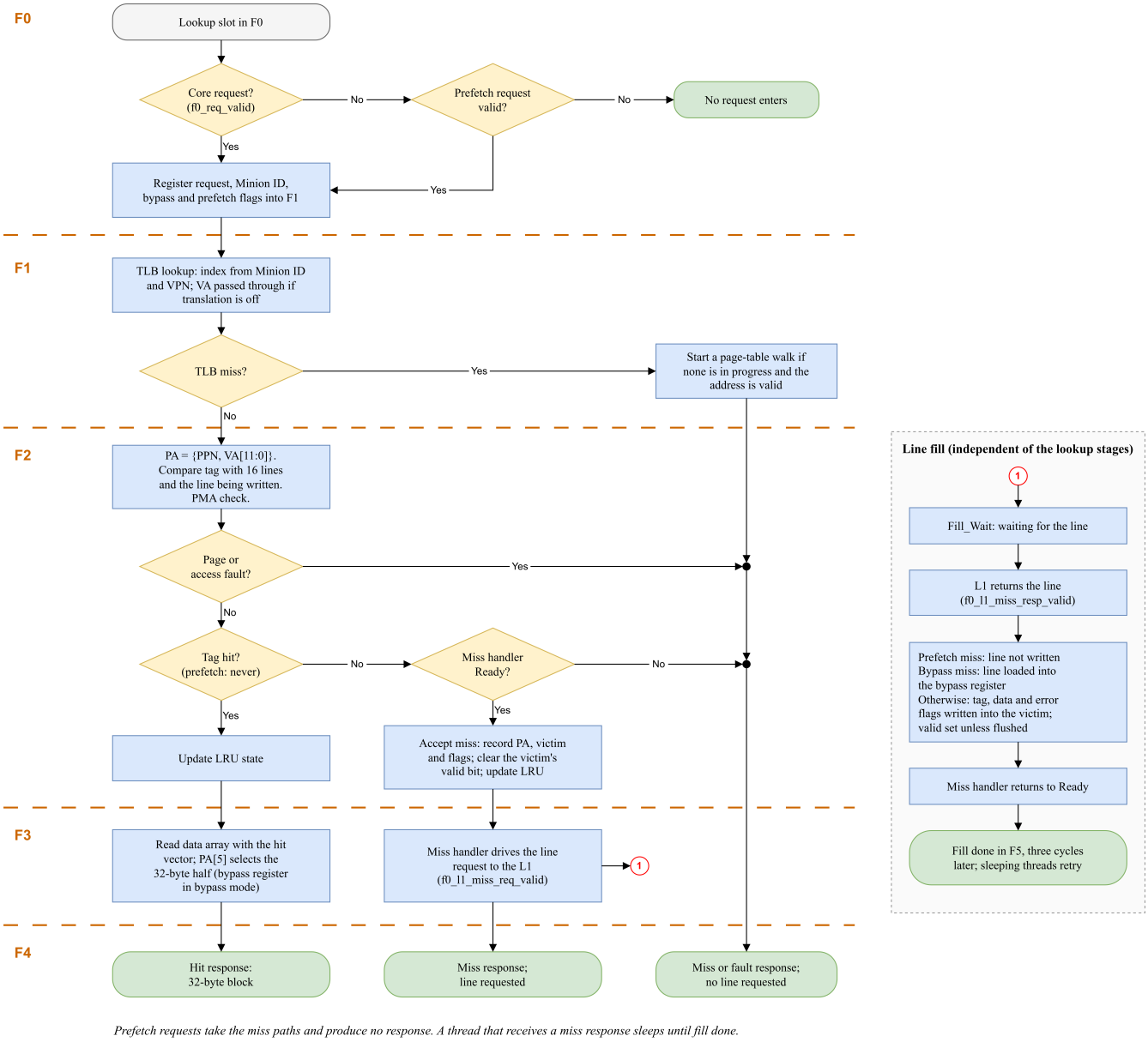
**F4 Stage**

The F4 stage registers the response and drives it to the four Minions, four cycles after the grant, for a hit and for a miss alike. A prefetch request produces no response.

**F5 Stage**

The F5 stage drives fill done, which tells the sleeping threads that a line or translation fill is complete. Fill done is generated in F2 and passes through F3 and F4 to F5. F5 is not part of a lookup.

Figure 32 shows the path of one request from F0 to F4 and the fill that a miss starts. Every request leaves the pipeline at F4 with a hit, miss or fault response. The fill path on the right runs independently of the lookup and ends with fill done in F5. The time to fill a line depends on the L1 instruction cache and is given in Section 3.5.




*Prefetch requests take the miss paths and produce no response. A thread that receives a miss response sleeps until fill done.*

*Figure 32. L0 micro-cache pipeline flow*

The transaction figures of the L0 micro-cache draw one transaction on the functional block diagram (Figure 29):

- **Path:** the dashed green line is the path of the transaction.
- **Steps:** a numbered green circle marks each step, in order.
- **Unused blocks:** a red circle or a dashed red line marks a block that the transaction does not use, or whose result it ignores.
- **Cycles:** two steps with different numbers can occur in the same cycle. The table under each figure gives the cycle of each step: a request counts its cycles from the grant, with the stage in brackets, and a fill counts them from the cycle *N* of its response.

Figure 33 shows a core request that hits in the instruction TLB and in the micro-tag array, the common case, and Table 42 lists its steps.

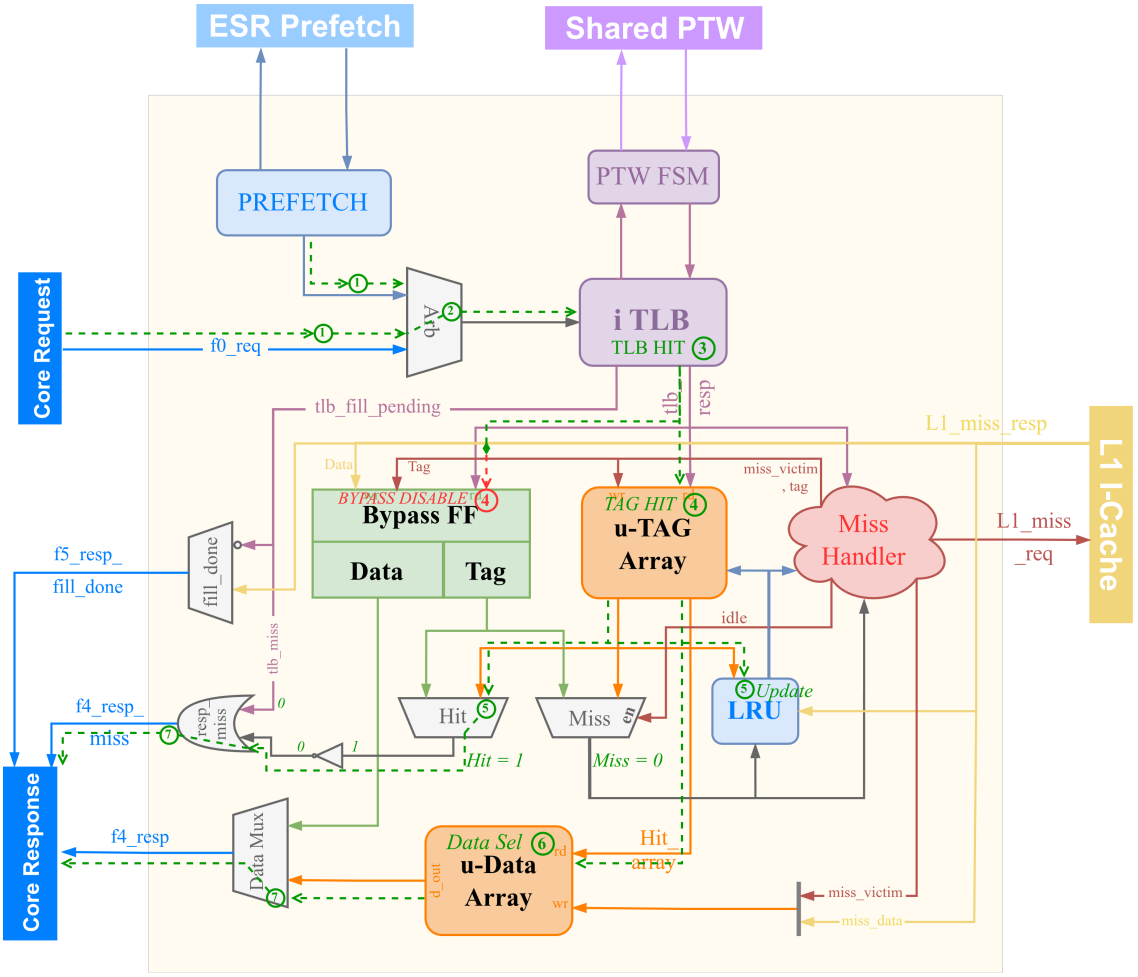



*Figure 33. Core request hit*

*Table 42. Core request hit, steps by cycle*

| Cycle | Step | Block | Action |
|---|---|---|---|
| 0 (F0) | 1 | Core request, code prefetcher | A core request and, if one is qualified, a prefetch request reach the arbiter. |
| 0 (F0) | 2 | Arbiter | Grants the core request, which always has priority. |
| 1 (F1) | 3 | Instruction TLB | Hits and returns the physical page number. |
| 2 (F2) | 4 | Micro-tag array | The physical tag hits one line. The bypass register is not used outside bypass mode. |
| 2 (F2) | 5 | Hit and miss logic, LRU | Hit is 1 and miss is 0. The LRU state records the line that hits. |
| 3 (F3) | 6 | Micro-data array | Reads the line with the one-hot hit vector and selects the 32-byte half. |
| 4 (F4) | 7 | Response | Drives the instruction block to the Minions, with the miss bit clear. |

### 3.4.5. Instruction TLB

The instruction TLB translates the virtual address of a request, from a core or from the code prefetcher, to a physical address in F1. It is `icache_tlb_array`, a wrapper around `tlb_top`. The data

cache of each Minion has its own instance of the same `tlb_top` (`dcache_tlb_array`), so the two caches share the design of the TLB but not its translations. Each wrapper only adapts `tlb_top` to its cache. The instruction wrapper sets 16 entries for four Minions, marks every request as an instruction fetch, and registers the address-translation registers of the four Minions (`satp_info` and `matp_info`) and `esr_vmspagesize`; `tlb_top` registers `esr_shire_coop_mode`. A change to any of these settings takes effect one cycle later.

A TLB miss needs a page-table walk, and the TLB has at most one walk outstanding (Section 3.4.5.3). The TLB does not block during a walk: it continues to look up every request, and a request that hits receives its translation. A request that misses during a walk starts no walk and receives a miss response with no data, and its thread retries after fill done.

Table 43 lists the interface of `icache_tlb_array`, grouped by the block at the other end. The request and the response connect to F1 of the micro-cache pipeline: the TLB answers in the same cycle, and the micro-cache registers the response into F2. The configuration inputs, the invalidation pulses and the walker ports pass through to ports of the micro-cache (Table 33).

*Table 43. Instruction TLB interface signals*

| Signal | Dir | Width | Description |
|---|---|---|---|
| **Request from the pipeline, F1** | | | |
| `req_valid` | in | 1 | A request is in F1, from a core or from the code prefetcher. |
| `req_min_id` | in | 2 | Minion of the request. Selects its address-translation registers and, in per-Minion mode, its four TLB entries. A prefetch request uses Minion 0. |
| `req_data.status` | in | `minion_vm_status` | Privilege level, `mprv`, `mpp`, `sum`, `mxr` and debug mode of the requesting thread. |
| `req_data.vpn` | in | 36 | Virtual page number, VA[47:12]. |
| `req_data.msb_err` | in | 1 | VA[48] differs from VA[47]. Used for the canonical-address check. |
| `req_data.passthrough` | in | 1 | Disables translation. Tied low in the micro-cache. |
| **Response to the pipeline, F1** | | | |
| `resp_data.ppn` | out | 28 | Physical page number. VA[39:12] when translation is disabled. |
| `resp_data.miss` | out | 1 | TLB miss. |
| `resp_data.xcpt_if` | out | 1 | Page fault (Table 45). |
| `resp_data.access_fault` | out | 1 | Access fault from the walk that filled the entry. |
| `resp_data.fill_pending` | out | 1 | A walk is in progress. Its fall marks the end of a translation fill. |
| **Configuration and invalidation** | | | |

| Signal | Dir | Width | Description |
|---|---|---|---|
| satp_info | in | 4 × minion_satp_info | Supervisor address-translation register of each of the four Minions (Table 35). Registered in the wrapper. |
| matp_info | in | 4 × minion_satp_info | Machine address-translation register of each of the four Minions, used for fetches in M-mode. Registered in the wrapper. |
| esr_vmspagesize | in | tlb_entry_type | Page size of every entry: 4 KB, 2 MB or 1 GB. Registered in the wrapper. |
| esr_shire_coop_mode | in | 1 | Cooperative mode: the four Minions share all 16 entries. Registered in tlb_top. |
| tlb_invalidate | in | 4 | One pulse per Minion. Clears the entries of that Minion, or all 16 entries in cooperative mode (Section 3.4.5.5). |
| **Page-table walker** | | | |
| ptw_req_valid | out | 1 | Walk request. Held until the walker accepts it. |
| ptw_req_ready | in | 1 | The walker accepts the request. |
| ptw_req_data | out | minion_ptw_req | Translation mode, root page number, privilege level and virtual page number (Table 36). |
| ptw_invalidate | out | 1 | OR of the four tlb_invalidate pulses, registered once. |
| ptw_resp_valid | in | 1 | The walk is complete. |
| ptw_resp_data | in | minion_ptw_pte | Last page-table entry of the walk, with the access-fault and cancel flags (Table 37). |

#### 3.4.5.1. Organization

The TLB has 16 entries, and all of them hold translations of one page size, 4 KB, 2 MB or 1 GB, set by esr_vmspagesize. Figure 34 shows the format of an entry and how the 16 entries are divided between the Minions. An entry holds 70 bits in a latch-based register file: the 34-bit tag, the 28-bit physical page number, the x, w, r, u, a and d bits of the page-table entry, the access-fault flag of the walk, and the execution mode of the walk. The valid bits are separate flip-flops. The tag field is 34 bits because that is the longest tag, the one of 4 KB pages in per-Minion mode; cooperative mode and larger pages use fewer bits and store them with zero bits (Table 44).

A walk can end without a translation. The walker returns the last page-table entry that it reads, and that entry is not a valid leaf when v is clear, when r and x are both clear, or when w is set and r is clear, a reserved encoding. A walk that ends in an access fault gives no translation either. The TLB still writes the entry, with x, w and r as 0. The thread's retry then hits that entry and takes a page fault, and also an access fault if the walk ended in one, instead of starting the same walk again.

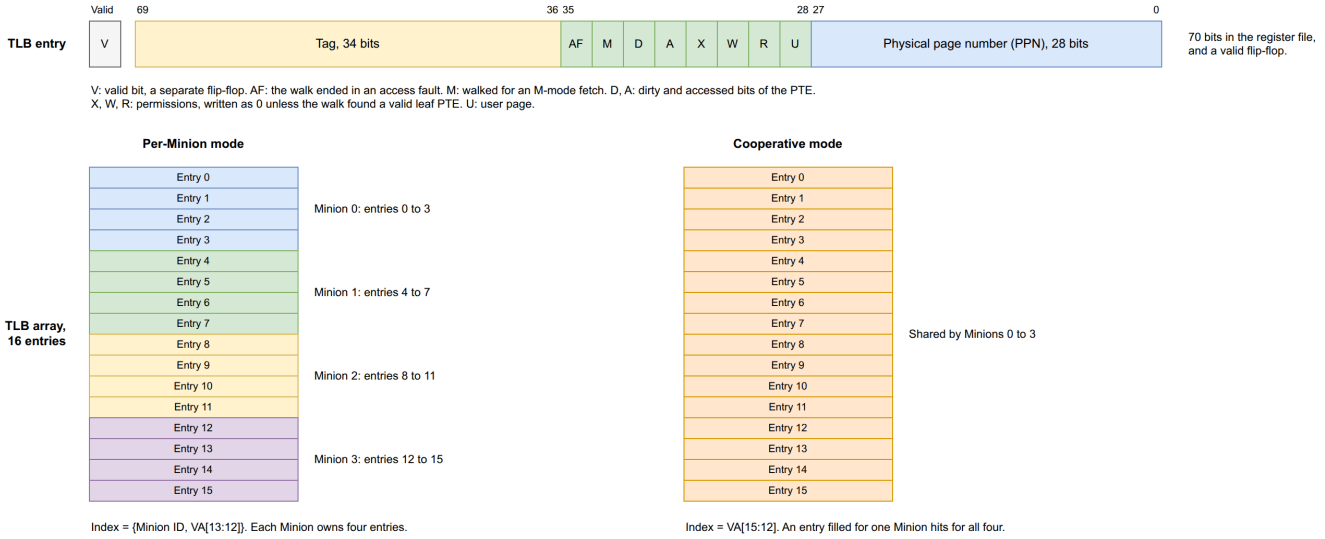



*Figure 34. Instruction TLB organization*

The TLB is direct-mapped in both of its modes: the index selects one entry, and a translation is either in that entry or not in the TLB. The mode, set by `esr_shire_coop_mode`, decides where the index comes from:

- **Per-Minion mode:** the micro-cache serves four Minions, so the 16 entries are divided into four groups of four, one group for each Minion. The upper two index bits are the Minion identifier, and the lower two come from the virtual page number. A Minion replaces only its own entries, so a Minion that touches many pages cannot evict the translations of the other three.
- **Cooperative mode:** all 16 entries are shared, and all four index bits come from the virtual page number. An entry carries no Minion identifier, so an entry filled for one Minion hits for the other three. Cooperative mode therefore assumes that the four Minions use the same translation, and in return the group has 16 entries instead of four for each Minion.

Figure 35 shows how the index and the tag are taken from the virtual page number for 4 KB pages, and Table 44 lists them for all page sizes. Each stored tag is written most-significant field first. Cooperative mode adds two zero bits at the top, because its index takes two more address bits; a 2 MB or 1 GB page adds 9 or 18 zero bits at the bottom, one page-table level of 9 bits for each step above 4 KB.

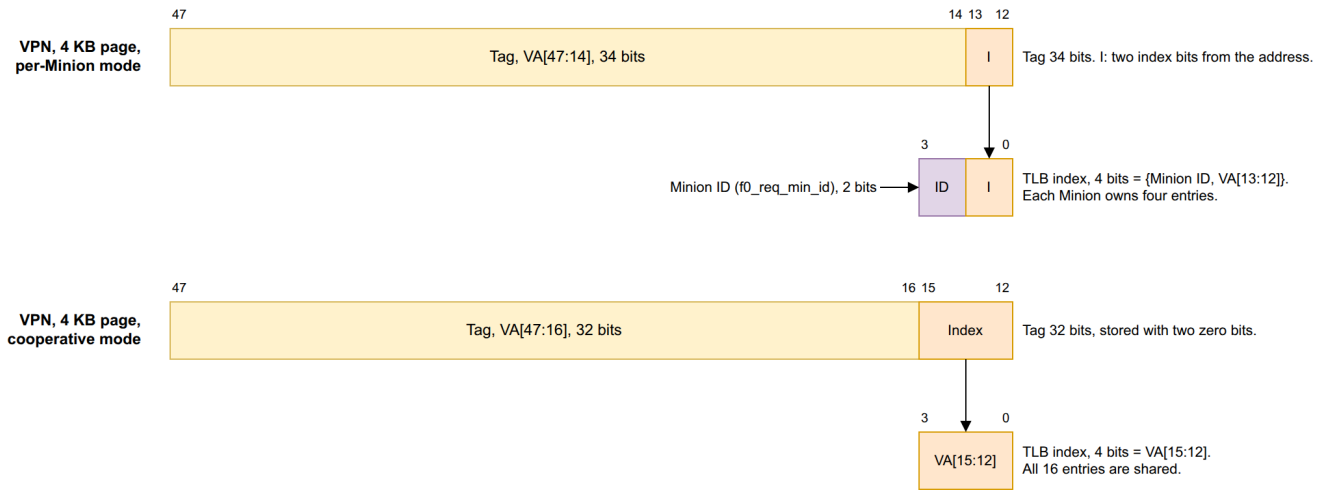



*Figure 35. Instruction TLB index and tag for 4 KB pages*

*Table 44. Instruction TLB index and tag*

| Page size | Mode | Index (4 bits) | Stored tag, 34 bits (significant bits) |
|---|---|---|---|
| 4 KB | Per-Minion | Minion identifier, VA[13:12] | {VA[47:14]} (34) |
| 4 KB | Cooperative | VA[15:12] | {2 zeros, VA[47:16]} (32) |
| 2 MB | Per-Minion | Minion identifier, VA[22:21] | {VA[47:23], 9 zeros} (25) |
| 2 MB | Cooperative | VA[24:21] | {2 zeros, VA[47:25], 9 zeros} (23) |
| 1 GB | Per-Minion | Minion identifier, VA[31:30] | {VA[47:32], 18 zeros} (16) |
| 1 GB | Cooperative | VA[33:30] | {2 zeros, VA[47:34], 18 zeros} (14) |

A change of `esr_shire_coop_mode` does not invalidate the entries. An entry filled in one mode stays valid in the other mode and can give a false hit, so the TLB must be invalidated after a change of mode (Section 3.4.5.5).

#### 3.4.5.2. Lookup and Permission Check

Figure 36 shows the lookup in F1 as a sequence of decisions from top to bottom. The canonical-address check comes first because it depends only on the mode of the selected register, not on whether translation is enabled. The privilege level of the fetch, `vm_status.prv`, selects the translation register: `matp_info` of the requesting Minion in M-mode, `satp_info` otherwise. For an instruction fetch `mprv` is ignored. Translation is enabled when the selected register is not in Bare mode and the thread is not in debug mode. When translation is disabled, the TLB returns VA[39:12] as the physical page number and never misses.

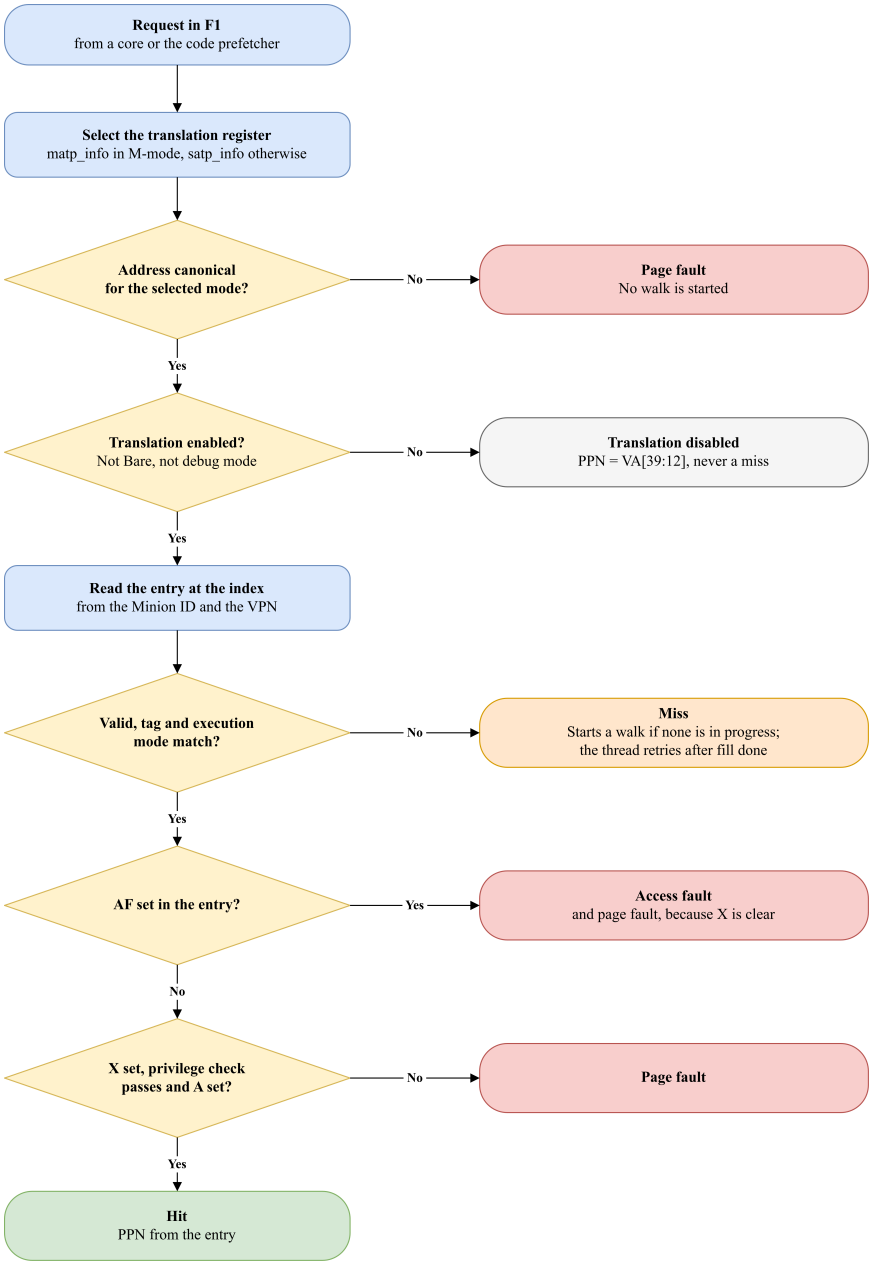



*Figure 36. Instruction TLB lookup and permission check*

An entry hits when it is valid, its tag matches, and its execution mode matches the fetch: an entry walked for M-mode hits only for M-mode fetches, and an entry walked for S-mode or U-mode only for those. A miss goes to the walk state machine (Section 3.4.5.3).

The TLB reports two faults, listed in Table 45. The page fault comes from three checks:

- **Privilege check:** in U-mode the entry must have `u` set; in S-mode and M-mode it must have `u` clear, unless `mstatus.SUM` is set.
- **Accessed bit:** the TLB does not set `a`. It raises a page fault instead, and software updates the page-table entry.
- **Canonical address:** in Sv48, VA[48] must equal VA[47]; in Sv39, VA[47:38] must also all be equal. A non-canonical address raises the page fault whether or not the entry hits, and starts no walk.

*Table 45. Instruction TLB faults*

| Signal | Condition |
|---|---|
| `xcpt_if`, instruction page fault | The entry hits and `x` is clear, the privilege check fails, or `a` is clear; or the address is not canonical. `x` is also clear when the walk finds no valid leaf entry (Section 3.4.5.1). |
| `access_fault`, instruction access fault | The entry hits and its walk ended in an access fault. |

A walk that ends in an access fault leaves `x` clear and AF set, so the retry reports both faults. The integer pipeline gives the access fault priority, and the thread takes an instruction access fault.

#### 3.4.5.3. Page-Table Walk

A page-table walk reads the page tables in memory, from the root table of the selected translation register down to the leaf entry that holds the translation. The TLB does not read the page tables itself: it sends the walk to the shared page-table walker (Section 3.6).

For the walk the TLB uses a three-state machine, shown in Figure 37:

- **Ready:** no walk is in progress. A TLB miss on a canonical address captures the Minion, the virtual page number and the walk request, and moves the state machine to Request. The request carries the translation mode and root page number of the selected register, the privilege level and the virtual page number (Table 36).
- **Request:** sends the walk request to the walker. `ptw_req_valid` is held until the walker accepts the request with `ptw_req_ready`, and the state machine then moves to Wait. An invalidation withdraws the request and returns the state machine to Ready (Section 3.4.5.5).
- **Wait:** waits for the walker's response. The response is registered for one cycle, and in the next cycle the entry is written at the index of the request's Minion and virtual page number, unless the walk is cancelled. The state machine returns to Ready in the same cycle.

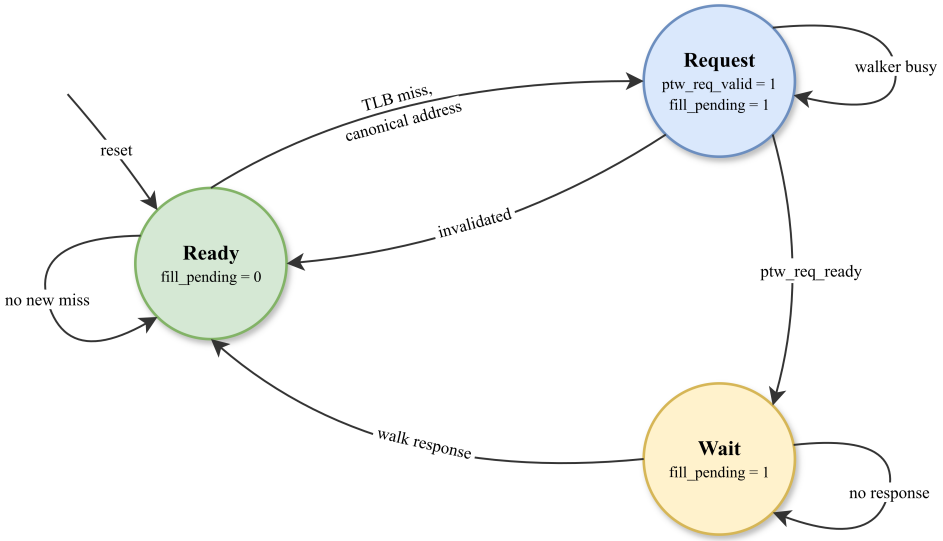



*Figure 37. Instruction TLB walk state machine*

A walk starts only from Ready, so one walk at most is in progress. A miss that arrives in Request or Wait receives a miss response and starts no walk. `fill_pending` is set in Request and Wait. The micro-cache detects the end of the walk as the fall of `fill_pending` and turns it into

`f2_tlb_fill_done`, which reaches the cores as `f5_resp_fill_done` three cycles later.

#### 3.4.5.4. TLB Miss and Translation Fill

A request that misses in the TLB receives a miss response and starts a page-table walk if none is in progress. It never takes the miss handler of the micro-cache, because it has no physical address to send to the L1 instruction cache. The thread sleeps until the translation fill and then retries. Figure 38 shows a core request that misses in the TLB, and Table 46 lists its steps.

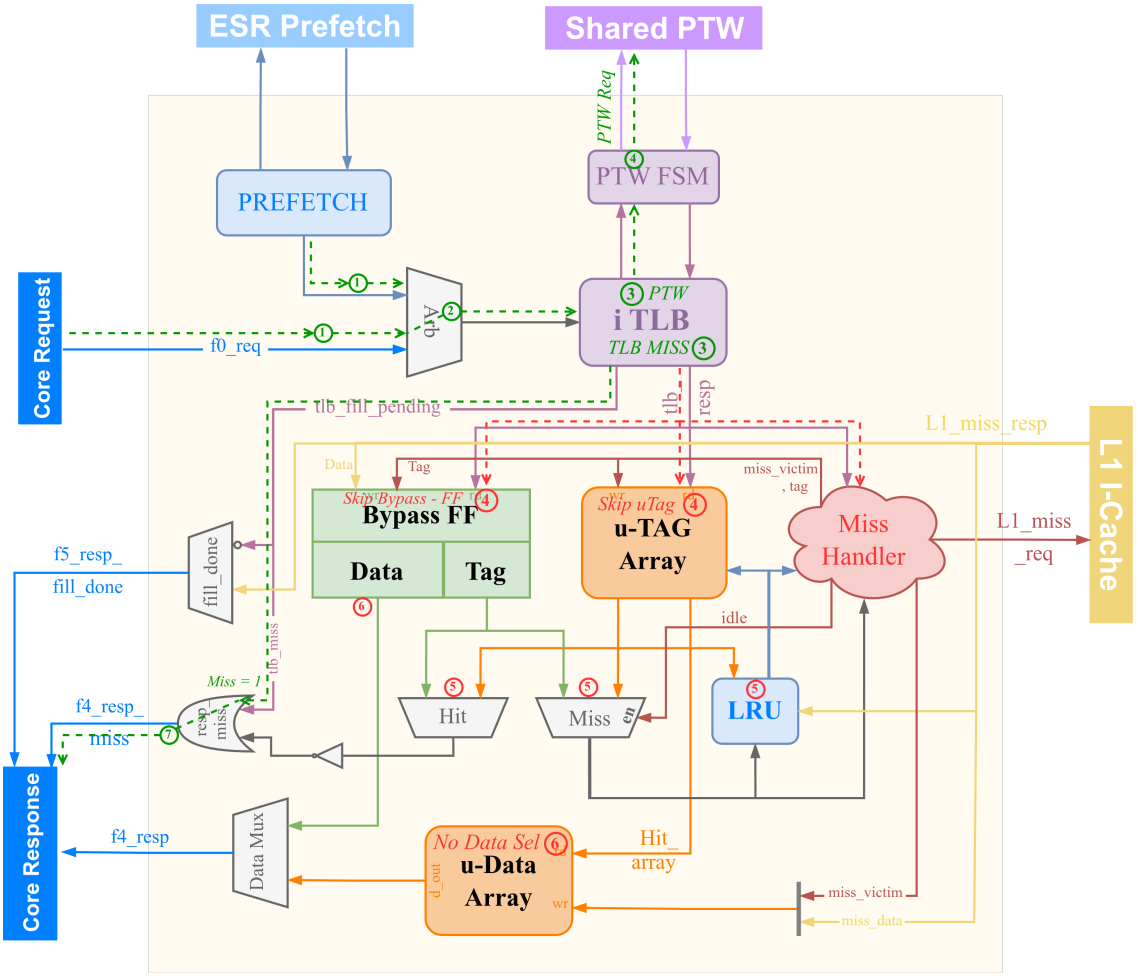



*Figure 38. Core request TLB miss*

*Table 46. Core request TLB miss, steps by cycle*

| Cycle | Step | Block | Action |
|---|---|---|---|
| 0 (F0) | 1–2 | Arbiter | Grants the core request, which always has priority. |
| 1 (F1) | 3 | Instruction TLB | Misses. The walk state machine is in Ready, so it captures the request and moves to Request. |
| 2 (F2) | 4 | Walk state machine | Drives the walk request to the shared page-table walker. The micro-tag array and the bypass register compare an address that is not valid, and their results are ignored. |
| 2 (F2) | 5 | Hit and miss logic, LRU | No miss is accepted, because the TLB missed. The LRU state does not change. |
| 3 (F3) | 6 | Micro-data array | Not read. |

| Cycle | Step | Block | Action |
|---|---|---|---|
| 4 (F4) | 7 | Response | The TLB-miss bit forces a miss response. The thread sleeps until fill done. |

Figure 39 shows a fetch by M0, TH0 that misses in the TLB. The miss is seen in F1 in cycle 1, the walk request is driven in cycle 2, and the miss response leaves F4 in cycle 4. The break stands for the walk. The walker responds in cycle 8, the entry is written in cycle 9, and fill done is driven in cycle 13. The thread then retries, hits in the TLB, and proceeds to the line lookup.

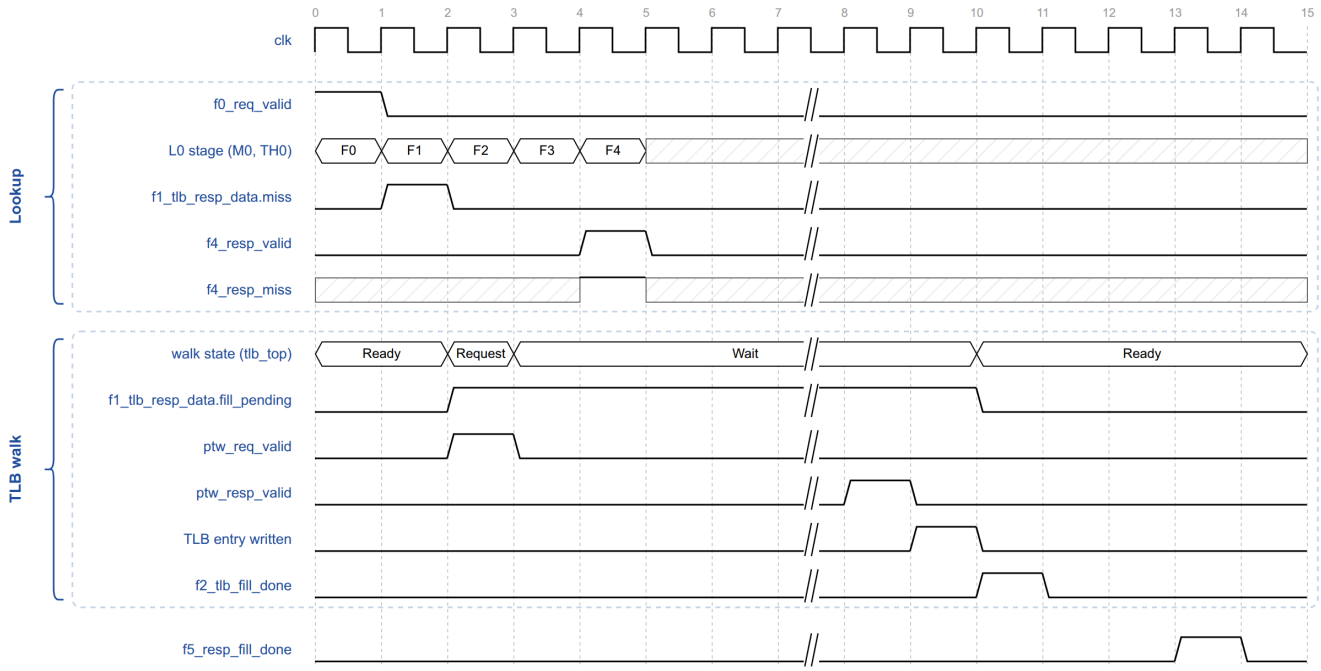



*Figure 39. TLB miss, page-table walk and fill done*

Figure 40 shows the end of a page-table walk, and Table 47 lists its steps. Cycle *N* is the cycle in which the walker response arrives. Fill done reaches the cores five cycles later: the response is registered, the entry is written, the fall of `fill_pending` is detected (Figure 46), and fill done passes F3 and F4.

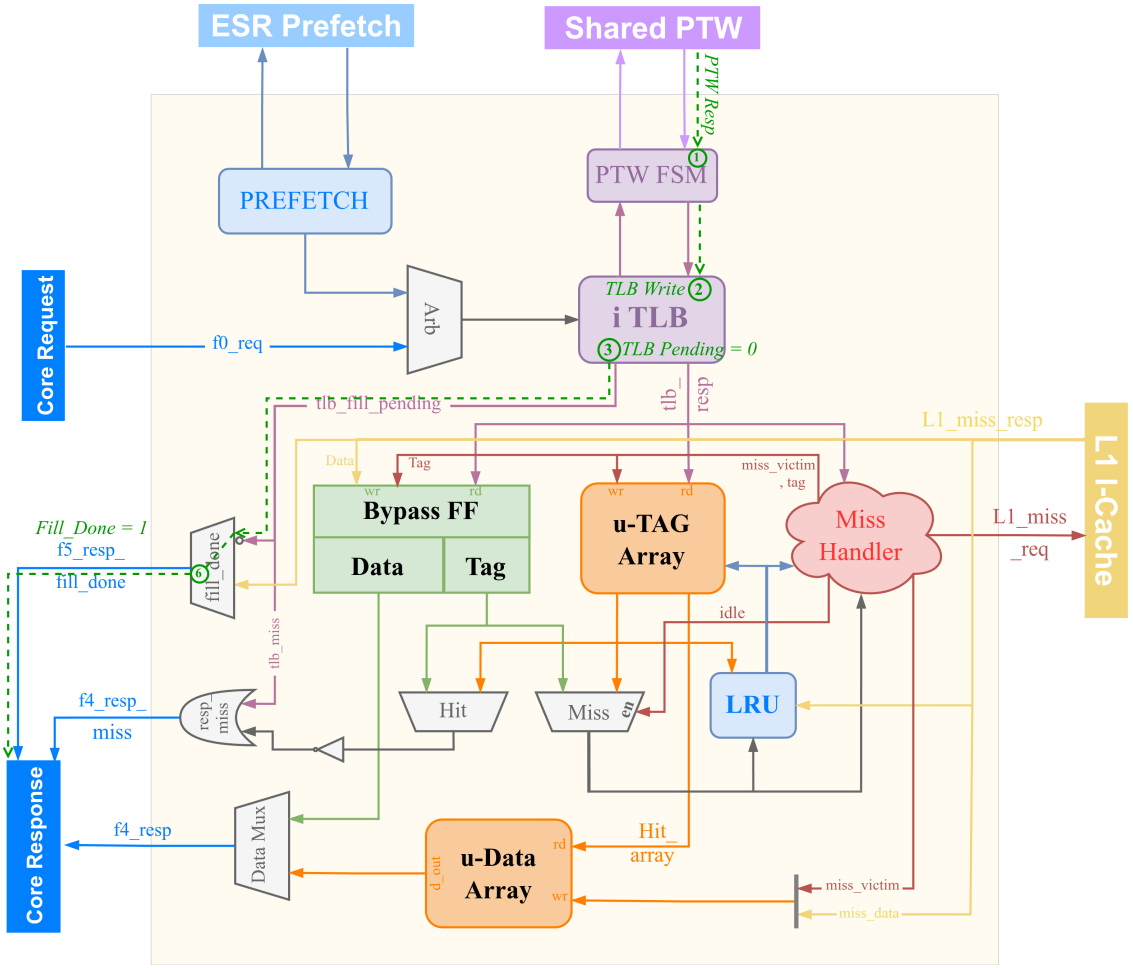



*Figure 40. Translation fill*

*Table 47. Translation fill, steps by cycle*

| Cycle | Step | Block | Action |
|---|---|---|---|
| *N* | 1 | Walk state machine | In Wait, receives the walker response and registers it. |
| *N* + 1 | 2 | Instruction TLB | Writes the entry at the index of the request, unless the walk is cancelled. The walk state machine returns to Ready. |
| *N* + 2 | 3 | Instruction TLB | `fill_pending` falls. The micro-cache detects the fall and generates fill done. |
| *N* + 5 | 6 | Fill done | `f5_resp_fill_done` wakes the sleeping threads. |

### 3.4.5.5. Invalidation

`tlb_invalidate` carries one pulse per Minion, driven when the Minion writes 1 to bit 1 of its `cacheInvalidate` CSR (0x7D0). Outside cooperative mode a pulse clears the four entries of that Minion; in cooperative mode any pulse clears all 16. A walk in progress for the invalidated Minion, or any walk in cooperative mode, is cancelled: in Request it is withdrawn, and in Wait its response is discarded. The invalidation is held until the outstanding walk ends, so a response from before the invalidation can never be written. The OR of the four pulses, registered once, is forwarded to the walker on `ptw_invalidate`.

The entries carry no address-space identifier, and a change of `satp_info` or `matp_info` does not invalidate them. Software must invalidate the TLB through `cacheInvalidate` after it changes the translation.

#### 3.4.5.6. Page Size

`esr_vmspagesize` selects 4 KB, 2 MB or 1 GB for every entry, and moves the index and the tag accordingly (Table 44). A change of `esr_vmspagesize` does not invalidate the entries either. The walker ends the walk at the page-table level of that size. A leaf found at a higher level is split into pages of the configured size, with the walker supplying the lower page-number bits from the virtual address. For a configured size of 2 MB or 1 GB the walker returns the page-number bits below that size as zero, and the micro-cache forms the physical address from the returned page number and VA[11:0] only.

### 3.4.6. Micro-Tag and Micro-Data Arrays

The micro-tag array and the micro-data array hold the 16 lines of the micro-cache, and the LRU selector chooses the line that a new miss replaces. The micro-tag array works in F2: it holds the physical tag of each line, and compares the tag of the request, translated by the instruction TLB in F1, with all 16 tags at once. The micro-data array works in F3: it holds the lines, and returns the 32-byte half that the request needs. The two arrays have the same 16 entries, so the entry that holds a tag also holds its line, and one one-hot vector selects both. Figure 41 shows the format of an entry of each array, and how a lookup passes from one array to the other.

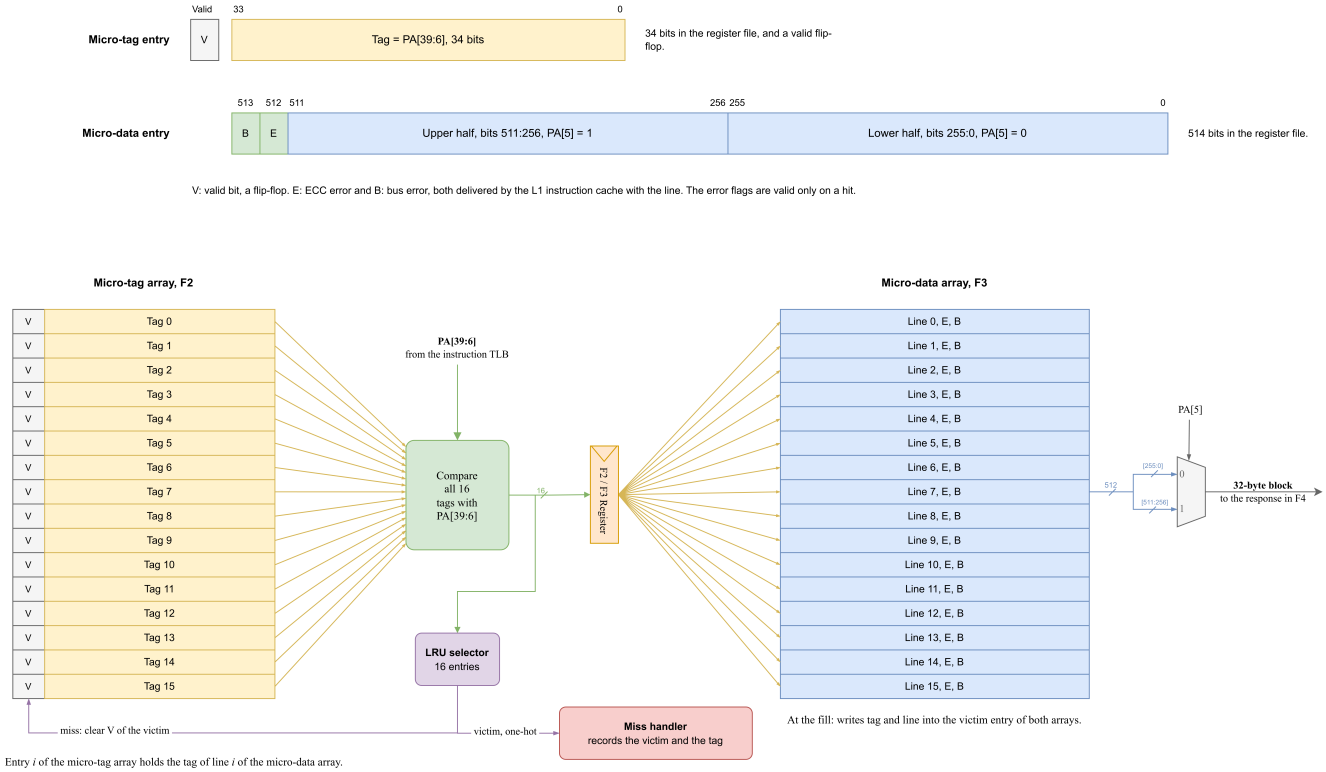



*Figure 41. Micro-tag and micro-data arrays*

#### 3.4.6.1. Micro-Tag Array

The micro-tag array holds 16 tags, one for each line. It has no index: the cache is fully associative, so a line can be in any entry. Each tag is the line address, PA[39:6], 34 bits, stored in a latch-based

register file that reads all 16 entries at once (`rf_latch_1r_1w_par`). Each entry also has a valid bit in a flip-flop beside the register file. Table 48 lists the ports of the array.

*Table 48. Micro-tag array ports*

| Port | Stage | Function |
|---|---|---|
| Read | F2 | Receives PA[39:6] of the request and returns the one-hot hit vector. |
| Victim | F2 | Receives the LRU victim of an accepted miss and clears its valid bit. |
| Write | Fill cycle | Writes the tag that the miss handler recorded into the victim entry, and sets its valid bit. APB debug writes share the port, and the fill has priority. |
| Flush | F0 | `f0_flush_data` clears all valid bits (Section 3.4.9). |
| APB | – | Reads and writes the tag and the valid bit of one entry (Section 3.4.12). |

**Lookup.** In F2 the array compares the tag of the request with all 16 tags in parallel. An entry hits when it is valid, its tag equals PA[39:6], it receives no write in this cycle, and no flush is in progress. A line is allocated only after a miss, so at most one entry hits, and the result is a one-hot hit vector of 16 bits. The vector goes to two places in the same cycle: to the LRU selector, which records the entry as the most recent, and through the stage register to the micro-data array, which reads the line with it in F3.

**Lookup in the fill cycle.** While a fill writes a tag, the array also compares the request with the tag that is written. The entry under write cannot hit on its old tag, but a request whose tag matches the new tag hits on that entry. A request that arrives in the fill cycle therefore sees the new line and does not miss on it (Figure 42).

**Miss.** When no entry hits and the miss handler accepts the miss, the array frees the victim that the LRU selector chooses: it clears the valid bit of the victim in F2, so the entry cannot hit while its new line is outstanding. The miss handler records the victim and the tag, and requests the line from the L1 instruction cache in F3 (Section 3.4.8.1). A bypassed or prefetch miss frees no victim, because its fill writes no array.

**Fill.** When the L1 instruction cache returns the line for the outstanding miss request, the array writes the tag that the miss handler recorded into the victim entry, and sets the valid bit (Section 3.4.6.4).

The valid bits change on four events:

- **Accepted miss:** the valid bit of the victim is cleared.
- **Fill:** the valid bit of the victim is set, unless a flush arrived while the fill was outstanding.
- **Flush:** all valid bits are cleared.
- **Debug write:** the valid bit is written with the tag.

#### 3.4.6.2. Micro-Data Array

The micro-data array holds the 16 lines, entry for entry with the micro-tag array. Each entry is 514 bits in a latch-based register file with one-hot read and write selects (`rf_latch_1r_1w_dec`): the 512-bit line, the ECC-error flag (bit 512) and the bus-error flag (bit 513) that the L1 instruction cache delivers with the line. Table 49 lists the ports of the array.

*Table 49. Micro-data array ports*

| Port | Stage | Function |
|---|---|---|
| Read | F3 | Receives the one-hot hit vector from F2 and returns the selected entry. APB debug reads share the port and wait for a cycle with no request in F3. |
| Write | Fill cycle | Writes the line and its error flags into the victim entry. APB debug writes share the port, and the fill has priority. |
| APB | – | Reads and writes one entry (Section 3.4.12). |

**Read.** In F3 the array reads the entry that the hit vector selects, and PA[5] selects the 32-byte half that goes to the response in F4. On a miss the hit vector is all zeros, and the response carries no instruction block. The error flags go to the response with the block, and they are valid only on a hit.

**Write.** The fill writes the line and its error flags into the victim entry, in the same cycle as the tag (Section 3.4.6.4).

#### 3.4.6.3. LRU Selector

The LRU selector (`lru_sel`) decides which line a new miss replaces. It keeps the exact order of use of all 16 entries, and the victim is always the entry that was used least recently. For each pair of entries the selector holds one bit that tells which of the two was used more recently. An entry is the victim when it is older than each of the other 15, and the selector presents the victim as a one-hot vector, in every cycle, to the micro-tag array and the miss handler.

Three events in F2 count as a use, and each makes the entry the most recent one:

- **Hit:** the entry that hits.
- **Accepted miss:** the victim, when the miss handler takes the miss.
- **Line fill:** the victim again, when its line is written. While the line was outstanding, other entries continued to hit and became more recent, so without this second update the new line could be the next victim as soon as it arrives.

The selector is not updated for a prefetch request or a bypassed request, because neither keeps a line in the arrays. The selector also ignores the valid bits, so it does not look for an empty entry first. Empty entries exist only after reset or a flush, and after reset the selector hands the entries out in order, entry 0 first.

#### 3.4.6.4. Fill Write

A line from the L1 instruction cache is written into the tag and data arrays in the cycle `f0_l1_miss_resp_valid` is asserted, at the victim recorded when the miss was accepted. The latch-

based register files need their write enable one cycle ahead of the write, and `f0_l1_miss_resp_early_valid` supplies it; the L1 derives it from the next-cycle value of its response valid. Prefetch and bypassed fills write neither array.

Figure 42 shows the write in cycle 3. All the requests in the figure are for the same line, line A: the outstanding miss in `f0_miss_addr`, which the L1 instruction cache answers in cycle 3, and the lookups of M1, TH0 and M2, TH0. M1, TH0 reaches F2 in cycle 2, before the write, and misses; the miss handler is still in Fill_Wait, so nothing is recorded for it. M2, TH0 reaches F2 in cycle 3, the write cycle, and hits on the line being written. Their responses leave F4 in cycles 4 and 5, the handler returns to Ready in cycle 4, and fill done is driven in cycle 6, three cycles after the write.

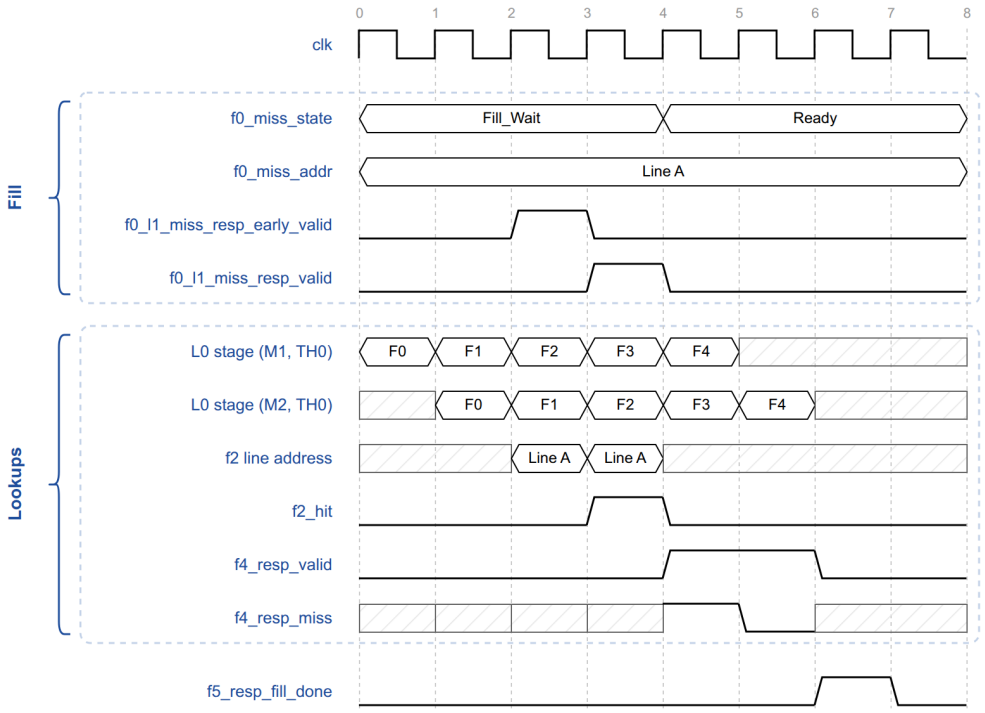



*Figure 42. Line fill and lookup in the fill cycle*

## 3.4.7. PMA Check

The PMA unit (`icache_pma_unit`) checks the physical address in F2 against the regions from which instructions may be fetched, and reports an access fault and whether the address is cacheable. Table 50 lists the regions, decoded from the address-map defines in `shire_defines.vh`. The privilege used is the privilege level of the fetch, `vm_status.prv`; for a prefetch request it is the privilege of the prefetch operation.

*Table 50. Instruction-fetch regions checked by the PMA unit*

| Region | Physical address | Fetch permitted | Cacheable |
|---|---|---|---|
| BootROM | 0x0_0200_8000 to 0x0_0200_9FFF, 8 KB | All privilege levels | Yes |
| SRAM | 0x0_0200_C000 to 0x0_0200_CFFF, 4 KB | All privilege levels | Yes |

| Region | Physical address | Fetch permitted | Cacheable |
|---|---|---|---|
| MRAM, M-mode box | From 0x0_4000_0000, the first $2^{mmode_size}$ 4-KB granules | M-mode only | Yes |
| MRAM, S-mode box | Above the M-mode box, up to $2^{smode_size}$ granules from the base | M-mode and S-mode | Yes |
| MRAM, other | The rest of 0x0_4000_0000 to 0x0_40FF_FFFF, 16 MB | All privilege levels | Yes |
| Any other address | | Access fault | No |

The two boxes exist only while `esr_mprot.mprot_en` is set; with it clear, the whole MRAM region is in the last MRAM row. The granule number is PA[23:12], so the box sizes are powers of two from 4 KB to 16 MB.

The access fault from the PMA unit is suppressed when the TLB misses, because the physical address is not valid then. Every region that passes the check is cacheable, so a fetch that does not fault is always reported as cacheable.

## 3.4.8. Miss Handling

### 3.4.8.1. Miss Handler

The miss handler fetches missing lines from the L1 instruction cache. It sends the line request, and it keeps the information about the miss that the fill needs when the line comes back. It serves three kinds of miss: a core request that misses in the micro-tag array, a bypassed core request that misses in the bypass register, and a prefetch request. A TLB miss never reaches the miss handler. Without a translation there is no physical address to send to the L1, so the walk state machine of the instruction TLB handles the TLB miss instead (Section 3.4.5.4).

The miss handler holds one line request at a time and uses a three-state machine, shown in Figure 43:

- **Ready:** no line request is outstanding, and the handler can take a miss. It takes one in F2, asserts `f2_miss` and moves to Request, when the request in F2 is valid, does not hit, has a translation, and has no page fault or access fault.
- **Request:** drives the line request on `f0_l1_miss_req_valid` with the physical address, and holds it until the holding register of the instance in `icache_top` takes it (`f0_l1_miss_req_ready`). The handler then moves to Fill_Wait.
- **Fill_Wait:** waits for the line. The line is written in the cycle `f0_l1_miss_resp_valid` arrives, and the handler is back in Ready in the next cycle.

A miss that arrives while the handler is in Request or Fill_Wait still receives its miss response, but nothing is recorded for it (Section 3.4.8.2).

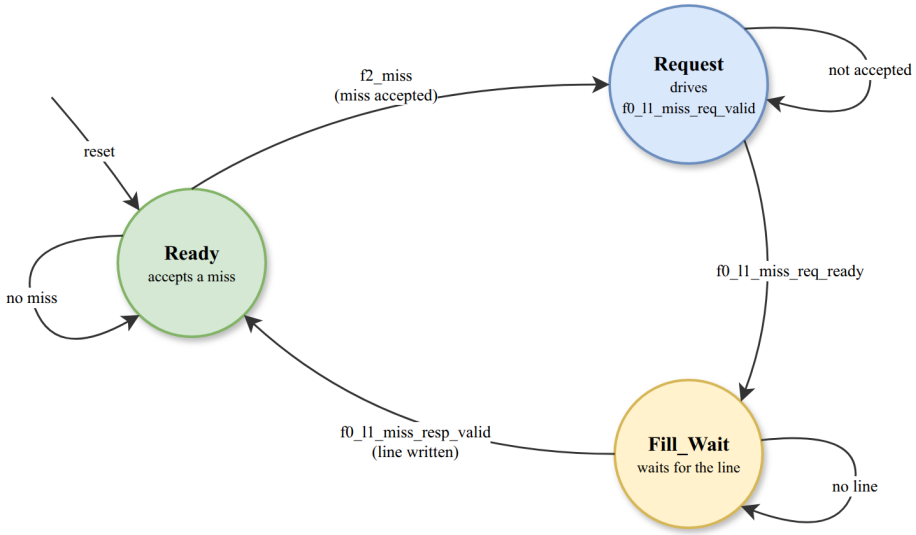



*Figure 43. Miss handler state machine*

The response from the L1 instruction cache carries the line and its two error flags, but not its address. So when the handler takes a miss, it records everything the fill needs later:

- **Tag:** PA[39:6] of the miss, in `f0_miss_addr`. It is the address of the line request, and the tag that the fill writes into the micro-tag array or into the bypass register.
- **Victim:** the one-hot entry that the LRU selector chose, in `f0_miss_victim`. The fill writes the tag and the line into this entry.
- **Flags:** bypassed, prefetch, and L0 write disabled, the last set for a bypassed or prefetch request. They decide where the line goes: into the arrays, into the bypass register, or nowhere in the micro-cache.

#### 3.4.8.2. Line Miss

A request that translates but does not hit receives a miss response. If the handler is in Ready, the miss is accepted and the line is requested; if it is busy, nothing is recorded, and the thread fetches the line after a later fill done, when it retries and finds the handler idle. A core miss that is not recorded also opens the window in which the code prefetcher holds back (unattended-miss state machine). Figure 44 shows a line miss that finds the miss handler in Ready, and Figure 45 one that finds it busy. Table 51 lists the steps of both side by side. The busy case is the path that the livelock guard of the Front End bounds (Section 3.3.8).

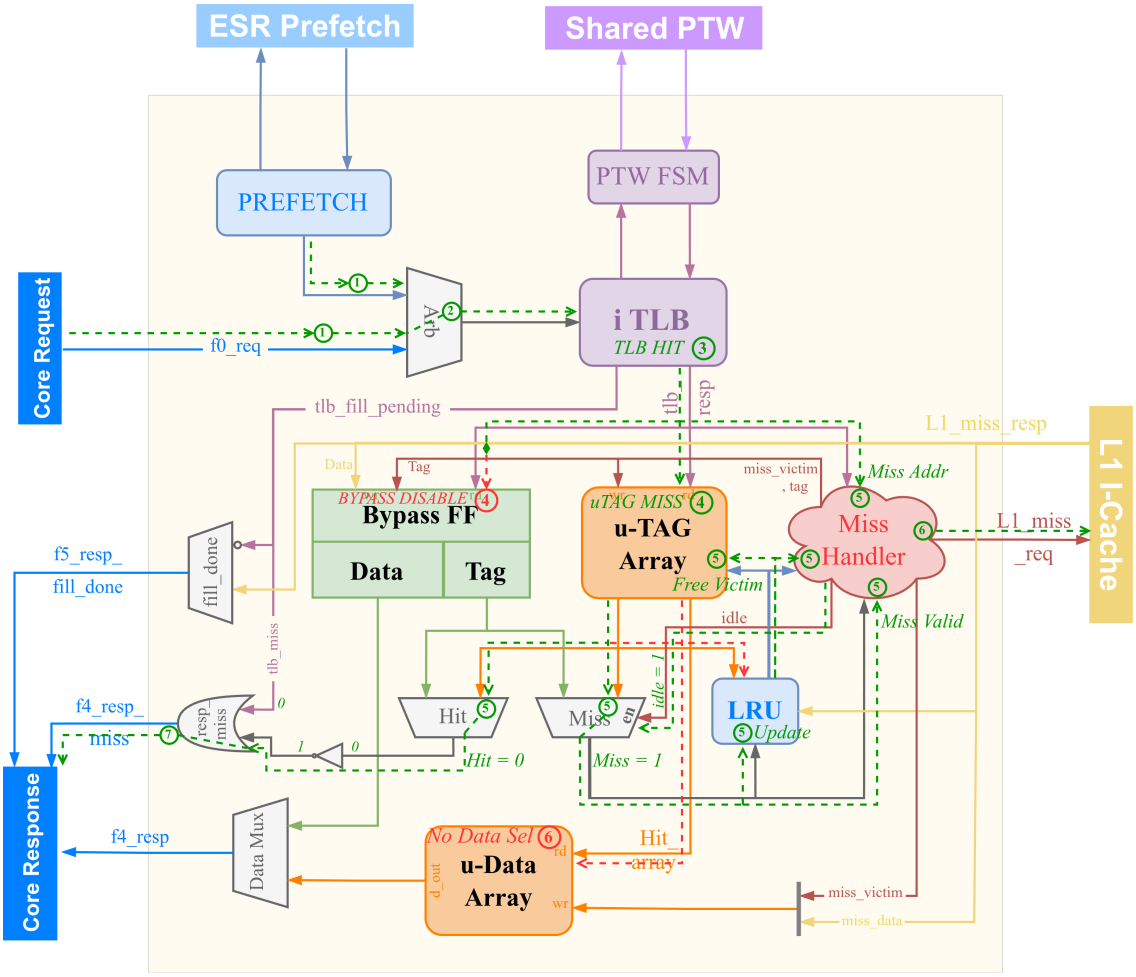



*Figure 44. Core request line miss*

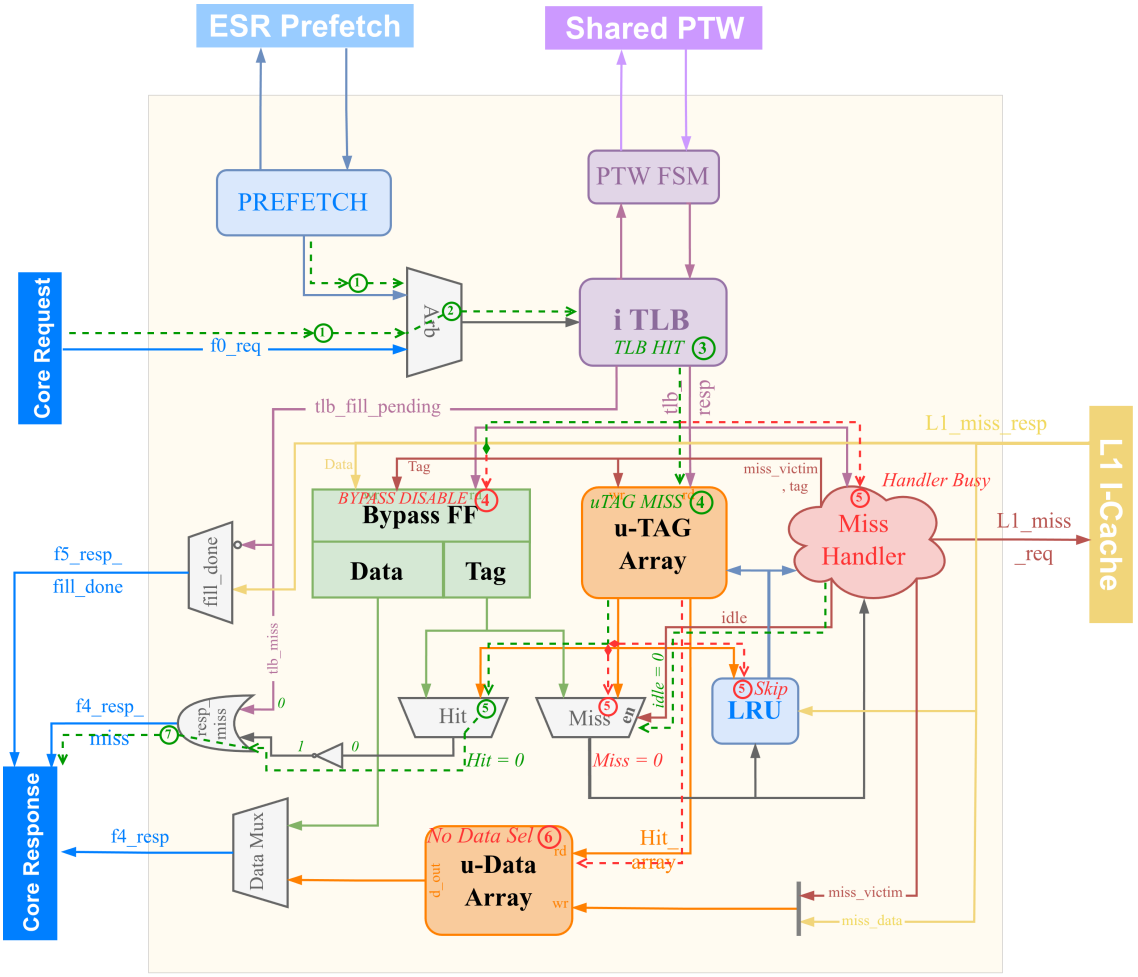



*Figure 45. Line miss with the miss handler busy*

*Table 51. Core request line miss, steps by cycle*

| Cycle | Step | Miss handler in Ready (Figure 44) | Miss handler busy (Figure 45) |
|---|---|---|---|
| 0–1 (F0–F1) | 1–3 | The arbiter grants the core request, and the instruction TLB translates its address. | |
| 2 (F2) | 4 | The physical tag hits no line in the micro-tag array. | |
| 2 (F2) | 5 | Hit is 0 and miss is 1. The miss handler records the tag, the LRU victim and the flags of the request. The valid bit of the victim is cleared, and the LRU state records the victim. | Hit is 0, and miss is 0 because the miss handler is not in Ready. Nothing is recorded, no victim is cleared, and the LRU state does not change. The unattended-miss state machine moves to FillDone_Wait and holds the code prefetcher back (unattended-miss state machine). |
| 3 (F3) | 6 | The miss handler, in Request, drives the line request to the L1 instruction cache. The micro-data array is not read. | The micro-data array is not read, and no line request is made. |
| 4 (F4) | 7 | The response is a miss. The thread sleeps until the next fill done, and then retries. | |

#### 3.4.8.3. Fill and Thread Wake-Up

Fill done tells the sleeping threads that a line or a translation has arrived, so that they retry. Figure 46 shows how the micro-cache generates it from two sources:

- **Line fill:** `f0_l1_miss_resp_valid`, in the cycle in which the L1 instruction cache returns a line.
- **Translation fill:** `f2_tlb_fill_done`, a one-cycle pulse at the end of a page-table walk. The micro-cache registers `fill_pending` from the instruction TLB into `f2_tlb_fill_was_pending`, and the pulse is high when the registered value is 1 and the current value is 0. This is the fall of `fill_pending`, one cycle after the TLB entry is written.

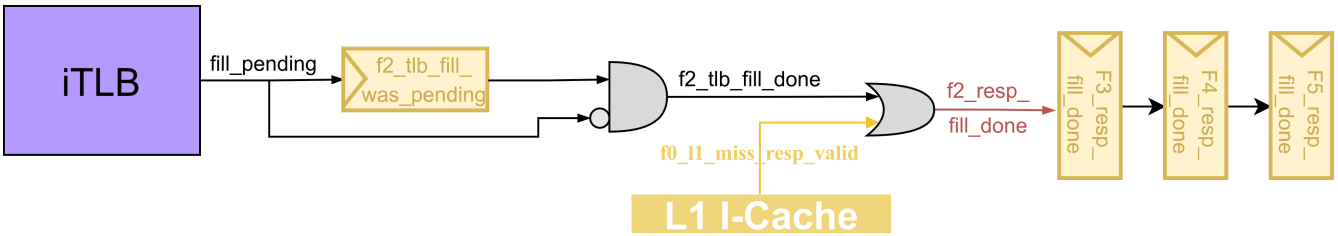



*Figure 46. Fill-done generation*

The OR of the two sources is `f2_resp_fill_done`. It passes through the F3, F4 and F5 stage registers and reaches the cores as `f5_resp_fill_done`, three cycles later. Every line fill produces fill done, including prefetch and bypassed fills, which write no array. The reason fill done follows the response by one stage is given in Section 3.3.5.

A core line fill writes the line into the arrays. Figure 47 shows the transaction, and Table 52 lists its steps; cycle *N* is the cycle in which `f0_l1_miss_resp_valid` is asserted. The fill of a bypassed miss and the fill of a prefetch miss are explained in their own sections (Section 3.4.10, Section 3.4.11), and the translation fill in Section 3.4.5.4.

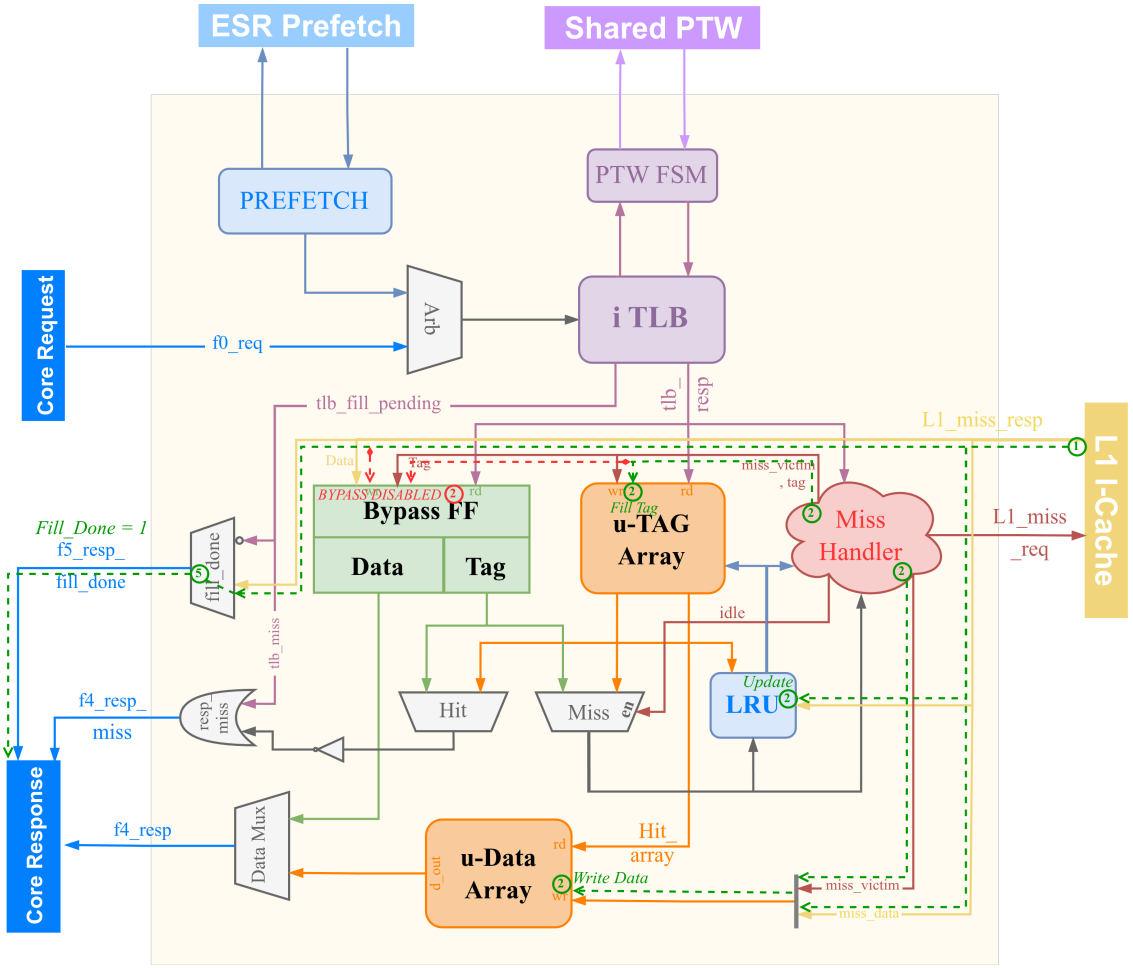



*Figure 47. Core line fill*

*Table 52. Core line fill, steps by cycle*

| Cycle | Step | Block | Action |
|---|---|---|---|
| *N* | 1 | L1 instruction cache | Asserts `f0_l1_miss_resp_valid` and returns the line and its two error flags. |
| *N* | 2 | Micro-tag array, micro-data array, LRU, miss handler | The line and its error flags are written into the victim entry of the micro-data array, and the tag recorded by the miss handler into the micro-tag array, which makes the entry valid. The LRU state records the entry. The miss handler returns to Ready, and fill done is generated. |
| *N* + 3 | 5 | Fill done | `f5_resp_fill_done` wakes the sleeping threads. |

### 3.4.9. Flush

`f0_flush_data` invalidates the micro-cache. `neigh_shared_icache` forms it as the OR of the flush requests of the eight Minions, registered once, and drives it to both micro-caches and to the L1 instruction cache. In the flush cycle the valid bits of all micro-tag entries are cleared, and no lookup hits. A fill outstanding at the flush is written when it arrives but left invalid, because the flush is remembered until the miss handler returns to Ready. The flush does not affect the TLB or the bypass register; TLB entries are invalidated through `tlb_invalidate` (Section 3.4.5.5).

### 3.4.10. Bypass Mode

Bypass mode turns off instruction caching in the Neighborhood. The cores still fetch their instructions, but neither the L0 micro-cache nor the L1 instruction cache keeps a line, so every new line comes from the Shire Cache, the shared L2 and L3 cache of the Minion Shire. Bypass mode is a chicken bit, a switch that disables a feature in silicon. It is set by the `bypass_icache` bit of the Neighborhood chicken ESR, which reaches the shared instruction cache as `esr_bypass_icache`.

#### 3.4.10.1. Request Path in Bypass Mode

The Front End is not connected to the Shire Cache directly. Its fetch port connects to the L0 micro-cache, the L0 micro-cache connects to the L1 instruction cache, and only the L1 instruction cache connects to the Shire Cache. A bypassed request therefore still passes through both caches, and the design must respect two rules of that path:

- **Physical addresses.** The fetch address is virtual, but the L1 instruction cache and the Shire Cache take physical addresses.
- **Fixed latency.** The Front End expects every response a fixed four cycles after the grant, for a hit and for a miss alike, and recognizes its own response by that timing alone (Section 2.2.4.2). A hit that is served from the Shire Cache takes many more cycles than that.

A core request first enters the L0 micro-cache, because the fetch port of the core is connected to it. The micro-cache pipeline already solves the translation: the instruction TLB translates the address in F1, and the PMA unit checks it in F2, before any request leaves for the L1 instruction cache. In F2 the micro-cache also sees that the request is bypassed. It skips the micro-tag array and, if the miss handler is in Ready, the miss handler registers the line request (Section 3.4.8.1). The request is sent to the L1 instruction cache in the next cycle, in F3.

The L1 instruction cache also sees the bypass bit. It treats every request as a miss and sends it to the Shire Cache. The line returns through its data RAM and reaches the micro-cache with its ECC and bus-error flags, as for any line fill. The L1 instruction cache writes no tag and does not update its LRU state, so it keeps no copy of the line.

#### 3.4.10.2. Bypass Register

The line request leaves the micro-cache in F3, and the core expects its response in the next cycle, in F4, as a hit or a miss. In one cycle the request cannot reach the L1 instruction cache, look up the Shire Cache and come back to the micro-cache, so the core always receives a miss. The line then arrives after a number of cycles that is not fixed, and by that time the thread sleeps. The micro-cache must hold the line until the thread retries, but its arrays must not keep it.

To hold the line, the micro-cache adds the bypass register, a small cache of one line built from flip-flops. A bypassed fetch of a new line then runs as follows:

1. The core request is translated and checked, and in F3 the miss handler sends the line request to the L1 instruction cache.
2. The response in F4 is a miss, and the thread sleeps.
3. When the line arrives from the L1 instruction cache, the micro-cache loads the line, its error flags and the tag that the miss handler recorded into the bypass register. It then pulses fill done,

as for a normal line fill, and the thread wakes.

4. The thread retries. Its request hits in the bypass register in F2, and the response in F4 carries the instruction block.

The next request, for the other 32-byte half of the same line, also hits in the bypass register. A request for a different line misses: the thread sleeps, the bypass register is loaded with the new line, fill done wakes the thread, and its retry hits. Figure 48 shows the complete flow.

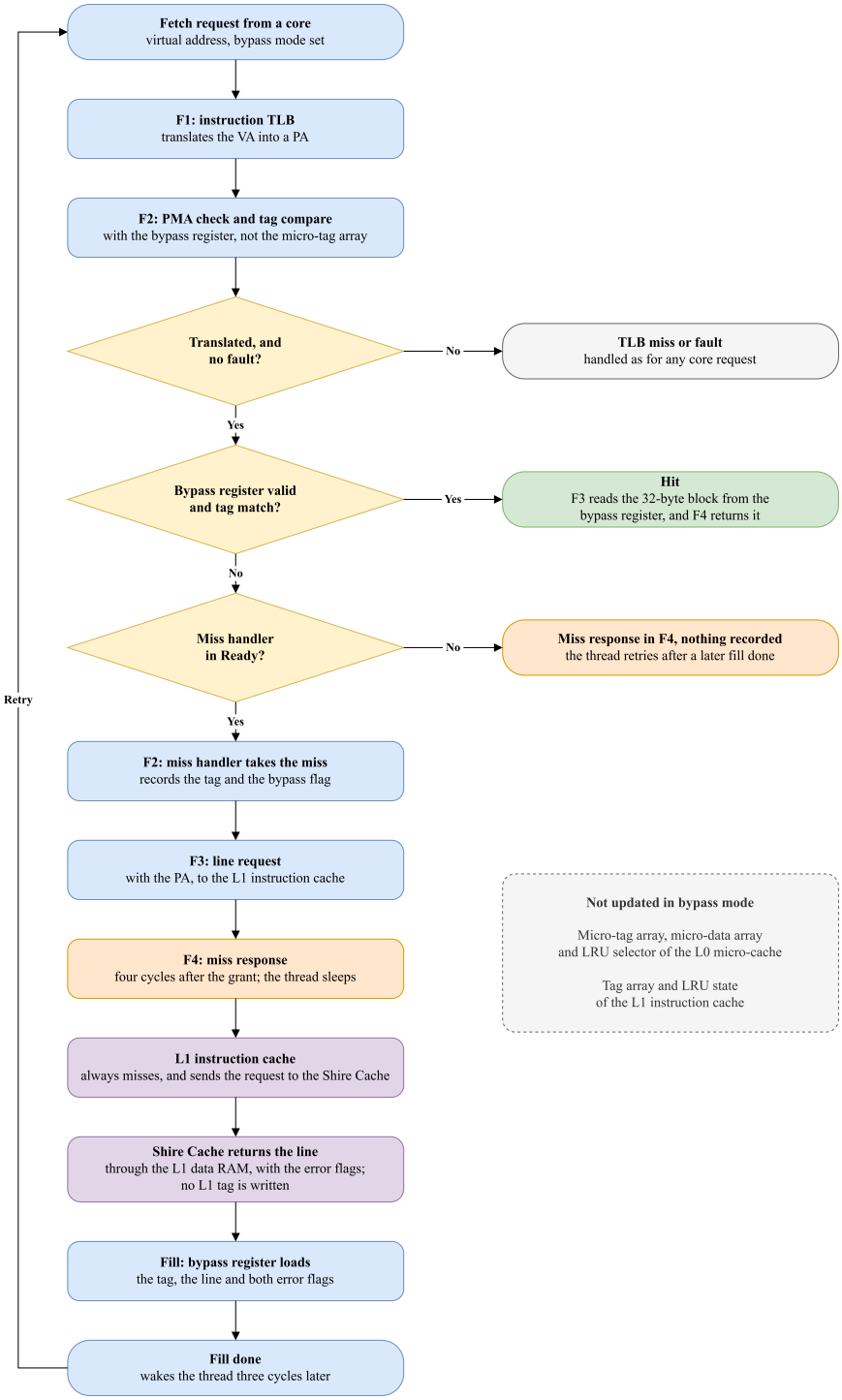



*Figure 48. Bypassed fetch flow*

Figure 49 shows the format of the bypass register: the tag, the 512-bit line, the two error flags and a valid bit.

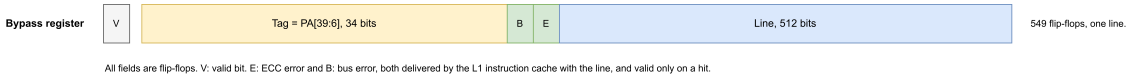



*Figure 49. Bypass register*

The bypass register is used as follows:

- **Lookup.** In F2 a bypassed request compares its tag with the bypass register instead of the micro-tag array. The register does not hit in the cycle in which it is loaded.
- **Fill.** The miss handler records the bypass flag with the miss, so the fill loads the bypass register instead of the arrays. The micro-tag array, the micro-data array and the LRU selector are not updated (Table 54).
- **Read.** In F3 the 32-byte half and the error flags come from the bypass register.
- **Invalidation.** The register is invalidated when a request that is not bypassed passes F2.

Bypass mode has two further effects:

- **Mode changes.** A change of bypass mode flushes neither cache level. The micro-tag and micro-data arrays keep the lines that they held before bypass mode, and those lines hit again when it ends. If the code in memory changed meanwhile, software must flush the instruction caches (Section 3.4.9).
- **Prefetch.** A prefetch request in bypass mode loads nothing, because the L1 instruction cache keeps no line and the bypass register takes only the lines of core misses. The code prefetcher is therefore of no use in bypass mode.

#### 3.4.10.3. Bypassed Transactions

Figure 50 shows a bypassed core request that hits in the bypass register, and Figure 51 one that misses. Table 53 lists the steps of both side by side. In both, the result of the micro-tag array is not used, the micro-data array is not read, and the LRU state does not change.

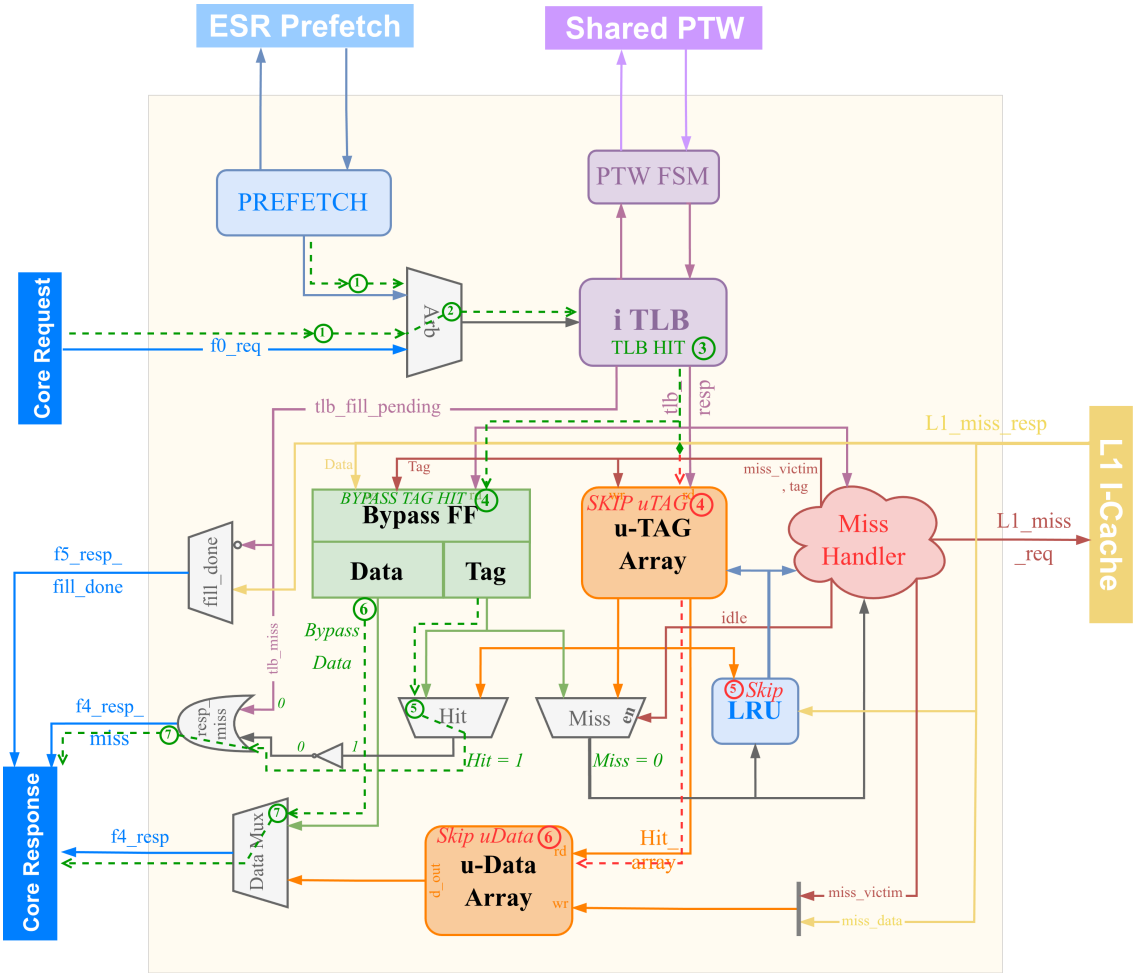



*Figure 50. Bypassed core request hit*

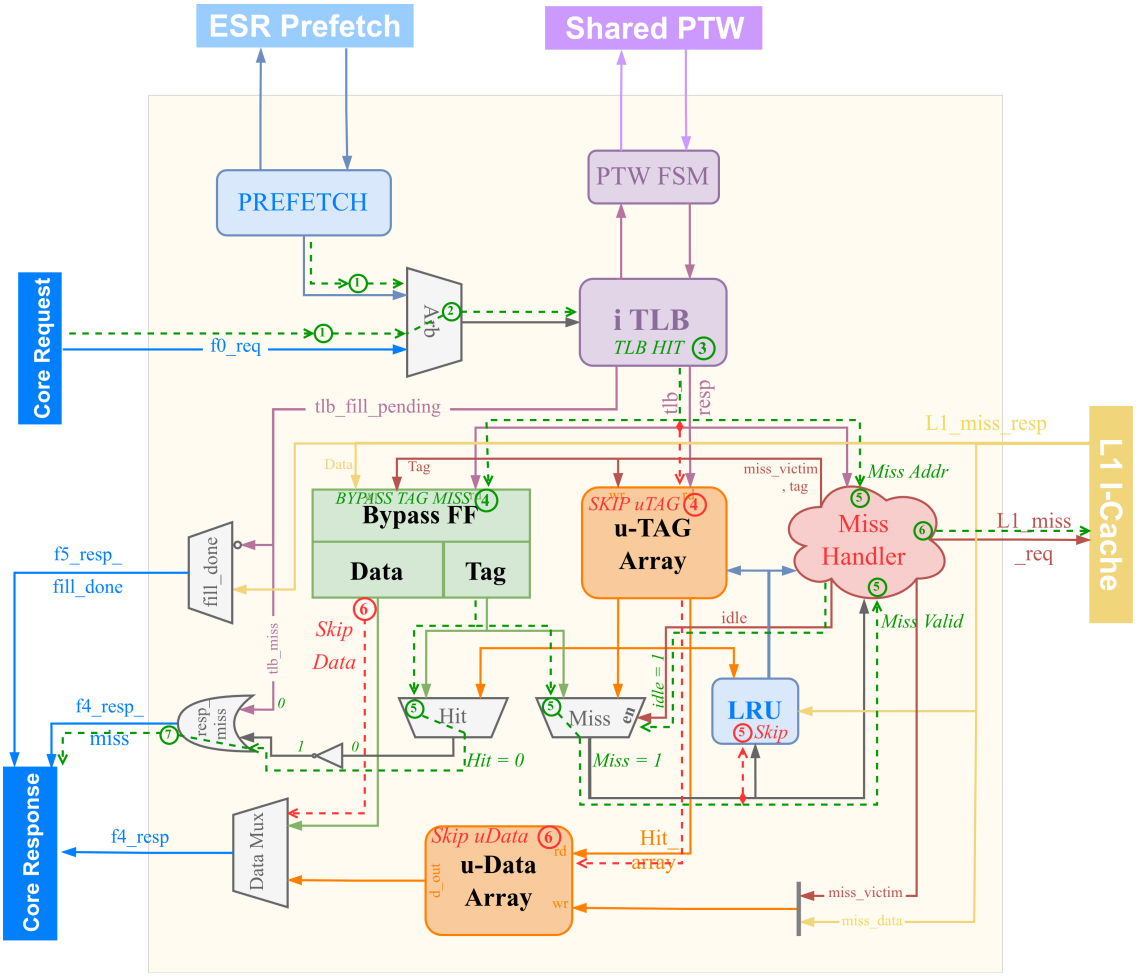



*Figure 51. Bypassed core request miss*

*Table 53. Bypassed core request, steps by cycle*

| Cycle | Step | Bypass register hit (Figure 50) | Bypass register miss (Figure 51) |
|---|---|---|---|
| 0–1 (F0–F1) | 1–3 | The arbiter grants the core request, and the instruction TLB translates its address. | |
| 2 (F2) | 4 | The physical tag matches the bypass tag. | The physical tag does not match the bypass tag. |
| 2 (F2) | 5 | Hit is 1 and miss is 0. | Hit is 0 and, with the miss handler in Ready, miss is 1. The miss handler records the tag and the bypass flag, and no victim. |
| 3 (F3) | 6 | The bypass register supplies the 32-byte half and the error flags. | The miss handler, in Request, drives the line request to the L1 instruction cache. |
| 4 (F4) | 7 | The response carries the instruction block, with the miss bit clear. | The response is a miss. The thread sleeps until fill done. |

Figure 52 shows the fill of the bypassed miss, and Table 54 lists its steps. The line goes into the bypass register, and the thread then retries and hits there.

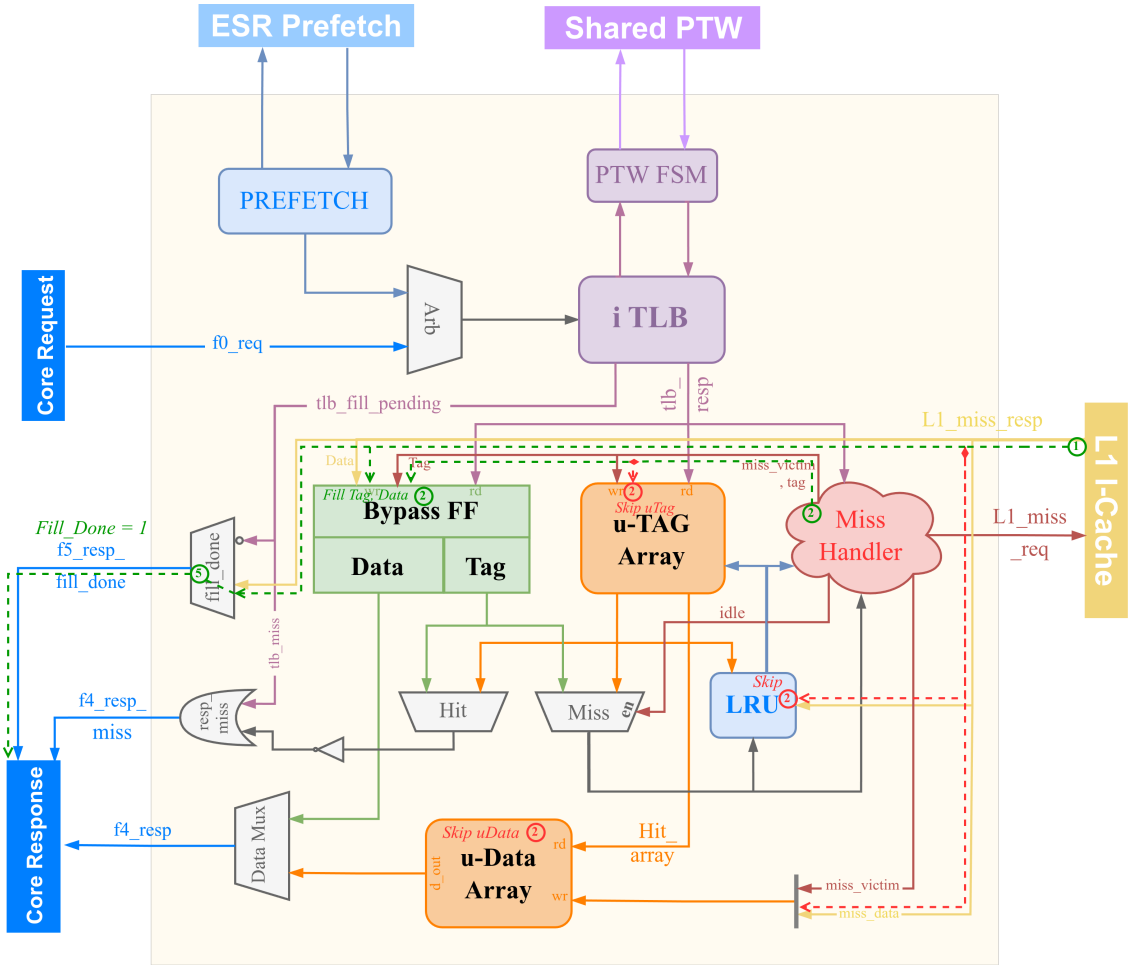



*Figure 52. Bypassed line fill*

*Table 54. Bypassed line fill, steps by cycle*

| Cycle | Step | Block | Action |
|---|---|---|---|
| *N* | 1 | L1 instruction cache | Asserts `f0_l1_miss_resp_valid` and returns the line and its two error flags. |
| *N* | 2 | Bypass register, miss handler | The bypass register loads the line data and its two error flags from the L1 instruction cache response, and the tag recorded by the miss handler, and becomes valid. The arrays are not written, and the LRU state does not change. The miss handler returns to Ready, and fill done is generated. |
| *N* + 3 | 5 | Fill done | `f5_resp_fill_done` wakes the sleeping threads. |

## 3.4.11. Code Prefetcher

The code prefetcher is a service that software uses to preload critical code into the L1 instruction cache before the cores execute it. Without it, the first fetch of each new line misses in both cache levels, and the thread sleeps until the line comes from the Shire Cache. A prefetch operation gives a start virtual address, a number of lines and a privilege level, and the prefetcher loads that many consecutive lines. A single operation prefetches at most 64 lines into the L1 instruction cache. Software writes the number of lines minus one, so the maximum is configured as 63 (Table 34).

The prefetcher is built into the north micro-cache, which serves Minions 0 to 3; the south micro-cache has no prefetcher (Section 3.4.11.1). The prefetcher and the cores work in parallel. The Minions do not stop during an operation: they continue to fetch, and their requests always have priority. The prefetcher uses only the cycles in which no core request enters the micro-cache, and it holds back while a core waits for the miss handler (unattended-miss state machine).

### 3.4.11.1. Prefetch Path

To load a line into the L1 instruction cache, a request for that line is enough. The L1 instruction cache answers every request with the line: on a hit it reads the line from its data RAM, and on a miss it fetches the line from the Shire Cache, stores it and returns it. Whether the request hits or misses, the line is in the L1 instruction cache when the response returns, which is all that a prefetch needs. A prefetch operation is therefore a sequence of line requests, one for each consecutive line.

Two facts decide where these requests come from. The L1 instruction cache has one request port, shared by the two micro-caches through a round-robin arbiter (Section 3.5.5), and a prefetcher with a physical address could simply be a third input of that arbiter. But the operation gives a virtual address. For its translation, the prefetcher can use the instruction TLB of the micro-cache, as a bypassed request does (Section 3.4.10.1). The prefetcher is therefore built into a micro-cache, and sends its requests through the micro-cache pipeline beside the core requests. One requester is enough, so only the north micro-cache has it.

Figure 53 shows the prefetcher in F0. The prefetch address and line-count registers load the operation from `esr_prefetch_conf` on `esr_prefetch_start`. For each new line, the address moves to the next line, 64 bytes higher, and the line count decreases by one. The prefetch state machine presents its request with `f0_prefetch_req_valid` only when the request is qualified (unattended-miss state machine). The 2:1 arbiter always selects the core request, so the prefetch request passes only in a cycle with no core request, with Minion identifier 0. `f1_is_prefetch` marks the prefetch request for the later stages.

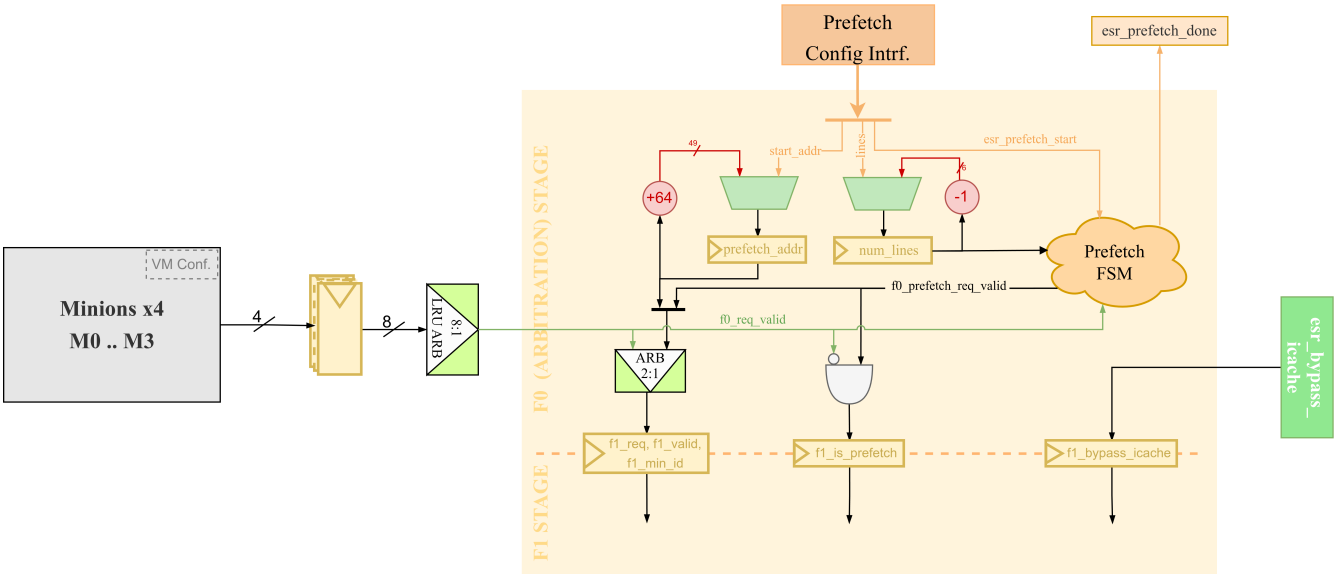



*Figure 53. Code prefetcher in the F0 stage*

A prefetch request passes the pipeline as follows:

- **F0.** The pipeline arbitrates between the core request and the prefetch request, and always gives

priority to the core request (Figure 53). A prefetch request enters only in a cycle with no core request, so it never delays a fetch.

- **F1.** The instruction TLB translates the address. A prefetch request carries Minion identifier 0, so its translation uses the translation configuration of Minion 0, with the privilege of the operation. On a TLB miss a page-table walk starts as for a core request.
- **F2.** The PMA unit checks the physical address. The request is forced to miss, and the result of the micro-tag array is not used, because the purpose of the request is a line request to the L1 instruction cache. If the miss handler is in Ready, it takes the request and records that it is a prefetch.
- **F3.** The miss handler sends the line request to the L1 instruction cache.
- **F4.** No response is sent, because no thread waits for the line.

The L1 instruction cache handles the line request with its normal behavior, and returns the line. The miss handler sees that the request was a prefetch and discards the line: the micro-tag array, the micro-data array and the LRU selector are not updated. The line stays only in the L1 instruction cache, where both micro-caches can use it. The fill still generates fill done to the Minions, as every line fill does.

#### 3.4.11.2. Starting an Operation

Software starts an operation by writing the start virtual address and the number of lines minus one to one of three Shire ESRs: `icache_uprefetch`, `icache_sprefetch` or `icache_mprefetch`. The ESR written sets the privilege of the operation. A write is accepted only in cooperative mode. Pending requests are served in privilege order: an M-mode request at once, and an S-mode or U-mode request only when no operation is in progress or the current operation has the same or lower privilege. The served request reaches the north micro-cache as `esr_prefetch_conf` (Table 34), with a pulse on `esr_prefetch_start` that starts the operation.

Figure 54 shows the format of the prefetch ESR that software writes, and of the configuration that the north micro-cache receives.

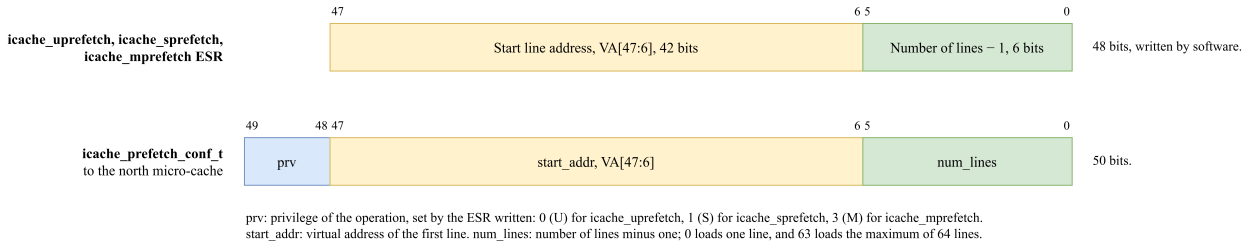



*Figure 54. Prefetch configuration*

#### 3.4.11.3. Prefetch State Machines

The prefetcher uses two state machines, shown in Figure 55:

- **Prefetch state machine.** Requests the lines of the operation one by one, and waits for each line before it requests the next.
- **Unattended-miss state machine.** Keeps the prefetcher from reducing the performance of the

Minions. The miss handler holds one line request at a time, and a core miss that finds it busy is not recorded: the thread sleeps and retries after the next fill done. This state machine notes that such a thread waits for the miss handler, and holds the prefetcher back until the thread retries.

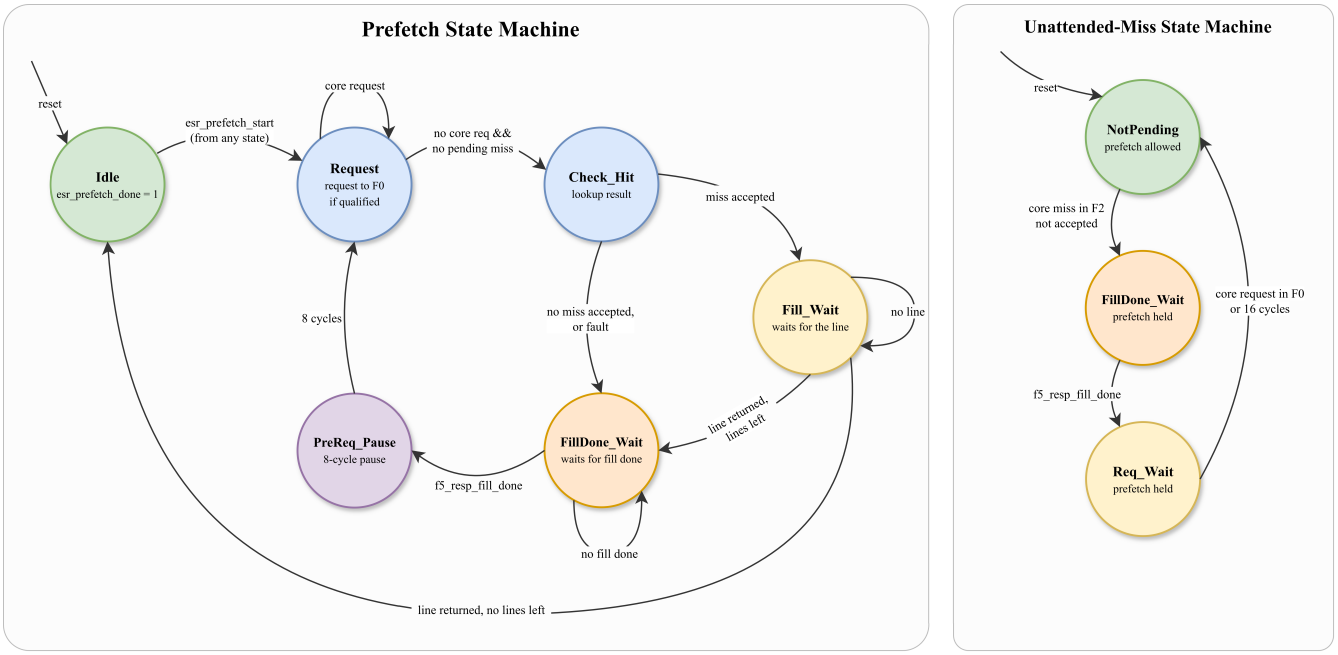



*Figure 55. Code prefetcher state machines*

The prefetch state machine has six states:

- **Idle.** No operation is in progress, and `esr_prefetch_done` is 1. A pulse on `esr_prefetch_start` loads the start address and the line count, and moves the state machine to Request. The pulse has this effect in every state, so a new operation cancels the current one and starts at once.
- **Request.** Clears `esr_prefetch_done` and presents the prefetch request to F0 when it is qualified. The state machine moves to Check_Hit when the request enters F0, in a cycle with no core request.
- **Check_Hit.** Waits for the request to reach F2. If the miss handler takes it, the state machine moves to Fill_Wait. If the request takes a page fault or an access fault, the line is skipped, and the state machine moves to the next line and to FillDone_Wait. If neither, because the miss handler is busy or the TLB missed, it moves to FillDone_Wait and requests the same line again later.
- **Fill_Wait.** Waits for the line from the L1 instruction cache. After the last line it moves to Idle and sets `esr_prefetch_done`; otherwise it moves to the next line and to FillDone_Wait.
- **FillDone_Wait.** Waits for `f5_resp_fill_done`. Fill done wakes the threads that sleep on a miss, and it also marks the end of the fill for which a busy miss handler or a TLB walk was waiting.
- **PreReq_Pause.** Waits eight cycles and then returns to Request. The north micro-cache serves eight threads, two in each of its four Minions, and the pause gives all eight threads the time to wake up and retry after fill done, before the next prefetch request.

The unattended-miss state machine has three states:

- **NotPending.** No core miss waits for the miss handler. A core request that misses in F2 while the miss handler is busy is not recorded, and moves the state machine to FillDone_Wait.

- **FillDone_Wait.** The thread of that core request sleeps. The state machine waits for `f5_resp_fill_done`, which wakes the thread, and then moves to Req_Wait.
- **Req_Wait.** The thread is awake, and its retry is expected. The state machine returns to NotPending when a core request enters F0, so that a new unattended miss can be noted. If no core request comes, it returns to NotPending after 16 cycles, so that the prefetcher is not held back for good.

A prefetch request is qualified when both state machines allow it. The prefetch state machine is in Request, so an operation is in progress and waits to request its next line, and the unattended-miss state machine is in NotPending, so no thread waits for the miss handler after a miss that was not recorded. In the first cycle in which no core request uses F0, the prefetcher then sends the request for its line into the pipeline.

#### 3.4.11.4. Prefetch Transactions

Figure 56 shows one prefetch request, and Table 55 lists its steps. The request is forced to miss, so it always takes the miss handler if the handler is in Ready, and it produces no response. Figure 57 shows its fill.

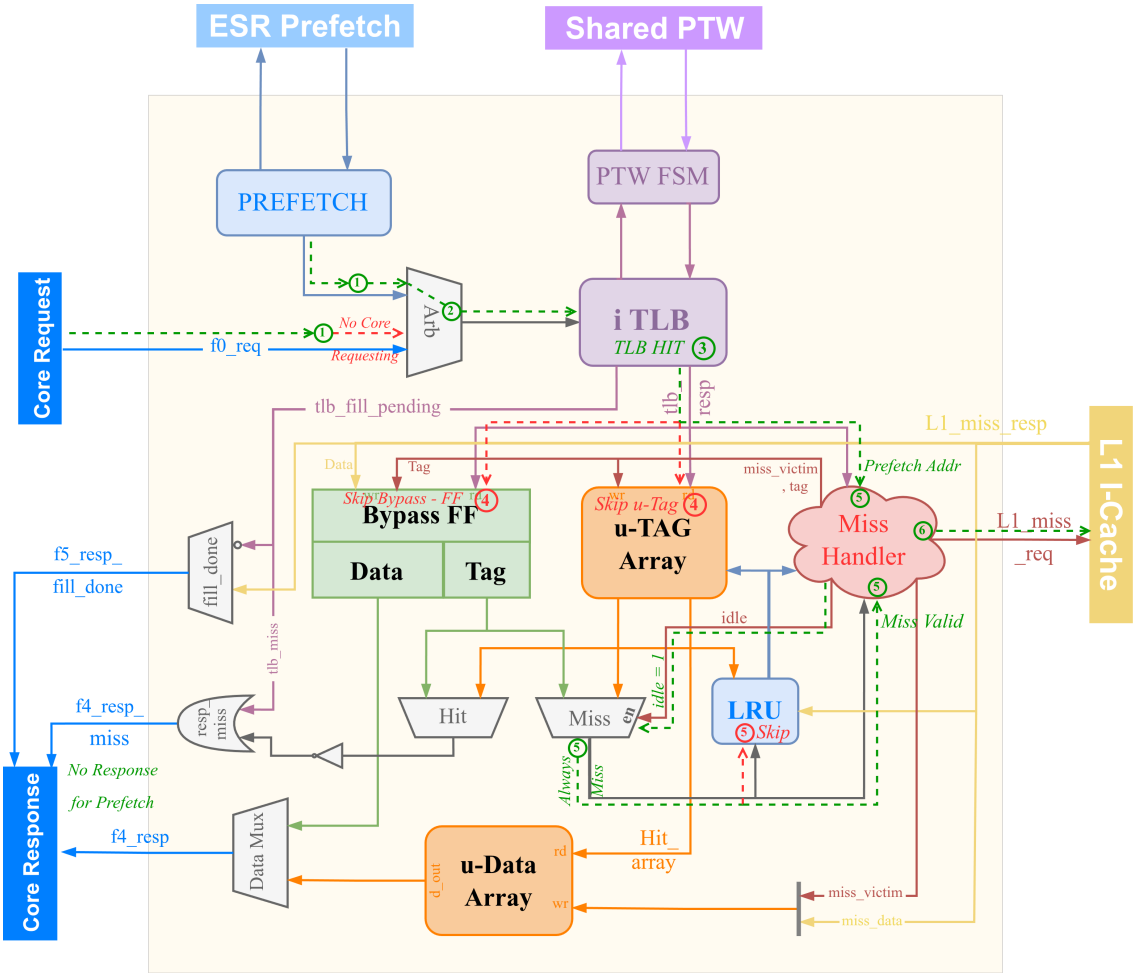



*Figure 56. Prefetch request*

*Table 55. Prefetch request, steps by cycle*

| Cycle | Step | Block | Action |
|---|---|---|---|
| 0 (F0) | 1 | Code prefetcher | The prefetcher is in Request and its request is qualified, and no core request is present. |
| 0 (F0) | 2 | Arbiter | Grants the prefetch request. |
| 1 (F1) | 3 | Instruction TLB | Hits and returns the physical page number. |
| 2 (F2) | 4 | Micro-tag array, bypass register | Their results are not used, because a prefetch request is forced to miss. |
| 2 (F2) | 5 | Hit and miss logic, miss handler | Miss is 1 whenever the miss handler is in Ready. The miss handler records the physical address and the prefetch flag. No victim is cleared, and the LRU state does not change. |
| 3 (F3) | 6 | Miss handler | In Request, drives the line request to the L1 instruction cache. |
| 4 (F4) | – | Response | None. A prefetch request produces no response. |

Figure 57 shows the fill of a prefetch miss, and Table 56 lists its steps. The miss handler recorded the prefetch with L0 writes disabled, so nothing is written in the micro-cache and only the L1 instruction cache keeps the line. The fill still produces fill done, and fill done starts the pause of the prefetcher.

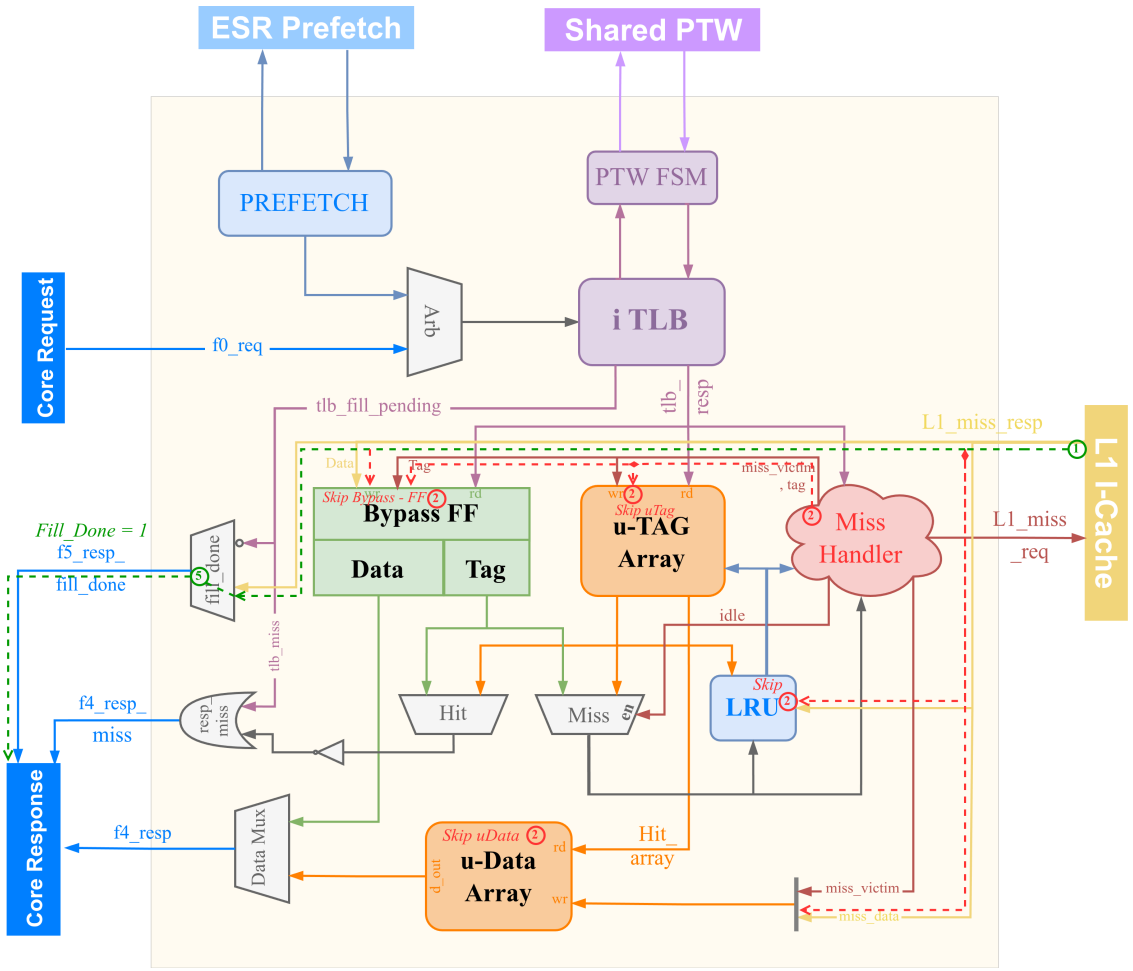



*Figure 57. Prefetch line fill*

*Table 56. Prefetch line fill, steps by cycle*

| Cycle | Step | Block | Action |
|---|---|---|---|
| *N* | 1 | L1 instruction cache | Asserts `f0_l1_miss_resp_valid` and returns the line and its two error flags. |
| *N* | 2 | Miss handler, code prefetcher | Nothing is written in the micro-cache, and the LRU state does not change. The miss handler returns to Ready, and fill done is generated. The code prefetcher leaves Fill_Wait for FillDone_Wait, or for Idle after the last line. |
| *N* + 3 | 5 | Fill done | `f5_resp_fill_done` wakes the sleeping threads, and the code prefetcher starts its eight-cycle pause. |

Figure 58 shows the first two lines of a two-line operation. The start pulse in cycle 0 moves the prefetcher to Request, and its request enters F0 in cycle 1, a cycle with no core request. The miss is accepted in F2 in cycle 3, and the L1 request is driven in cycle 4. The line returns in cycle 9 and the prefetcher advances to line A+1. Fill done in cycle 12 starts the eight-cycle pause, shown compressed by the second break. The request for line A+1 enters F0 in cycle 18.

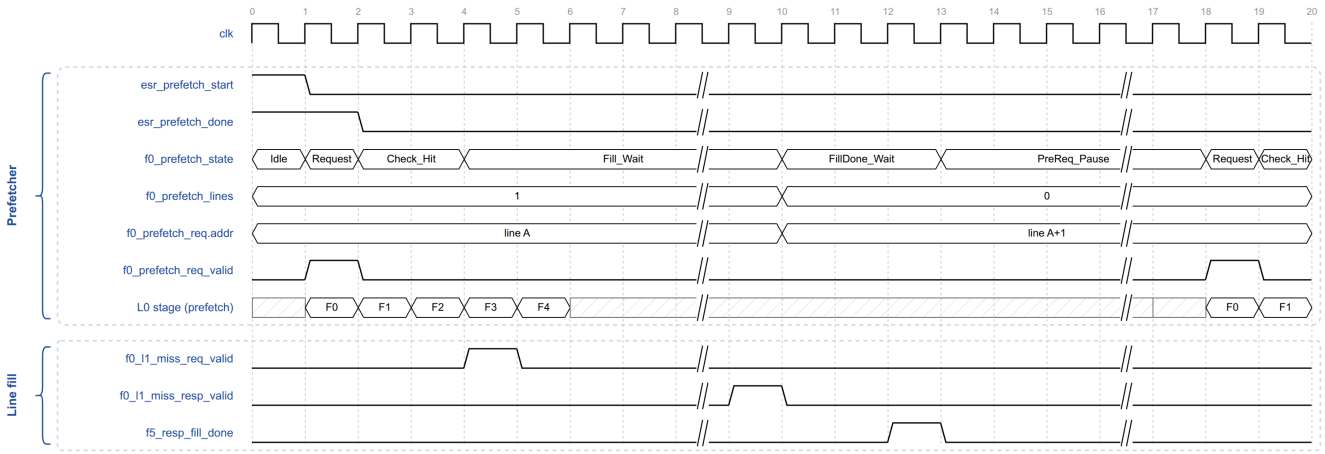



*Figure 58. Code prefetch sequence*

#### 3.4.11.5. Completion

`esr_prefetch_done` resets high, is set by a start pulse, is cleared when the prefetcher enters Request, and is set again when the fill of the last line returns. It is returned to the Shire ESR block.

## 3.4.12. Debug Access

The micro-tag and micro-data arrays of both instances can be read and written over APB. `icache_top` decodes the upper bits of its 10-bit APB address and passes the low eight bits to the selected instance (Table 57).

*Table 57. Micro-cache debug address*

| Bits | Use |
|---|---|
| 9 | 1: micro-cache space. 0 selects the L1 tag array. |

| Bits | Use |
|---|---|
| 8 | Instance: 0 for the north micro-cache, 1 for the south. |
| 7 | 0: micro-data array. 1: micro-tag array or error flags. |
| 6:3 | Line number, 0 to 15. |
| 2:0 | With bit 7 = 0: 64-bit word within the line. With bit 7 = 1: bit 2 selects the tag (0) or the error flags (1). |

- **Tag access:** a read returns the tag in bits 33:0 and the valid bit in bit 34; a write sets both from the same bits. A write waits for a cycle in which no fill writes the array.
- **Data access:** a read returns one 64-bit word of the line. A write is a read-modify-write of the whole 514-bit entry: the line is read, the selected word replaced, and the entry written back. Reads wait for cycles in which F3 does not read the array, and writes for cycles in which no fill writes it.
- **Error flags:** bit 0 is the ECC-error flag and bit 1 the bus-error flag of the line, read or written in the same way.

`apb_pslverr` is never asserted.

# 3.5. L1 Instruction Cache

## 3.5.1. Overview

*Figure: L1 instruction cache block diagram.*

## 3.5.2. Organization

## 3.5.3. Tag, LRU and Data Arrays

## 3.5.4. External Data RAM

## 3.5.5. Request Arbitration Between L0 Micro-Caches

## 3.5.6. Miss Path

## 3.5.7. Errors and ECC

# 3.6. Shared Page-Table Walker

## 3.6.1. Clients and Arbitration

## 3.6.2. Walk Sequence and Page Sizes

*Figure: Page-table walk state machine.*

### 3.6.3. PTE Cache and Invalidation

## 3.7. Memory Request and Response Path

### 3.7.1. Miss, Evict and Fill FIFOs

*Figure: Neighborhood memory request and response path.*

### 3.7.2. Request Arbitration

### 3.7.3. Response Routing and Backpressure

## 3.8. Tensor Support

### 3.8.1. Cooperative Tensor Load

### 3.8.2. Tensor Store Buffer

### 3.8.3. TBOX Router

## 3.9. Synchronization, Messaging, Registers and PMU

### 3.9.1. Fast Local Barrier

### 3.9.2. Local Message Network

### 3.9.3. IPI and FCC Paths

### 3.9.4. Neighborhood Registers and APB Access

### 3.9.5. Debug Channel

### 3.9.6. Performance Monitoring Unit